ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Systems@ETH Zürich

Bachelor's Thesis Nr. 435b

Systems Group, Department of Computer Science, ETH Zurich

Large-Scale, Out-of-Memory Approximate Nearest Neighbor Search Using An In-Memory Index

by

Timon Fercho

Supervised by

Prof. Gustavo Alonso, Wenqi Jiang

September 2022–March 2023

D INFK

Abstract

In this research project, we aim to build a fast distributed vector retrieval engine through algorithm innovation and solid system implementation which addresses the issues of poor load balancing and high I/O overhead of previous approaches. More specifically, we will design a system containing a root node and several storage nodes with search capabilities that abstract access to the lists of vectors similar to a distributed key-value store. Given a query vector, a root server first searches a graph-based index to select the nearest centroids of all partitions and queries only the storage servers holding the corresponding partitions. Using this more fine-grained index overall results in better load balancing, compared to previous approaches. Each selected storage server then searches for the nearest vectors among those they have locally stored on disk. In the final step, the root server gathers, ranks, and returns the resulting vectors to the user.

For the scope of this thesis, we implemented the system for the single-server case of the storage node and a scan-based root index. The storage node manages the on-disk partitions and enables efficient nearest neighbor search, making use of the high-performance optimizations SIMD and OpenMP. We extensively evaluated our system and showed that we can achieve search throughput speedups of 14x and 3-9x in latency over our system without SIMD and OpenMP. Although FAISS, a state-of-the-art system for similarity search, outperforms our system in terms of throughput by 5x, we in turn achieved latency improvements of 3-6x over their system.

Contents

Contents **iii**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Work Scope . . . 2
1.3 Outline . . . 3

2 Background **5**
2.1 Approximate Nearest Neighbor Search . . . 5
2.2 Indexing Algorithms . . . 7
2.2.1 HNSW . . . 7
2.2.2 Inverted File Index . . . 8
2.3 High-Performance Optimizations for the CPU . . . 9
2.3.1 SIMD . . . 9
2.3.2 OpenMP . . . 10

3 Design **13**
3.1 Goals . . . 13
3.2 Overview . . . 13
3.3 Index Construction . . . 15
3.4 Root Node . . . 15
3.5 Storage Node . . . 16
3.6 Work Scope . . . 17

4 Implementation **19**
4.1 Setup . . . 19
4.2 Storage Node . . . 21
4.2.1 Input Files . . . 21
4.2.2 Storage Lists . . . 22
4.2.3 Storage Index . . . 25

4.3 Root Node . . . 28
4.3.1 Root Index . . . 28
4.4 Index Construction . . . 29
4.5 Prototype . . . 29

5 Evaluation **31**
5.1 Setup . . . 31
5.1.1 Hardware . . . 31
5.1.2 Software . . . 31
5.1.3 Datasets . . . 31
5.2 Benchmarking . . . 32
5.2.1 Parameters . . . 32
5.2.2 Reproducibility . . . 33
5.2.3 Metrics . . . 34
5.3 Correctness . . . 36
5.4 Performance . . . 37
5.4.1 Optimization Goals . . . 37
5.4.2 Bulk Insertion . . . 38
5.4.3 Search . . . 43

6 Limitations and Future Work **71**
6.1 Limitations . . . 71
6.2 Future Work . . . 72

7 Conclusion **75**

8 Acknowledgements **77**

9 Appendix **79**
9.1 Project Structure . . . 79
9.2 System implementation . . . 80
9.3 Testing and benchmarking . . . 125
9.4 Evaluation . . . 177
9.5 Preprocessing . . . 196
9.6 Prototype . . . 210

Bibliography **215**

Chapter 1

Introduction

Vector similarity search is at the heart of search engines and recommender systems. A search engine, for example, encodes web pages as vectors by machine learning models and stores the vectors in a database. Search is then a two-step process including candidate generation and page re-ranking. First, the engine encodes the query as a vector and retrieves relevant contents in the database based on the similarity between the query vector and the vectors in the database. Second, the retrieved candidate pages are re-ranked by another machine learning model such as `BERT` [6] to select the top-K hits. The two-stage processing method also applies in recommender systems, except that the encoded contents are advertisements or products instead of web pages. Apart from search engines and recommender systems, vector similarity search is also used in image retrieval, video analysis and retrieval, language model training, audio fingerprinting and retrieval, and plagiarism detection.

1.1 Motivation

Previous approaches for large-scale vector similarity search have primarily focused on approximate nearest neighbor search in memory where the search is performed on a subset of the data and the index resides in memory. This has proven to achieve high-recall search at low latency but due to extreme memory demands, such in-memory solutions become infeasible for the billion-scale as in web search. As a result, there has been a recent trend towards hybrid solutions, such as `DiskANN` [9] and `HMANN` [19], which reduce the memory cost by storing the majority of the data on less expensive disks and only using a small and cost-efficient index in memory.

To further improve memory efficiency, researchers at Microsoft have proposed a hybrid approach called `SPANN` [3] which allows access to the uncom-

pressed vectors on disk through an in-memory inverted index (IVF) which holds the uncompressed centroids of the posting lists. To scale the system to the distributed setting, they propose to cluster the dataset into a few smaller partitions and distribute them across the available servers. This approach suffers from two main problems. The first of which is poor load-balancing, meaning the distribution of servers a query is dispatched to is skewed which results in hot-spot servers receiving a majority of the requests. Second, the percentage of vectors that need to be scanned across all servers is high since one server only holds a single partition of the data and queries are routed based on this coarse granularity, resulting in high I/O overhead.

In this research project, we aim to build a fast distributed vector retrieval engine through algorithm innovation and solid system implementation which addresses these issues. More specifically, we will design a system containing a root node and several storage nodes with search capabilities that abstract access to the lists of vectors similar to a distributed key-value store. Given a query vector, a root server first searches a graph-based index to select the nearest centroids of all partitions and queries only the storage servers holding the corresponding partitions. Using this more fine-grained index overall results in better load balancing, compared to previous approaches. Each selected storage server then searches for the nearest vectors among those they have locally stored on disk. In the final step, the root server gathers, ranks, and returns the resulting vectors to the user.

1.2 Work Scope

This thesis targets the system implementation for large-scale approximate nearest neighbor search. Specifically, this project focuses on out-of-memory approximate nearest neighbor search on a single machine. The system contains an in-memory index, in which each centroid is associated with the list of vectors stored on disk. Upon receiving a query, the system will first search the in-memory index and then probe the selected lists of on-disk vectors.

We will restrict our work to the following work items:

1. Implementation of a root node prototype based on a graph-based index.
2. Implementation of the storage node for the system on a single server, including:
   a) An out-of-memory inverted file index with search capabilities for the storage node.
   b) A scan-based, in-memory index resembling the root node.
3. Evaluation and optimization of the system on datasets of various sizes.

1.3 Outline

The thesis is structured as follows. In chapter 2, we will review the background of approximate nearest neighbor search, relevant indexing algorithms and high performance optimizations for the CPU. In chapter 3, we will describe the system design of our vector retrieval engine. Based on this, we dive deeper into the implementation in chapter 4. In chapter 5, we will evaluate the correctness and performance of the system. Lastly, in chapter 7, we will summarize our results and discuss limitations and potential future work. For the appendix in chapter 9, we have compiled the most relevant source listings for the prototype, preprocessing, system implementation and evaluation.

Chapter 2

Background

2.1 Approximate Nearest Neighbor Search

Applications Nearest neighbor search is a fundamental problem in data management, which has a wide range of applications areas such as information retrieval, recommender systems, computer vision, pattern recognition and natural language processing. Vector databases used for nearest neighbor search often contain vector embeddings which are dense data representations obtained by transforming unstructured data, such as images, video or text into a high-dimensional vector format using machine learning models, such as `ResNet` [7] or `BERT` [6].

Problem Definition Given a query vector, which was transformed in the same way as the vectors in the database, the goals of nearest neighbor search is to efficiently retrieve the most similar vectors to the query vector (2.1). The similarity is generally expressed as the distance between two vectors, computed using a distance metric over some vector space. Common distance metrics include euclidean distance (2.2) and cosine similarity (2.3). Distance computations for large and high-dimensional datasets are expensive which makes approximate nearest neighbor search an attractive alternative.

$$i = \underset{i \in [1,n]}{\operatorname{argmin}}\, d(\vec{q}, \vec{v}_i)$$

Figure 2.1: Nearest neighbor search expressed as an optimization problem over n vectors.

Approximate Nearest Neighbor Search Approximate nearest neighbor search aims to reduce the search space by finding a subset of vectors that is

$$d(\vec{a}, \vec{b}) = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}$$

Figure 2.2: Euclidean distance between two vectors of dimension n.

$$d(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

Figure 2.3: Cosine similarity between two vectors.

likely to be similar to the query vector. The search relies on an index data structure that is populated during the index construction phase. A common approach is to use the centroid vectors obtained during the partitioning of the data as representatives of each data subset. During query processing, the index is queried to determine which subsets should be searched. In the second step, only the subsets containing the vectors that are likely to be similar to the query vector are retrieved from the database for distance computation. As the name suggests, this approach is only an approximation as we might miss some nearest neighbors which are not contained within the preselected subsets. However, advancements in the field of approximate neighbor search were instrumental in enabling applications such as web search over datasets on the billion scale, achieving both high search quality and performance.

Metrics Typical metrics for evaluating the quality of the approximate nearest neighbor search are recall and precision. Recall measures the ability of the algorithm to find all the relevant items in the dataset (2.4), while precision measures the accuracy of the algorithm in retrieving only the relevant items. Approximate nearest search algorithms can be tuned based on parameters such as the number of lists the data should be partitioned in, the number of partitions to probe, the number of nearest neighbors to return or the type of distance metric to use. When optimizing these parameters there is generally a trade-off between performance and quality of the results. We will discuss the performance-related metrics in chapter 5.

$$\text{recall} = \frac{\text{true nearest neighbors retrieved}}{\text{total number of true nearest neighbors}}$$

Figure 2.4: Recall as a quality metric for nearest neighbor search.

2.2 Indexing Algorithms

In recent years, numerous algorithms for approximate nearest neighbor search have been proposed, including hash-based, tree-based and graph-based algorithms. In this section, we will discuss two of them which are most relevant for our work: HNSW and the inverted file index.

2.2.1 HNSW

Index Construction Hierarchical navigable small worlds (HNSW) [14] is a graph-based indexing algorithm that has recently shown a great rise in popularity due to its high performance and search quality, especially for high-dimensional data. The algorithm achieves this by constructing a hierarchical graph structure that shortens the path that needs to be traversed to find the nearest neighbors of a query vector. Similar to a skip list, a common data structure used for efficient traversal of lists, each level of the graph contains a subset of the points of the dataset where each point is connected to a small number of nearest neighbors of the same level and the level below, a so-called proximity graph. All points are inserted at the lowest level and are also added to the levels above with exponentially decreasing probability (2.6).

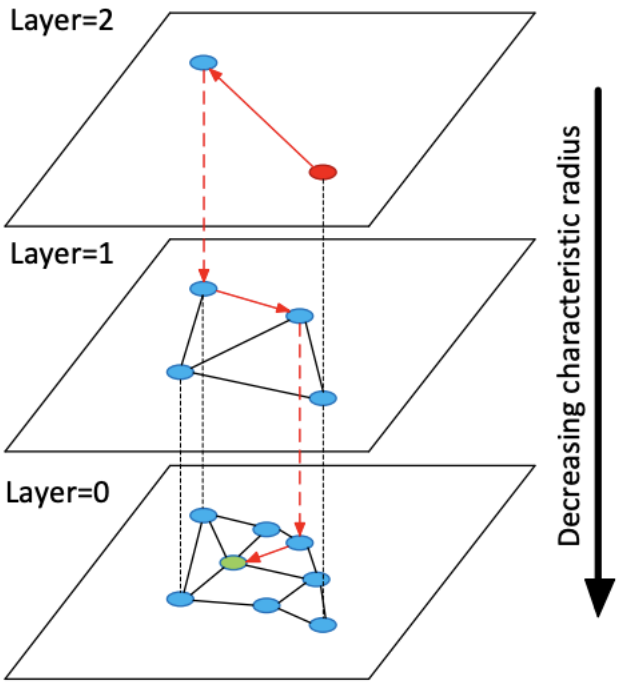


Figure 2.5: A HNSW graph during query processing [14]. The search starts from some node at the top layer and greedily traverses the graph to the nearest neighbors of the query at the bottom layer, reducing the search radius at each level thanks to the hierarchical structure.

Index Querying When the index is queried, the search starts at some root node at the top layer and iteratively traverses the graph to the nearest neighbors at the bottom layer in a greedy manner (2.5). At each level, the algorithm follows the graph edges until it has found a local minimum where the current node is closer to the query vector than all its neighbors. If no closer neighbor can be found, the search will continue on the next level below. `HNSW` terminates based on some stopping criteria, for example, the number of nodes it should explore. The sparse hierarchical structure and the fixed maximum degree, which is independent of the network size, reduce the number of hops and distance computations that need to be performed during the search, resulting in logarithmic runtime.

$$l = \lfloor -\ln(\text{uniform}(0,1)) \cdot m_L \rfloor$$

Figure 2.6: The insertion level of a point in the `HNSW` index with m_L levels is determined by a random variable with a geometric distribution, assigning a higher probability to the lower levels.

2.2.2 Inverted File Index

Applications The inverted index is a widely-used data structure in the field of information retrieval. It is commonly used to index large corpora of text and create a mapping between terms and the documents they occur in (2.7), in contrast to a forward index which maps documents their content. Other application areas are natural language processing as well as search over images, videos or spatial data.

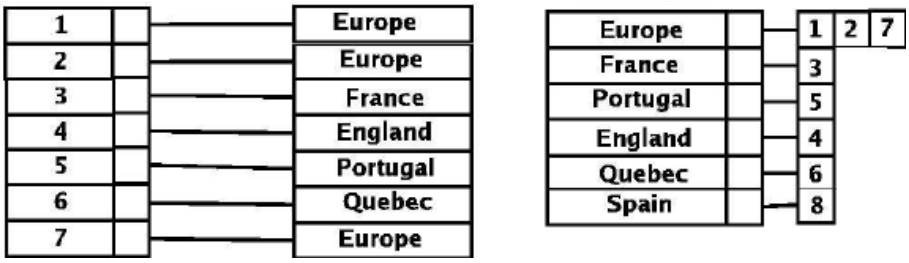


Figure 2.7: *Left:* A forward index associating a document with terms that appear in it [1]. *Right:* An inverted index associating terms with the documents they appear in.

Index Construction For the scope of this work, we are most interested in the inverted file index as a data structure for indexing high-dimensional vectors. The principal idea is the same in that we map a vector to its location in the search space. To achieve this, we first partition the data into disjoint sets of vectors, each with a representative centroid. These centroids are then inserted

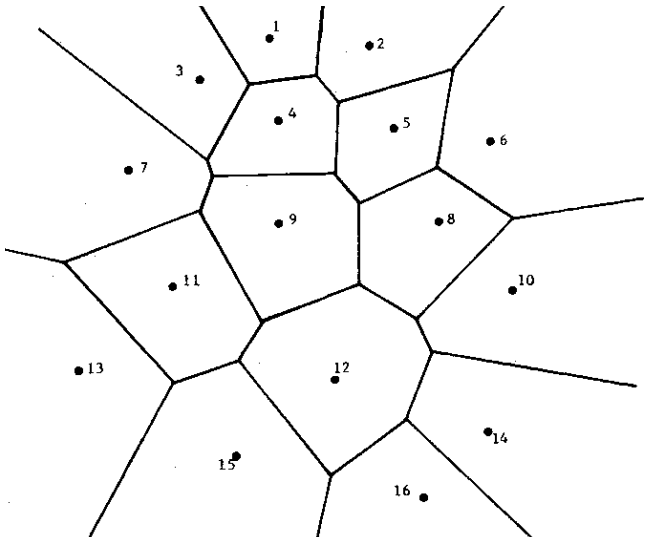


Figure 2.8: A Voronoi diagram for a set of 16 points in a two-dimensional space [12]. A Voronoi cell of a point is the region of space that is closer to that point than any other point.

in an inverted index structure which maps them to their corresponding partition. The concept can be seen as a generalization of dividing a two-dimensional space into Voronoi cells (2.8) where each cell is associated with a centroid. Depending on the use case and performance requirements, there is a plethora of variants to choose from including various partitioning strategies, auxiliary data structures, and compression techniques.

Index Querying When the index is queried, we can efficiently select a subset of the partitions by comparing the query vector with all centroids in the index. For each selected centroid, it searches all vectors within the associated list and combines the results. The number of lists to preselect and the number of nearest neighbors to return are configurable parameters that impact the performance and quality of the search as outlined in 2.1.

2.3 High-Performance Optimizations for the CPU

2.3.1 SIMD

Applications `SIMD`, or single instruction, multiple data, is a popular type of parallel processing architecture for the CPU that enables simultaneous execution of a single instruction on multiple data elements. `SIMD` exploits data-level parallelism, which can be most efficiently utilized by algorithms operating on vectors of data. Applications like image, video and signal processing can greatly benefit from `SIMD` acceleration. Coincidently, that is also the case for nearest neighbor search which heavily depends on efficient

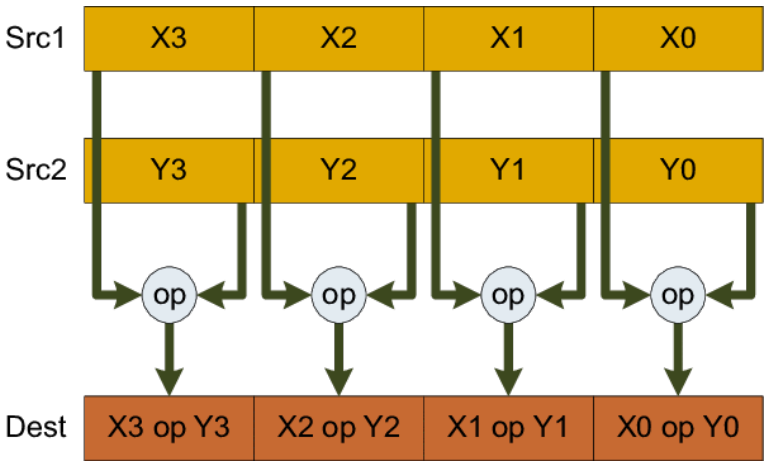


Figure 2.9: A SIMD instruction operating over four-element wide registers [16].

distance computations. As the squared differences between the corresponding components of two vectors can be computed independently (2.2), the computation of euclidean distances can be parallelized by performing the same operation, in this case, subtraction and multiplication, on multiple vector components at once.

Parallel Processing Architecture Depending on the architecture of the processor, it may support different flavors of SIMD. Extensions most modern systems support are SSE and AVX which differ primarily in the types of operations and the register widths they support. For instance, while AVX allows performing eight 32-bit floating point operations in parallel, SE can only execute four instructions at once which is illustrated in figure 2.9. Although SIMD instructions have significant overheads in power, and circuit area and often require manual intervention during development, they offer great performance benefits, efficiently utilizing the available computational resources.

2.3.2 OpenMP

Shared-Memory Parallelism OpenMP [4], meaning Open Multi-Processing, is a standard for shared-memory parallel programming. It provides the user with an accessible interface to exploit parallel computing resources available on the CPU. In contrast to a distributed-memory model like MPI [20] where different processes located within separate address spaces communicate by passing messages between them, OpenMP coordinates the execution of threads on a single, shared memory space. Based on an initial scalar program, the developer can iteratively extend the code with OpenMP directives which signal the compiler how the program should be parallelized. OpenMP is designed to

be portable across many different architectures and operating systems. The standard is supported by most compilers and is widely used in the scientific community.

Worksharing Constructs `OpenMP` supports a variety of parallelization constructs that can be used to express the parallelism in a program. They are defined using directives of the form `#pragma omp [construct] [parameters]` which are inserted into the source code. All constructs must be enclosed by a parallel region which is defined using the `omp parallel` directive. At this point of the program, the main thread spawns the parallel region and creates a team of threads, depending on the hardware configuration and the number of threads specified by the user. One way to define the maximum number of threads to spawn is by defining the `OMP_NUM_THREADS` environment variable which is supplied at runtime. Two common constructs that can be used to direct which parts of the program the threads should be executed in parallel are `omp for` and `omp task`. The former is used to distribute iterations of a loop among the threads in the team, while the latter is used to create units of work that can be executed in a more fine-grained and dynamic manner. The `omp for` construct is illustrated in figure 2.10. `OpenMP` also supports ways to combine the work of the different threads by specifying the `reduction` parameter after the construct. For our purpose, another parameter is of particular interest, the `schedule` parameter [18]. It allows the user to specify how the iterations of a loop are distributed among the threads. The default scheduling policy depends on the implementation and can be chosen among the following:

- `static`: The iterations are divided into chunks of equal size and statically assigned to the threads in a round-robin fashion. The default chunk size is one.
- `dynamic`: The iterations are divided into chunks of equal size and are assigned to the threads on demand as soon as they have finished their previous chunk. The default chunk size is one.
- `guided`: The iterations are divided into chunks of varying size and are assigned to the threads on demand as soon as they have finished their previous chunk. The chunk sizes are reduced exponentially down to the user-specified minimum chunk size. The default minimum chunk size is one.
- `runtime`: The schedule can be specified at runtime by the `OMP_SCHEDULE` environment variable.

Limitations and Ways to Overcome Them Introducing parallelism into a program can be challenging. `OpenMP` offers ways to deal with some of the

```
#pragma omp parallel for
for (i = 0; i < n; i++) {
  a[i] = b[i] + c[i];
}
```

Figure 2.10: A `OpenMP` `for` construct [4], which is used to distribute iterations of a loop among the available threads. `omp parallel for` is a shorthand for `omp parallel` and `omp for`.

potential issues. Since threads share the same memory area, this might lead to data races on the resources they share. A common way to deal with this is to introduce synchronization, meaning allowing only a single thread to execute critical sections of the program. This can be achieved in `OpenMP` either by only allowing one thread to execute the region using the `omp single` directive or allowing all threads to execute it sequentially by enforcing mutual exclusion using the `omp critical` directive. However, this comes at a performance penalty as it introduces sequential sections into the parallel program which limits the maximum theoretical speedup of a program (2.11). One option to reduce the need for critical sections is by allowing each thread to work on its copy of the resource which is facilitated by `OpenMP`'s management of data scopes. By default, variables defined inside a parallel region are private and must be explicitly declared as shared to allow other threads access. Other problems that can arise are deadlocks, where two or more threads block each other by trying to access an exclusive resource that another thread owns. This can be avoided by carefully ordering the execution of the threads, for example, using the `omp ordered` directive. Finally, there are performance-related issues that are more difficult to deal such as false sharing, which can occur when two threads access the same cache line and invalidate each other's data. Although alternative, distributed-memory frameworks such as `MPI` may provide more flexibility, `OpenMP` provides a convenient way to exploit the great potential of parallel shared-memory computing.

$$\texttt{Speedup}(f, S) = \frac{1}{(1-f) + \frac{f}{S}}$$

Figure 2.11: Amdahl's law [5]. The speedup of a program accelerated by a factor S is limited by the fraction of the program that cannot be parallelized $(1 - f)$.

Chapter 3

Design

In this chapter, we will describe the design of our system. We will start with an overview of the distributed system, then take a closer look at the specific details of each component and finally limit our discussion to the scope of this thesis.

3.1 Goals

The goal of our system is to provide a scalable and efficient solution for approximate nearest neighbor search in a billion-scale, out-of-memory setting. Further, we target exact search, meaning the order of query results and respective distances should be accurate. We aim to minimize the required I/O operations for a given query and maximize the throughput under some latency bound. We will evaluate the trade-offs between these goals and how they have influenced our design decisions in chapters 4 and 5.

3.2 Overview

Let us first give a high-level overview of the distributed system. Since this thesis is only concerned with the single server case, we will leave the remaining details for the continuation of this work.

Index Construction The system consists of two main components: the root node and several storage nodes. During index construction, the set of vectors is clustered into lists of vectors. The partitioning results in a set of tuples each with a list id, the respective list of vectors that have been assigned to that cluster as well as a centroid vector that is representative of the vector partition. The lists of vectors together with the list ids are then distributed across the available storage nodes so that each storage node stores a subset of the lists on its local disk. To improve reliability, each list is replicated across

multiple storage nodes. We query the storage nodes through a distributed key-value store-like interface based on gRPC where the root node sends the query vector and list ids and the storage nodes return the top `k` results. This design choice allows for easier scaling of the system since storage nodes are independent of each other and more can be added to the system on demand. The centroids we obtained from the clustering are inserted into a graph-based index at the root node where each centroid is associated with the set of storage nodes that hold the respective partition.

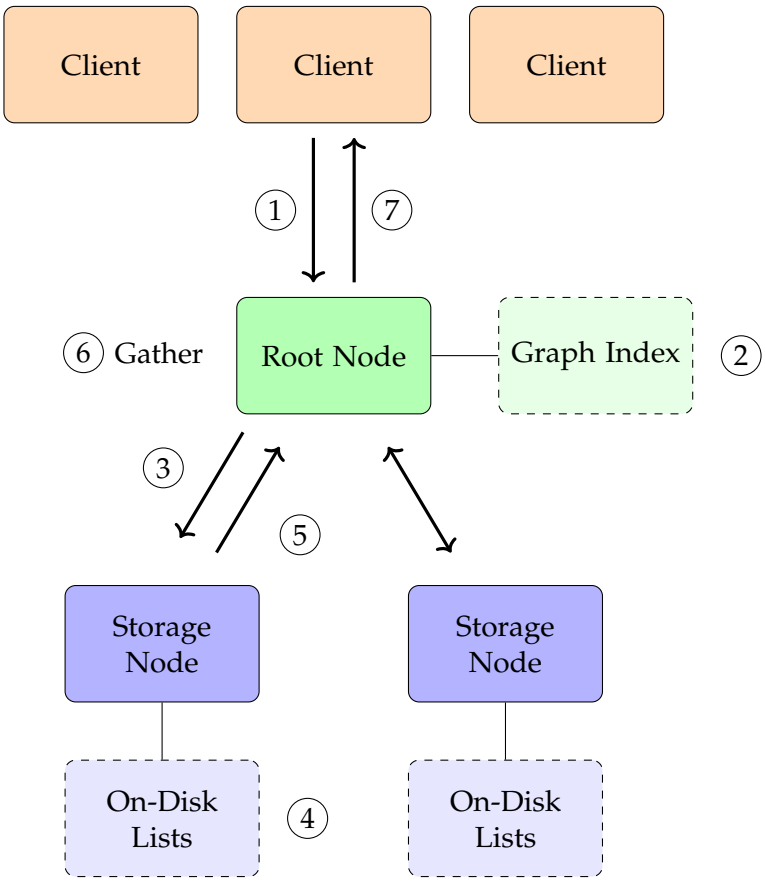


Figure 3.1: Overview of the distributed system during query processing. In step 6, the root node gathers and reranks the results from the storage nodes.

Query Processing During query processing (3.1), when a client sends a query to the root node, the root node first searches through the graph-based index to find the closest centroids to the query vector. For each centroid, we retrieve the associated list of storage nodes holding the respective partition. The root node then issues a search request only to these selected storage nodes. Given the query vector and the set of lists to probe, all selected storage

nodes then search in parallel through the respective lists of vectors and return the nearest neighbors along with their respective distances back to the root node. The root node then merges the results from the storage nodes, reranks them and returns the top `k` nearest neighbors to the client.

3.3 Index Construction

Good index performance, meaning high recall at low latency, is not only dependent on the right choice of the indexing algorithm but also on effective index training. Since our approach relies on the clustering of the dataset in lists of vectors, we will go into more detail about how this partitioning is performed.

Clustering Approaches There have been two main approaches to the partitioning of the datasets. The first approach is to randomly partition the vectors into a fixed number of partitions. The second approach is to cluster the vectors into some number of partitions based on similarity which generally leads to better index performance. As we have motivated in 1.1, we aim to address the issues of poor load balancing and high I/O operations across the system by clustering the vectors into more fine-grained partitions. Common clustering approaches in our setting are k-means clustering and hierarchical clustering. The advantage of hierarchical clustering is that it does not require prior knowledge of the data distribution and allows for fine-tuning the number of clusters after training at the cost of higher runtime. As the number of clusters directly affects the length of the lists and therefore also influences the search performance, hierarchical clustering lends itself well to our use case. As the topic of efficient clustering alone is already a non-trivial problem, we are not going to focus on this aspect of the system in this thesis.

Distribution of Partitions The partitions we obtain from the clustering are then shuffled randomly across all nodes to deal with skews within the data distribution and avoid potential hotspots at a subset of the storage nodes. Since the partitions are chosen to be small, the probability of frequently accessed lists being collocated on the same node is low, alleviating the load-balancing issues of previous approaches. We will discuss in the following section how the root node selects the lists to probe based on the clustering results.

3.4 Root Node

Responsibilities When a query sent by a client enters the system, the root node is responsible for selecting the vector lists to be searched among the

entire dataset of lists that are distributed across the storage nodes. The centroids obtained from the clustering will be used to identify a subset of lists to search on the storage nodes. Based on the results of the search, the root node then distributes the query along with the list ids to search to the storage nodes holding the respective lists. After the storage nodes have processed the query, it gathers the results from the storage nodes, re-ranks all candidates together based on their distance to the query vector and returns the `k` nearest neighbors to the client. Let us go into more detail about the root node's index and the selection of lists to probe.

Choice of Index The root node is equipped with a global index which we train on the entire dataset to allow for a precise selection of the lists. The index resides fully in memory, allowing us to make use of existing state-of-the-art indexing algorithms. HNSW [14] is an efficient graph-based in-memory index for approximate nearest neighbor search which achieves low-latency, logarithmic querying at high-recall through its hierarchical structure. Since our goal is to achieve a high-precision selection of the lists to probe while not sacrificing performance at the latency-critical part of our system, we choose to base our index implementation for the root node on `HNSW`. This allows us to reduce the number of lists to probe and therefore also the number of queries to be dispatched to the storage nodes. Further, it also reduces the overall I/O cost of the system, which is the main design goal of our system.

Extensions To achieve better fault tolerance, we redundantly perform the distribution of lists, meaning a single list is stored on multiple storage nodes. We will extend the implementation of the `HNSWlib` index to allow us to not only insert the centroids and the corresponding list ids but also the ids of all storage nodes which hold corresponding lists. Further, the root node should support batching of queries to reduce the overhead of the network communication. The root node issues query requests to the storage nodes using a remote procedure call interface, which enables transparent and scalable communication of our distributed system. For more details about the design and implementation of the root node, we refer to the continuation of this work.

3.5 Storage Node

Responsibilities Given a query received from the root node over the network, the storage node searches the preselected lists of on-disk vectors based on their distance to the query vector. For each query, it merges the results from all lists and returns the top nearest neighbors to the root node. The storage node has two main components: The first provides a layer of abstraction of the on-disk representation of the vectors, their ids and the lists they are

associated with. Resembling a key-value store-like interface, it is designed to be highly scalable. The second component enables efficient search over the lists of vectors it manages. Its search functionalities are exposed to the root node in form of a remote procedure call interface.

Extensions As mentioned in 3.4, the storage node is also capable of accepting and processing batches of queries. Aside from the reduction of network traffic, this also enables some more performance optimizations on the storage node side. Using the partitioned vectors obtained from the clustering, the storage node supports bulk insertion to bring the storage nodes into their initial state during index construction. Additionally, we allow for the insertion and update of vectors as well as the creation of new lists. Although it introduces new challenges on the clustering side, it gives the system the ability to dynamically scale which is highly desirable for online applications.

Data Layout and Datastructures The lists of vectors and ids are stored in a single memory-mapped file on disk. To be able to allow the insertion of vectors for arbitrary lists and efficiently access any list in at most a single disk seek, we choose to use an in-memory hashmap mapping list ids to offsets within the memory-mapped file. Lists are always stored contiguously in memory allowing for better use of the cache as well as enabling hardware acceleration techniques. We will discuss the implementation details and optimizations we applied in chapter 4.

3.6 Work Scope

Due to the limited scope of this thesis, as stated in 1.2, this work is mainly concerned with the implementation, evaluation and optimization of the system for the storage node. For the root node, we have limited the choice of the index to a linear-time, scan-based implementation. However, we will also briefly cover the simplified graph-based implementation for node which we needed for the system prototype. Further, since we are only concerned with the single server setting, we will leave the implementation of the network communication between the root node and storage nodes to the continuation of this work. For testing and benchmarking purposes, we will directly access the interface of the root node and storage node. In the next chapter, we will discuss the implementation of our system in detail.

Chapter 4

Implementation

To give a better idea of how the design elaborated in chapter 3 was implemented, we will now present selected components of the system itself as well as some parts around it in more detail. We focus on the functionality and potential design decisions we had to make and provide code samples when appropriate. After outlining how the project is set up, we focus our attention on the implementation of the storage node. Then, we will briefly discuss the root node, preprocessing as well as the design prototype.

4.1 Setup

Software The project was mainly written in C++ since it enables efficient low-level implementation of our design. Additionally, Python was used to write scripts for the prototype, preprocessing, benchmarking and evaluation.

Hardware Aside from the optimization techniques which require support for SIMD and OpenMP, the project is written to be hardware-agnostic. We have tested the system on a single machine with the hardware specification as described in 5.1.1.

Dependencies The core project has no dependencies. However, we make extensive use of the `Catch2` testing and micro benchmarking framework which we will introduce in 5.1.2. Moreover, the clustering and the prototype are based on the `FAISS` [11] and `HNSWlib` [14] libraries.

Source Code In figure 4.1, we have compiled a file tree as a reference overview of all relevant project files. The most relevant source listings can be found in the appendix (chapter 9). For the full source code, please refer to the repository at
https://github.com/TimonFercho/ann-distributed-key-value-store.

```
ann-distributed-key-value-store/
├── cluster/
│   ├── data/   datasets used, see 5.1.3
│   ├── out/   clustered data according to 4.2.1
│   ├── tmp/   intermediate indices
│   ├── cluster-dataset.py   see 4.4
│   └── sort-input-files.py   see 5.4.2
├── evaluation/
│   ├── baseline   see 5.4
│   ├── benchmarks/   output of the benchmarks
│   ├── benchmarks.json   configuration file for benchmarks
│   ├── batch-bench.py   automates benchmarks, see 5.2
│   └── evaluation.ipynb   used for postprocessing, see 5.2.3
├── include/   header files
├── lib/
│   └── catch2.hpp   unit testing framework, see 5.1.2
├── out/   compiled binaries
├── prototype/   see 4.5
│   ├── data/
│   ├── hnswlib/
│   └── ann-prototype.py
├── src/
│   ├── root-node/
│   │   ├── RootIndex.cpp   see 4.3.1
│   │   └── RootNode.cpp
│   ├── storage-node/
│   │   ├── StorageIndex.cpp   see 4.2.3
│   │   ├── StorageLists.cpp   see 4.2.2
│   │   └── StorageNode.cpp
│   ├── L2Space.cpp   used in 4.2.3, 4.3.1
│   ├── main.cpp
│   └── Query.cpp   used in 4.2.3, 4.3.1
├── tests/   unit tests and benchmarks, see 5.1.2
│   ├── IndexTest.cpp
│   ├── StorageListsTest.cpp
│   ├── StorageListsTestUtils.cpp
│   └── TestMain.cpp
└── Makefile   see 5.2
```

Figure 4.1: Project structure as a file tree

4.2 Storage Node

4.2.1 Input Files

Before we continue with the discussion of the storage node, we want to define the precise format of the input files needed for index construction of the storage and root nodes and highlight some assumptions about them. As described in 3.3, the partitioning gives us lists of vectors, each with a representative centroid. They are needed to be able to initialize the storage nodes with the lists of vectors they have been assigned. Further, the root node requires the centroids to be able to preselect the lists to probe at query time. We will now discuss the format of these files in more detail.

Format We choose to separate the input data into four different binary files, three for the storage node containing the vectors, vector ids and list ids and one for the root node containing the centroid vectors. Further, the files do not include any metadata, so the number of vectors, the number of lists and the vector dimension have to be specified during initialization and must match the data layout in the files. This approach reduces both the file size and complexity of the file format. The files are expected to be little-endian where all entries are expected to be stored sequentially in the following format:

File	Bytes per Entry	Number of Entries
vectors	`vector_dim * sizeof(vector_element)`	`n_entries`
vector ids	`sizeof(vector_id)`	`n_entries`
list ids	`sizeof(list_id)`	`n_entries`
centroids	`vector_dim * sizeof(vector_element)`	`n_lists`

Table 4.1: Input file formats

Here, `vector_element` is the data type used to represent a single vector component (e.g. `float`), `vector_id` is the data type used to represent a vector id (e.g. `unsigned long`) and `list_id` is the data type used to represent a list id. All vector components of a vector file entry are also stored sequentially. `n_entries` is the number of vectors that should be inserted into the index and `n_lists` is the number of lists or clusters. To give an example for the format, the first entry of the vectors, vectors ids and list ids files would be the first vector (with its components laid out sequentially), its vector id and the list id it is assigned to, respectively.

Assumptions We make no additional assumptions about the values, order or distribution of the vectors, vector ids or list ids, as long as the entries of the respective files match. These weak assumptions make the system more

flexible and scalable but also have implications for the implementation and performance. We will discuss the resulting design choices in the following sections and explore in chapter 5 what performance impact they have.

4.2.2 Storage Lists

Separation of Concerns As we have mentioned in 3.5, the storage node is responsible for both managing the lists of vectors on disk as well as searching them during query processing. We chose to separate these two responsibilities into their own component. We will first discuss how the storage lists component manages the vector lists and then go into more detail about how the storage index component makes use of this interface to expose search functionalities over these lists.

Responsibilities The Storage Lists is responsible for storing vectors of an some fixed dimension and their ids on disk, each vector being associated with a list. It allows to create and resize lists as well as to insert, retrieve and update entries. During query processing, the storage index accesses the vectors of the preselected lists through this interface.

Out-Of-Memory Mapping To allow storing datasets of vectors exceeding the memory capacity of a single storage node, the Storage Lists uses an on-disk file for storing and manipulating data. It accesses the file through a region of memory that is mapped to the file residing on disk. Since this is managed by the operating system it simplifies implementation at the cost of fine-grained control. We will discuss in 6.2 how this could be improved.

```
1    base_ptr = (uint8_t *)mmap(
2        nullptr,
3        total_size,
4        PROT_READ | PROT_WRITE,
5        MAP_SHARED,
6        fileno(f),
7        0);
```

Listing 4.1: Memory mapping of the file

Data Layout of the Memory-Mapped File All lists are laid out sequentially in a single file. Compared to mapping a single file for each list, this improves performance and scalability since there is only one region of memory and one file that needs to be managed. Further, any list can be accessed with at most one disk seek which will improve query times. However, since we also want to support the insertion of vectors after the index construction, we need to resize and potentially move lists to a different location within the file. We will go into more detail on this later.

Data Layout of an Inverted List A single list is always allocated contiguously in memory. Each list is allocated to allow space for a fixed number of entries. An entry corresponds to a vector and its id. To improve caching and allow data-level parallelism, all vectors are allocated at the beginning of the list's region of memory. Similarly, all vector ids are stored sequentially after the last allocated vector. This layout is independent of the number of entries already used in the list. The number of entries allocated for a list is always a power of two and is limited from below by some configurable parameter. We will discuss the allocation of lists in more detail later.

In-memory Data Structures To keep track of the lists and free space within the file, Storage Lists uses two data structures that reside in memory. The first is a list of free slots. A free slot is characterized by its offset relative to the beginning of the region and its size in bytes. For example, when the file is initially created, there is only one slot spanning the whole size of the region, starting at offset 0.

```
1  struct Slot
2  {
3    size_t offset;
4    size_t size;
5  };
```

Listing 4.2: Data structure storing metadata of a single free slot

Storage Lists also maintains a hashmap of inverted lists. The key is the list id and the value is an object keeping track of a list's metadata. Similar to a slot, this includes the offset within the region, but it also stores how many entries of the list have been allocated and used. The reason for this choice of data structure as opposed to another list is that we want to support insertion lists with list ids of arbitrary distribution which would not be possible by directly indexing into a list.

```
1  struct InvertedList
2  {
3    size_t offset;
4    len_t allocated_entries;
5    len_t used_entries;
6  };
```

Listing 4.3: Data structure storing metadata of a single list

Allocation and Freeing of Slots When a new list is created or resized, we traverse the list of free slots until we find a slot that is large enough to hold the new list. If there is no suitable slot available, we need to grow the

region to accommodate a new slot which will be placed at the end of the list. To avoid constantly having to grow the region, it will always be doubled, starting from some initial size. If the slot we found is larger than the list, we split the slot into two, one for the new list and one for the remaining free space. If the slot is exactly the size of the list, we simply remove it from the list of free slots.

When a list is resized, for example, to accommodate more entries, we may need to deallocate the slot it uses. If the slot to be freed is adjacent to another free slot, we merge them into one. This is done to reduce fragmentation which would occur, for example, when multiple smaller adjacent lists are freed.

Creating Lists To create a new list for a given number of entries, we first determine the number of entries to allocate, based on some minimum list length and the condition that the length is always a power of two. As a result, similarly to how the region is grown, we avoid having to constantly resize lists when new entries are inserted. Finally, we allocate a slot of the corresponding size and insert a new entry into our in-memory hashmap which keeps track of the lists' metadata.

Resizing Lists When a list needs to be resized, we first check whether the list needs to be reallocated. This decision is also based on the condition that a list should always be at least half-full, considering the case where the list is shrunk. If the list does not need to be reallocated, we just update the number of used entries.

If the list does need reallocation, we proceed as follows: First, we free the current slot of the list and allocate a new slot of the required size. Since we want to preserve existing entries of the list, we need to ensure that only those old entries are moved to their new location which should still be accessible in the new list. Finally, we update the list's metadata in our hashmap.

Reading Vectors and Ids Storage Lists allows direct read-only access to the vectors and ids of a list through pointers. We compute the pointer to the first vector of the list based on the memory location of the beginning of the memory-mapped file and the offset of the list. The first vector id is located after the last allocated vector of the list.

Updating Entries To update the vectors or ids of a list, we just copy them from the provided buffers to their respective positions on the file. Updating is only possible if the list already exists and the range which is to be updated is within the bounds of the list.

Inserting Entries Inserting entries into a list is done by first resizing the list to the required size and then updating the entries at the end of the list.

Bulk Insertion of Entries During index construction, following the clustering of the data, we end up with the vector, vector ids, list ids and centroids files in the format, as described in 4.2.1. When provided with the first three files, we support the insertion of all listed entries across an arbitrary number of lists.

Due to the weak assumption on the input files, as detailed in 4.2.1, the initial approach was a two-pass algorithm. In the first pass through the list ids file, we determine the number of vectors for each list. Using this information, we then preallocate lists of the required sizes. In the second pass, now over all files, we copy the vectors and ids of each entry into the corresponding preallocated lists within the file.

Bulk Insertion Optimizations As we will detail in chapter 5, this initial implementation of the bulk insertion turns out to be fairly inefficient due to the random writes caused by having to write to an arbitrary list for each entry. For large datasets which do not fit into memory, this problem is magnified due to the high latency of disk seeks. Although it may not be applicable in practice, we will explore in chapter 5 how strengthening the assumptions on the input files by enforcing some order on the list ids leads to great performance improvements. Based on this, we propose an alternative single-pass algorithm inspired by the `SPIMI` algorithm [8]. Due to time limitations, we will only outline in 6.2 the extensions to our initial implementation that would be necessary to support it.

Combining the insights from the previous approaches, we came up with another insertion algorithm, dynamic insertion, which is a single-pass algorithm like `SPIMI` but did not require significant extensions to our existing implementation. Instead of initially traversing the list ids file to pre-initialize the lists, we only traverse all files once and allocate the necessary lists on-the-fly. Since we initially do not know how many entries we need to allocate per list, we may need to relocate existing lists during the insertion process to accommodate more entries. As we will discuss in chapter 5, the overhead of reallocating lists is offset by only having to pass over the input files once. Specifically, it results in a significant speedup compared to the two-pass algorithm based on list pre-initialization.

4.2.3 Storage Index

Responsibilities The storage index component is responsible for performing the nearest neighbor search over the lists to probe as specified in the query. It

implements two functions to perform the search: `search_preassigned` and `batch_search_preassigned`. The former performs the search given a single query while the latter performs the search over a batch of queries. We will detail the implementation of both functions in the following.

search_preassigned Given a query, `search_preassigned` searches the lists of vectors specified and returns the `k` nearest neighbors. Through the interface of Storage Lists, we access all vectors of each list and compute the distance of each vector to the query vector. We keep track of the vectors which are closest to the query vector using a priority queue that can hold up to `k` tuples of distances and vector ids. That allows us to access the worst candidate at the top of the queue in constant time and we can decide efficiently whether the current vector is closer to the query vector than any of the `k` closest ones found so far.

```
1     for (size_t j = 0; j < list_size; j++)
2     {
3       const vector_el_t *vector = &vectors[j *
            vector_dim];
4       float distance = distance_func(vector,
            query->get_query_vector(), &vector_dim);
5       const vector_id_t vector_id = ids[j];
6       QueryResult result = {distance, vector_id};
7       add_candidate(query, result, candidates);
8     }
```

Listing 4.4: Searching a single list

Search Optimizations To improve the search performance of the system, we support batching of queries. This means that the root node can collect multiple queries in a query batch which is used to search the root index and the storage nodes for multiple queries simultaneously. The storage nodes then respond with a set of nearest neighbors along with their respective distances for each query of the batch. With `SIMD` (2.3.1), we also support distance computations based on vectorization. Using these special processor instructions, we can exploit data-level parallelism to achieve high performance. Finally, we used `OpenMP` (2.3.2) to fully parallelize the search on the storage nodes both over all queries within a batch as well as the individual lists within a single query. This allows us to further improve both the throughput and latency of the system. Let us now go into more detail about how we implemented these optimizations in `batch_search_preassigned`.

batch_search_preassigned For `batch_search_preassigned`, we need to perform the same function as `search_preassigned` but now we deal with a batch of queries. As we have just described, we make use of SIMD to speed

up distance computations and OpenMP to parallelize the search. Assuming 128-dimensional vectors in euclidean space, we can process 8 vector components simultaneously using `AVX` instructions on 256-bit wide vector registers. We will analyse the performance gains of SIMD in chapter 5.

```
#if PMODE != 0
#pragma omp parallel for schedule(runtime)
#endif
    for (len_t i = 0; i < queries.size(); i++)
    {
      results[i] = search_preassigned(queries[i]);
    }
```

Listing 4.5: Search parallelized over queries

Now, let us delve deeper into the specifics of OpenMP to improve our search. Aside from sequential search, we support two additional modes of operation. The first processes all queries in parallel. Since searching is read-only and each query has its own result set, there is no shared data and therefore minimal resource contention between threads. For the second mode of operation, we parallelize over both queries and lists. To support a variable number of lists to probe for each query, we first create a list of work items. A work item corresponds to a query and one of the list ids which needs to be probed. We then distribute the work items over the available threads. Each thread maintains its private candidate list and merges its results with the results of the corresponding query. Since there may be multiple threads trying to access a single query's result set, we need to ensure that this step is performed atomically. We will discuss the performance trade-offs between the different parallelization modes and explore the various scheduling policies OpenMP offers in chapter 5.

```
#pragma omp parallel for schedule(runtime)
    for (len_t i = 0; i < work_items.size(); i++)
    {
      len_t query_index = work_items[i].first;
      const Query *query = queries[query_index];
      list_id_t list_id = work_items[i].second;
      heap_t local_candidates;
      search_preassigned_list(query, list_id,
         local_candidates);
#pragma omp critical
      {
        while (local_candidates.size() > 0)
        {
          QueryResult result = local_candidates.top();
          local_candidates.pop();
          add_candidate(query, result,
             candidate_lists[query_index]);
```

```
16          }
17        }
18      }
19  #pragma omp single
20      for (len_t j = 0; j < queries.size(); j++)
21      {
22        results[j] = extract_results(candidate_lists[j]);
23      }
```

Listing 4.6: Search parallelized over queries and lists

4.3 Root Node

Restrictions As we have mentioned in 1.2, this thesis focuses primarily on the storage node. Therefore, we have limited our choice of the index for the root node to a simple linear scan-based algorithm over the centroids. We refer to the continuation of this work for the graph-based implementation of the root node which will be used in the final system.

4.3.1 Root Index

Responsibilities In the following, we will describe our initial implementation of the root node. Specifically, we will focus on the root index component which is responsible for searching the centroids and selecting the lists to probe.

In-Memory Data Structures The Root Index only needs to allocate space for the centroid vectors on the heap. We store all vectors sequentially in memory, allowing for fast access during query processing.

preassign_query Given a query and the number of lists to probe, `n_probe`, `preassign_query` searches all centroids and returns the list ids corresponding to the `n_probe` closest centroids. The search is performed analogously to `search_preassigned` with the difference that we conceptionally only search a single list.

batch_preassign_queries Given a batch of queries and for each the number of lists to probe, `batch_preassign_queries` performs the same function as `preassign_query` but now for a batch of queries. Similarly to `batch_search_preassigned`, we can exploit data-level parallelism using SIMD to improve search performance. Since we only need to search a single list of centroids, `batch_search_queries` can solely support parallelization using OpenMP over the queries of a batch.

4.4 Index Construction

Purpose As part of the index construction, we will briefly discuss the preprocessing steps we need to perform to obtain the input files used for the initialization of the storage nodes and the root node. We have implemented the preprocessing steps in Python based on the `FAISS` library developed by Meta's Fundamental AI Research group [11]. We refer to cluster/cluster-dataset.py for the full source code of the clustering script.

Algorithm Although other clustering algorithms like hierarchical clustering may be more suitable for the final system, the k-means algorithm as implemented by `FAISS` is sufficiently fast and accurate for the scope of this work. It is a well-known and widely used clustering algorithm that is used to partition a set of vectors into k clusters. K-means iteratively assigns each vector to the closest cluster centroid and then updates the centroids based on the assigned vectors. The algorithm converges when the centroids do not change anymore. For a more detailed discussion about the partitioning, we refer back to 3.3.

For billion-scale datasets like `BIGANN`, we had to work around the limitations of memory capacity. Since such datasets exceed during clustering even the memory capacity of 250 GB available on our machines, we had to resort to a batching approach. We first split the dataset into batches of e.g. 100 million vectors and construct the index for each batch separately. We then merge the resulting indices into a single on-disk index.

Output For each dataset, the output of the clustering includes the list ids each vector is assigned to as well as a list of centroids representing each cluster of vectors. Together with the original vectors and their vector ids we have all the information we need to build the inverted file index at the storage nodes and the index at the root node. We have already discussed the format of the input files generated by the clustering in section 4.2.1.

4.5 Prototype

Purpose The prototype served multiple purposes. First, it allowed us to explore our design rapidly by combining the functionalities of existing libraries through a high-level interface and getting familiar with the existing implementations. Further, it allowed us to test the feasibility and correctness of our design. These were crucial steps since we later based our implementation on these preexisting pieces of work. We refer to prototype/ann-prototype.py for the full source code of the prototype. We will now describe how modeled each main component of our design.

Root Node For the index of the root node, we used `HNSWlib` [14] which is an open-source library implementing a graph-based index. The reason for this choice is that we want to achieve a fine-grained and accurate selection of the lists to probe to minimize the amount of I/O at the storage nodes which have to retrieve the lists from the disk. Moreover, this graph index achieves high query search performance in logarithmic time due to its hierarchical structure.

Storage Node For the partitioning of the vectors and searching over the list of vectors, our prototype is based on the inverted file index provided by `FAISS` [11]. We chose `FAISS` since it is a state-of-the-art open-source library for similarity search and clustering which offers high performance and a wide selection of indices. Since our setting is out-of-memory and exact search (3.1), we were most interested in their implementation of on-disk inverted lists and the inverted file index without quantization. The prototype was mainly intended for proof of work, so we used the in-memory variant of their inverted file index to cluster and search the vectors.

Chapter 5

Evaluation

5.1 Setup

5.1.1 Hardware

All experiments were performed on one out of the 14 servers part of the Systems group's CPU cluster. These machines are equipped with the following:

- Intel Xeon E5-2640 v3 2.40 GHz
- 2 sockets with a total of 16 cores
- 2 Threads per core
- 256 GB RAM
- Network-attached hard drives
- No use of hardware accelerators

5.1.2 Software

Catch2 We used `Catch2` [17] extensively to test the correctness of our code as well as to run most of our microbenchmarks for performance evaluation. `Catch2` is a modern, C++-native, test framework for unit tests. The `Catch 2.x` we are using is header-only, which allows us to easily integrate it into our build system. It offers a feature-rich and intuitive API while maintaining a light footprint. We refer to their open-source repository at https://github.com/catchorg/Catch2/tree/v2.x for more details.

5.1.3 Datasets

BIGANN Our system was evaluated using the `BIGANN` dataset [2] and subsets thereof. The BIGANN benchmark is known as the largest publicly

available dataset for approximate nearest neighbor search and consists of 1 billion 128-dimensional SIFT image descriptors extracted from approximately 1 million images, with a total size of 128 GiB. The dataset contains the parts as listed in table **??**.

Subset Name	Size	Description
Base Vectors	1 billion	The dataset of vectors to be searched
Learning Vectors	100 million	A set of vectors used for computing indexing method parameters
Query Vectors	10000	Vectors submitted to the system for search
Ground Truth Indices	10000	The nearest neighbors of the query vectors
Ground Truth Distances	10000	The exact square distance between the query vectors and their k nearest neighbors

Table 5.1: Subsets included in the BIGANN dataset.

The creators have also released subsets of the dataset and the respective ground truth indices and distances, ranging in size from 1 million to 1 billion. During our evaluation, we indicate the specific subsets of `BIGANN` we used, namely SIFT1M, SIFT10M, SIFT100M, and SIFT1000M/SIFT1B.

SIFT1M For the design prototype, we used a smaller SIFT1M dataset [10] (not to be confused with the 1 million subset of `BIGANN`) which also consists of 1 million 128-dimensional SIFT image descriptors. It is based on Mikolajczyk implementation of Hessian-affine detector [15], unlike the `BIGANN` dataset which is based on Luwe's implementation [13].

Both datasets are available in the public domain at `http://corpus-texmex.irisa.fr`.

5.2 Benchmarking

5.2.1 Parameters

Our implementation is fully parametrized using preprocessing statements defined during compilation. This allows us to evaluate it over all parameters listed in table 5.2. For each parameter, we have specified the default value which will apply unless it is mentioned elsewhere. Some parameters also

refer to the configuration of the unit tests and microbenchmarks we created using `Catch2`.

In section 5.4, we will discuss the effects of all relevant parameters on the performance of the system based on the metrics we will describe in 5.2.3. In section 5.2.2, we will outline how automation enabled us to efficiently run our benchmarks and collect the results of many metrics over large parameter ranges.

Parameter	**Default Value**
OpenMP enabled	Yes
OpenMP schedule	dynamic
Parallelization mode	over queries and lists
Number of OpenMP threads	32
SIMD enabled	Yes
Insertion strategy	dynamic
Compiler optimization level	3
Number of benchmarking samples	3
Buffer size used in bulk insertion in entries	100
Minimum size of the memory-mapped region in bytes	32
Minimum number of entries per list	1
Dataset	SIFT1M
Number of lists	1024
Number of probes	16
Number of results	10
Vector dimension	128

Table 5.2: Parameter default values, based on listing 9.18.

5.2.2 Reproducibility

Even if some of the parameters listed in table 5.2 can only take categorical values, the parameter space to explore is exponential. Further, running the benchmarks is an immensely time-consuming process, especially since the iterative nature of the development process requires us to run the benchmarks many times. Lastly, a single sample is not sufficient to argue about the performance of the system, for example, due to fluctuating interference. Therefore, including warmup, we decided to sample at least four runs for a single data point. Even then, collecting the samples for a single plot may take several days due to large dataset sizes or many combinations of parameters.

To improve efficiency and ensure the reproducibility of the results, we have fully automated the benchmarking process. Based on a configuration file

that lists the parameters of all microbenchmarks run for the evaluation (9.18), we have created a simple script (9.19) that will compile the binary with the specified parameters, set the environment variables, run the benchmarks, and output the results. We further parsed the outputs, computed the metrics and plotted the results using a single reusable python notebook.

5.2.3 Metrics

In the following, we will present the metrics we used to evaluate our system. We will first discuss the metrics we used to evaluate the performance of our system, followed by the metrics we used to evaluate its correctness. Note that, all metrics are computed based on the mean and standard deviation of the execution time of the benchmarked function as well as the program parameters specified for each run.

Throughput The throughput of a system is the number of operations it can perform or the amount of data it can process per unit of time. Depending on the method being measured, we used various throughput metrics to evaluate our system. For throughput-related metrics, we measure the overall runtime of execution of all queries. Assuming uniformly distributed vectors among the lists, we computed the corresponding metrics of the system as follows:

$$\text{queries_per_s} = \frac{\text{N_QUERIES}}{\text{time_s}} \tag{5.1}$$

$$\text{queries_per_s_std} = \frac{\text{N_QUERIES} \cdot \text{time_s_std}}{\text{time_s}^2} \tag{5.2}$$

$$\text{size_mb_bulk_insert_entries} = \text{N_VECTORS} \cdot \text{VECTOR_SIZE_MB} \tag{5.3}$$

$$\text{size_mb_preassign_queries} = \text{N_LISTS} \cdot \text{VECTOR_SIZE_MB} \cdot \text{N_QUERIES} \tag{5.4}$$

$$\text{size_mb_search_preassigned} = \frac{\text{N_VECTORS} \cdot \text{N_PROBES} \cdot \text{VECTOR_SIZE_MB} \cdot \text{N_QUERIES}}{\text{N_LISTS}} \tag{5.5}$$

$$\text{mb_per_s} = \frac{\text{size_mb}}{\text{time_s}} \tag{5.6}$$

$$\text{mb_per_s_std} = \frac{\text{size_mb} \cdot \text{time_s_std}}{\text{time_s}^2} \tag{5.7}$$

Latency Latency, especially tail latency [5], is a critical metric of distributed systems for assessing the responsiveness of a system. Since measuring latencies over a set of queries is not natively supported by `Catch`, we manually measure the latency for each query separately (listing 9.13). We perform the measurements `N_SAMPLES` times excluding an additional initial warmup run over all queries. We compute the median and 95th percentile on the sorted latencies as follows:

$$\text{latency_median} = \text{latencies}\left[\frac{\text{N_QUERIES}}{2}\right] \tag{5.8}$$

$$\text{latency_95th} = \text{latencies}\left[\frac{\text{N_QUERIES} * 95}{100}\right] \tag{5.9}$$

Then, we compute the mean and standard deviation over all samples of each metric as follows:

$$\text{latency_mean} = \frac{1}{\text{N_SAMPLES}} \sum_{i=1}^{\text{N_SAMPLES}} \text{latency_runs}[i] \tag{5.10}$$

$$\text{latency_std} = \sqrt{\frac{1}{\text{N_SAMPLES}} \sum_{i=1}^{\text{N_SAMPLES}} (\text{latency_runs}[i] - \text{latency_mean})^2} \tag{5.11}$$

Speedup We compute the speedup to compare the performance improvements between different configurations of our system based on the following formulae or the reciprocal thereof, depending on the metric:

$$\text{speedup} = \frac{\text{new_value}}{\text{old_value}} \tag{5.12}$$

$$\text{speedup_std} = \text{speedup} \cdot \sqrt{\left(\frac{\text{new_value_std}}{\text{new_value}}\right)^2 + \left(\frac{\text{old_value_std}}{\text{old_value}}\right)^2} \tag{5.13}$$

Recall As we have already discussed in 2.1, recall is a metric that measures the quality of the results returned by the system. In our case, it serves primarily as a way to evaluate the correctness of our implementation as the final system will be based on a different index implementation at the root node. We compute the recall by comparing the results of the search with the ground truth we obtain from 5.1.3.

Recall@R Recall@R is defined as the number of queries where the true nearest neighbor appears in the top R results 5.14. For our setting, it is sufficient to consider Recall@1 since we only consider exact search. As opposed to algorithms like product quantization which are based on compression, exact distance computations guarantee that the true nearest neighbor will always be the first among the results. One exception we faced is the case where there are two true first nearest neighbors with the same distance to a given query vector of the dataset and the ordering behaved non-deterministically. To cover this case, we had to enforce an ordering, based on the vector ids, which matches the ground truth of the dataset (9.8). As one can see in listing 9.13, we compute the recall@r as follows:

$$\text{Recall@R} = \frac{\text{number of true nearest neighbors in top R}}{\text{N_QUERIES}} \tag{5.14}$$

Assertions Last but not least, we have made extensive use of assertions provided by `Catch` to verify the correctness of our implementation. They allow us to test if the system behaves as excepted. We will go into more detail about the correctness in 5.3.

5.3 Correctness

Unit Tests and Constraint Random Testing `Catch` provides a powerful framework for defining unit tests guided by behavior-driven development (BDD) which allows us to verify the expected behavior of our system (see 9.12, 9.13 and 9.16). Although the absence of failed tests does not imply the absence of unintended behavior, features such as data generators enable us to test the system with constraint random testing techniques. Constraint random testing is known to be effective in finding bugs by generating a large number of meaningful test inputs within a large search space. We have omitted any test results as the procedure outlined in 5.2.2 also enables running the unit tests based on the same configuration file (9.18).

Recall Benchmarks Based on the ground truth nearest neighbors the `BIGANN` dataset provides, we have measured the recall of our system from end to end. This allows us to verify the correctness of both the index implementation at the root node and the search at the storage lists. This is the case because the root node is responsible for the selection of the partitions to search and the storage node will then search the selected lists for the k-nearest neighbors. Finally, we can compare the results by the storage node for the nearest neighbors with the ground truth vector ids. As mentioned in 5.2.1, we chose to use k equal 10 and compute the recall@1 while varying the number of lists and number of probes. As one can see in figure 5.1, we achieve a recall

of about 90% for all list sizes and 16 probes and a recall of over 98% for all lists and 64 probes. As the final system will not contain our current index at the root node, the recall will only be affected by the graph-based index implementation as well as the partitioning approach, making our results only useful for correctness verification.

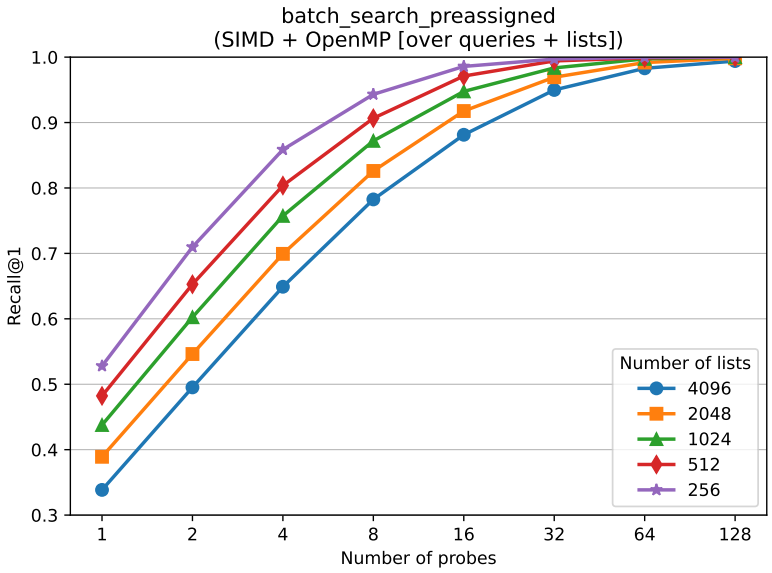


Figure 5.1

5.4 Performance

5.4.1 Optimization Goals

As mentioned in 3.1, we aim to build a scalable and efficient solution for approximate nearest neighbor search in a billion-scale, out-of-memory setting. We aim to minimize the required I/O operations for a given query and maximize the throughput under some latency bound. For the scope of this work, we will mostly focus on the throughput and latency of the system as we vary the search parameters as well as the parameters of our system (see 5.2). We will also discuss the scalability of the system mainly for the case of bulk insertion as the search components on the storage node primarily operate on small subsets of the data at query time. We will start our discussion with the performance of the bulk insertion and then move on to the search performance of the root and storage node.

5.4.2 Bulk Insertion

Buffer Size Firstly, we would like to discuss the effect of the buffer size on the throughput of the bulk insertion. The buffer size parameter describes the number of elements that are read from the three input files into an area on the heap before they are inserted into the memory-mapped file. As we can see in figure 5.2, the buffer size does not have any significant effect on the performance of the bulk insertion. This might be due to the following reason: Since both the file storing the inverted lists and the input are memory-mapped and therefore managed by the operating system, additional buffering in the user space may not have any effect. This might especially apply to this benchmark, as the dataset was chosen to be only 10 million vectors in size and therefore all files may already fully reside in memory, making buffering in the user space unnecessary.

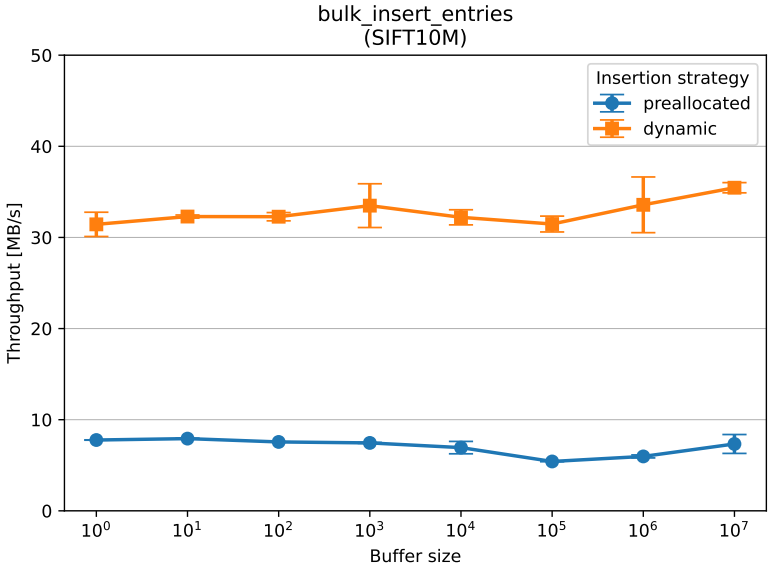


Figure 5.2

Relative and Absolute Performance What has a more significant effect on the performance, however, is the choice of the insertion strategy (see 4.2.2). For this benchmark, we compare the insertion with preallocated lists with the dynamic insertion on the SIFT10M dataset. As one can see in plot 5.3, dynamic insertion significantly outperforms the preallocated strategy on this dataset size by over a factor of 4. Despite this great relative performance improvement, we would like to note that the absolute throughput of the bulk insertion with both strategies is still very low, achieving only 8 MB/s for the preallocation strategy and around 32 MB/s for the dynamic insertion.

This was also the reason, why we decided to come up with alternative implementations for the initial preallocation strategy. We will discuss a major contributing factor to this, the memory access pattern, later in this section. We will also then investigate if the performance gap between the two strategies is maintained when scaling to billion-scale datasets.

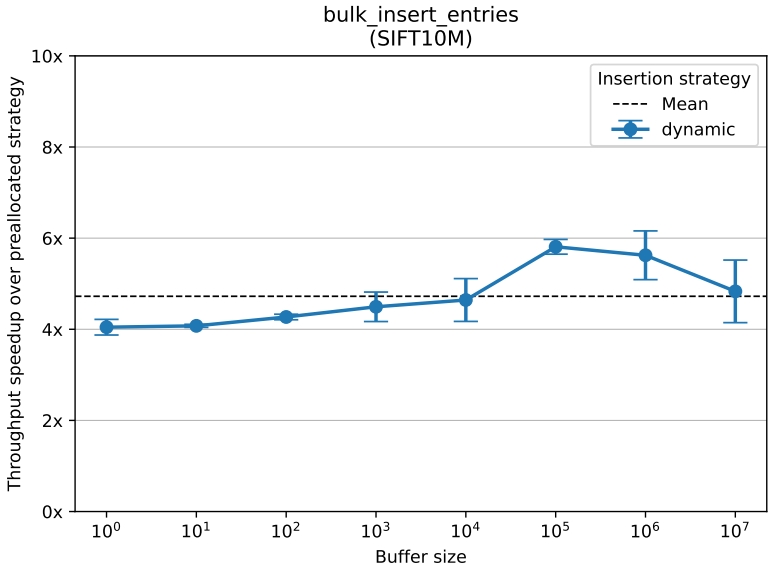


Figure 5.3

Insertion Strategies The main difference between the two is that the dynamic insertion strategy is a single-pass algorithm as opposed to the preallocation strategy which requires passing over the list ids file once at the beginning. As outlined in 4.2.1, we have no assumptions on the distribution of vectors among the lists, so we do not know how much space needs to be allocated for each list. Therefore, the original implementation made use of a hash map to count the number of vectors assigned to each list and preallocated the space in the list ids file. On the contrary, the dynamic insertion strategy allocates lists on demand, meaning entries are just inserted on-by-one into the respective lists. This case, however, requires the lists to be dynamically resized once they run out of space from their initial allocation size. Even though the lists need to be moved to accommodate more vectors, we presume that this overhead must be minimal. Further, the SIFT10M dataset may allow these moves to be performed in memory, where the costs of such operations is significantly less, compared to the on-disk setting. Therefore, we can conclude that the dynamic insertion algorithm achieves significantly better performance compared to the initial strategy based on preallocating the list.

Number of lists Now we investigate the dynamic insertion for a dataset size of 1 million (we consider the dynamic insertion strategy as the default, see 5.2). Compared to the previous plot which was based on the SIFT10M dataset, the throughput in figure 5.4 is slightly higher at around 40 MB/s over 30 MB/s. As we will discuss the scaling later in more detail, we want to emphasize an obvious trend we notice when increasing the number of lists. As the number of lists increases, the throughput drops from about 60 MB/s at 256 lists down to 30 MB/s for 4096 lists. We would like to notice here that the dataset size is constant throughout the experiment, meaning that the number of vectors per list decreases as the number of lists increases. The main reason for this is likely caused by caching behavior. More specifically, when the number of lists increases, more locations in the file need to be written to. Since it is sufficient to cache the tail of each list to be able to insert more entries, the cache misses increase as the number of lists increases as there is not enough space to cache all list tails. Overall, the throughput drops by a factor of 2 when increasing the number of lists by a factor of 16, which is significant but an acceptable trade-off when scaling to a higher number of lists.

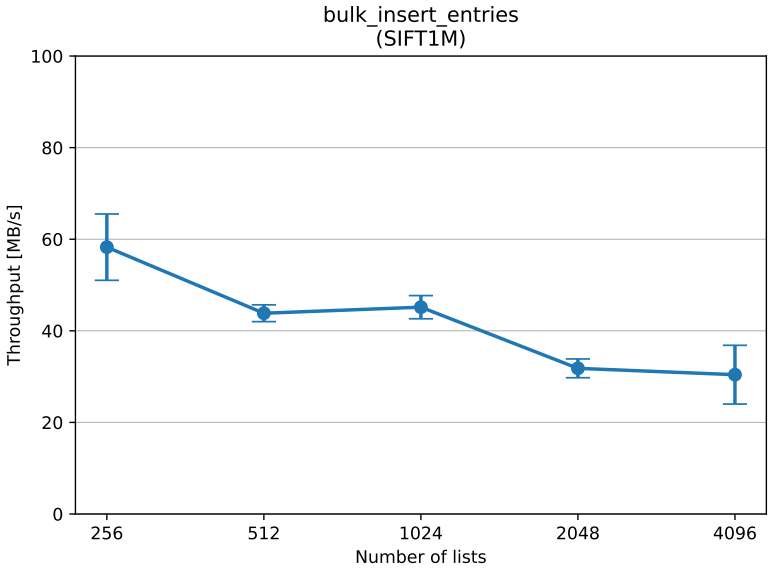


Figure 5.4

Memory Access Pattern In this section, we would like to go back to the discussion about the different insertion strategies. While investigating the performance of the bulk insertion based on preallocation for the first time, we found that although the input files are read sequentially, the locations in the file where the entries are written to are random. Considering that we do

not put any constraints on the order of vectors or their list ids during the clustering phase, the random access pattern is expected. Unfortunately, randomly writing to memory or even worse to disk is very inefficient, especially for such small units of data. This explains why the initial run to insert the SIFT1B dataset took close to a day to complete, not considering the warmup run which `Catch` performs by default. To put these assumptions to the test, we conducted two experiments. The first was to trace the memory writes during the insertion of the SIFT1M dataset. The result can be seen in figure 5.5 and shows that indeed the memory writes are random.

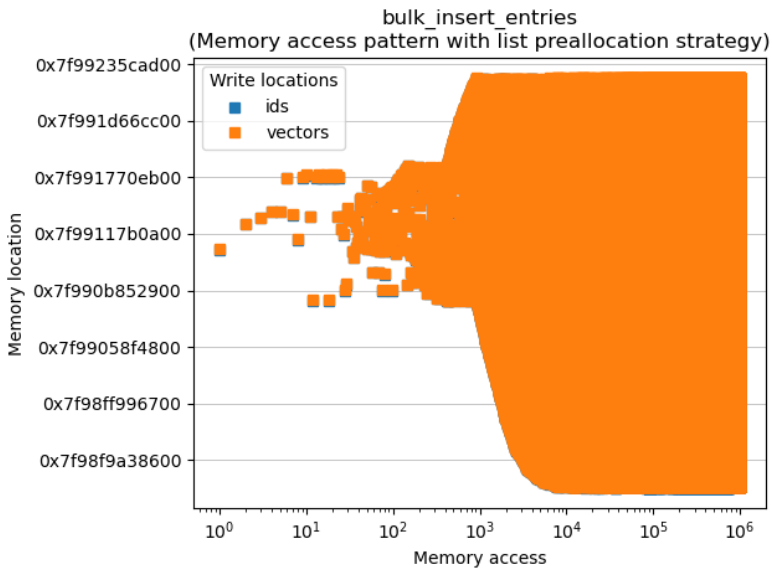


Figure 5.5

Strengthening Assumptions on the Input Files As a second experiment, we tried to see how the bulk insertion performs when we strengthen the assumption on the list ids of the input files. Although not practical for large datasets, we did so by sorting the input files before insertion (see 9.21). We left the algorithm based on preallocation untouched and observed that the throughput improved by more than an order of magnitude, from 8 MB/s to over 100 MB/s when compared to the unsorted input files and the insertion strategy based on preallocation. This motivated us to come up with another alternative implementation of the bulk insertion which was inspired by the `SPIMI` algorithm [8]. The idea was to avoid random writes to disk by building multiple intermediary indices in-memory and merging them in a single pass. Unfortunately, due to time constraints, we were not able to implement this algorithm, however, we will elaborate on the extensions which are needed for our current implementation in 6.2.

Insertion Strategies on Larger Datasets Despite this, this paved the way for the dynamic insertion strategy which is a single-pass algorithm like `SPIMI` but only required minor changes to the current implementation. Although it does not alleviate random writes, we can still achieve a significant performance improvement over the initial strategy based on preallocation. Figure 5.6 shows that the dynamic insertion strategy outperforms preallocation by a factor of 5 for small datasets with decreasing speedup as dataset size increases, down to around 2.8x for the SIFT1B dataset. We would consider this to be a significant improvement, making the dynamic insertion strategy a viable alternative to the preallocation strategy, even for large datasets. Further, we would like to note that this improvement is marginal compared to the speedup when we enforce an order on the list ids of the input files, which ranges between 15x and 25x. The large variance is likely caused by fluctuations in the measurements for the short-running dynamic (sorted) strategy which are amplified in the speedup computation due to numerical errors during floating point division. Since sorting the data is likely not practical for large datasets, we have omitted to benchmark the dynamic (sorted) insertion for the 1 billion dataset. Although the results for the sorted inputs do not reflect the time needed for sorting the input data, speedups of one order of magnitude seem realistic. They may also motivate the implementation of a `SPIMI`-like algorithm in the future if its implementation is considered feasible and still yields a significant performance improvement when moving from HDD to SSDs in the final system. Such an algorithm would become even more attractive for larger datasets since costly paging from disk is more likely to occur and would be significantly reduced with such a one-pass algorithm.

Scaling Bulk Insertion As promised in the previous section, we now investigate the performance of the bulk insertion with varying dataset sizes (see figure 5.7). Considering the dynamic insertion strategy, we can observe that the throughput declines from 40 MB/s for the SIFT1M dataset down to 16 MB/s for the SIFT1B dataset, which is equivalent to a decrease of 2.5x. Considering the dataset size is increased by a factor of 1000x at the same time, the decrease in throughput is very acceptable and expected primarily due to a reduction in the locality of the access pattern during the insertion. Interesting to note is that although the preallocation strategy overall performs worse with less than 10 MB/s, with a drop of 1.6x, it is not affected as much by the scaling of the dataset size. This might be caused by the fact that it is already bottlenecked by the I/O bandwidth which would also explain why both dynamic insertion and the preallocation strategies which share a similar random access pattern seem to trend toward the same throughput for datasets beyond 1 billion vectors.

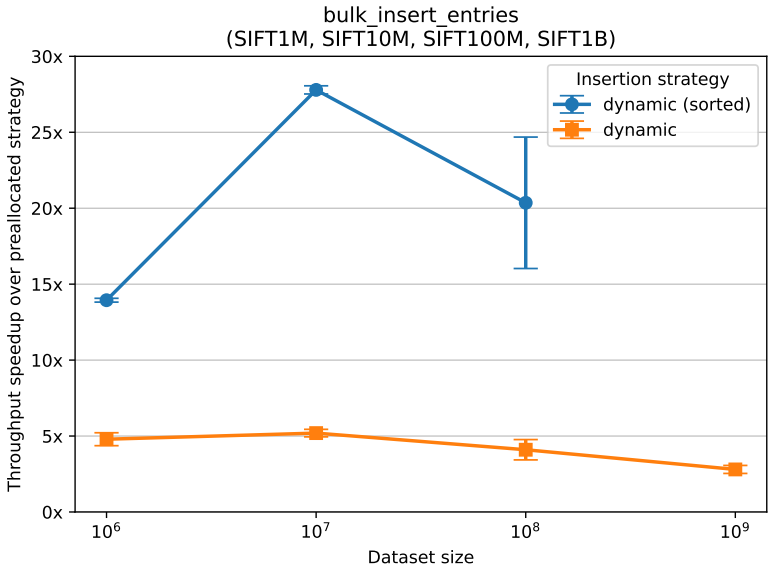


Figure 5.6

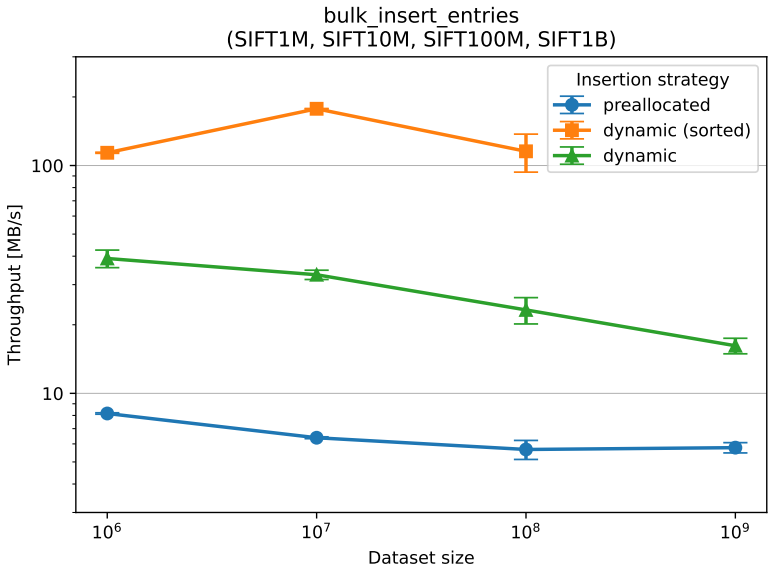


Figure 5.7

5.4.3 Search

Let us now move on to examining the search performance of the system.

Search Space

- The Search space describes the amount of data that needs to be accessed

by the storage node during a search or our (preliminary) implementation of the root node while pre-assigning queries to lists. Having an intuition about how the search space behaves with varying parameters, will help us in explaining some trends in the results.

- Figure 5.8 shows that the space independent while searching the centroids is independent of the number of probes. However, it does increase linearly with the number of lists. This is not surprising since each list has one representative centroid and the root node needs to compare each query vector against all of the centroids in our implementation.
- The search space while searching on the storage node (figure 5.8) grows linearly with the number of probes and decreases linearly with the number of lists. Based on the assumption that vectors are distributed uniformly across the lists and the fact that the dataset size is fixed for this theoretical experiment, we can conclude that a low number of lists results in long lists and the more probes we have the more lists we need to search through.

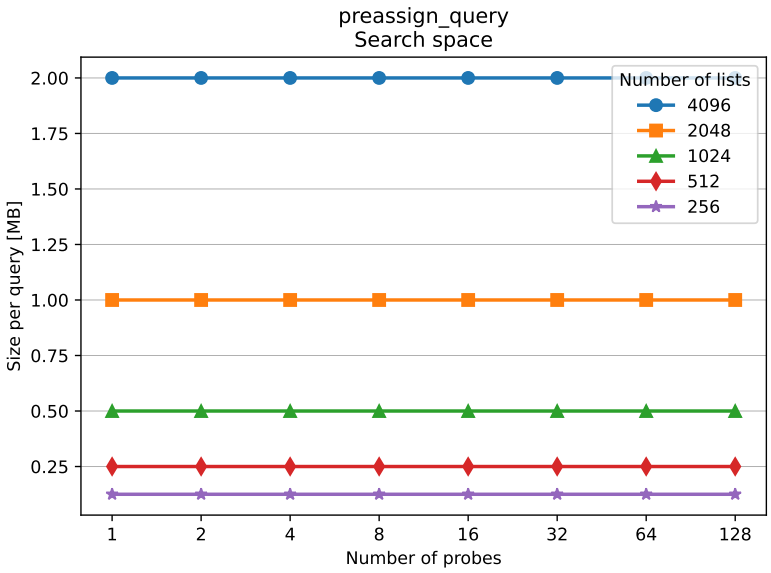


Figure 5.8

Vanilla (no OpenMP, no SIMD)

- We will begin with the discussion of the vanilla case, meaning we do not consider any optimizations. Each of the sections will include a discussion of the throughput and the latency for `Preassign` and `Search`. While `Preassign` refers to the scan-based centroid search performed by the root node of our implementation, `Search` refers to

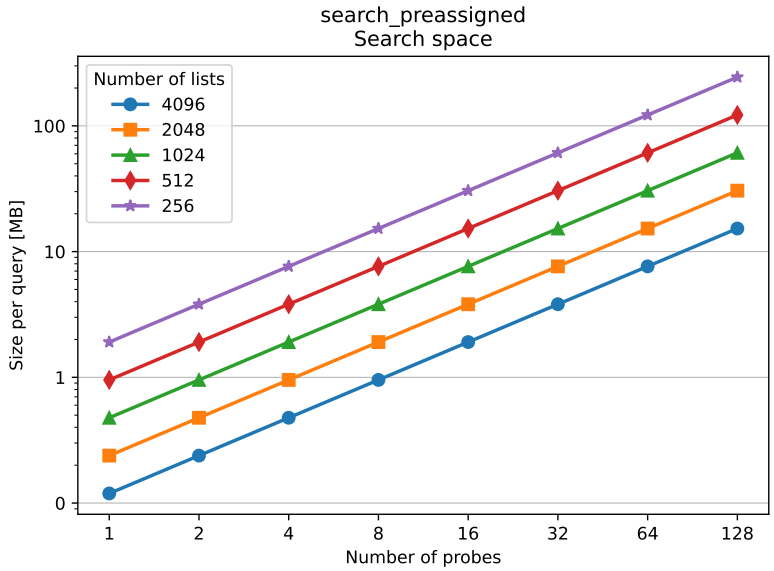


Figure 5.9

the search performed by the storage node over the selected lists on disk. Later sections discussing the effects of the SIMD and OpenMP implementations will also describe the speedup compared to a previous version with fewer optimizations.

- **Preassign Throughput** (figure 5.10): Here, we consider the vanilla case without any optimizations, meaning no SIMD acceleration or parallelization with OpenMP. The throughput slightly drops from around 4 to around 3 GB/s when the number of probes surpasses 16. This trend can be explained by the fact that the root index needs to keep track of n_probes many candidates in its heap. As the size of candidates increases, the heap which contains pairs of distances and list ids can likely no longer fit in the lower caches. When further the number of lists is increased, we observe a relative the search performs better. This is likely because the search space also increases (figure 5.8) and therefore the throughput improves, as overheads become less significant for searches that take more time.
- **Preassign Latency** (figure 5.11): The 95th latency does not increase significantly when increasing the number of probes. There is, however, a noticeable bump when n_probes is large, which we already found in the throughput. The Latency scales linearly with the number of lists, as expected, between 32 us and 500 us, ranging the number of lists from 256 to 4096. We see a similar trend in median latency (figure 5.12) and notice that the absolute latencies are almost identical, meaning that the search is very consistent. Unlike the search at the storage node,

where list lengths differ, the root node always searches a single list of centroids on the heap which results in a very consistent latency.

- **Search Throughput** (figure 5.13): First, we would like to note that the number of results k is chosen to be 10 for most of the following plots (see 5.2.1). Without any optimizations, the throughput of search at the storage nodes is relatively constant between 3 and 4 GB/s. Contrary to pre-assign queries, it increases slightly with an increasing number of probes and also with decreasing number of lists. This is because, in the case of the search on the storage nodes, both parameters affect the search space, as the number of results k is fixed to 10 for most of our experiments, resulting in a heap of constant size. Therefore we observe a slight throughput increase when increasing the search space (see figure 5.9).

- **Search Latency** (figure 5.14): Fixing the number of lists to 1024, the 95th percentile latency ranges between 126 us for 1 probe and 18 ms for 128 probes. When increasing the number of probes, which means searching through more lists, we observe a linear increase in latency. The latency also decreases linearly with the number of lists as individual lists become shorter for a fixed dataset size and therefore reducing the time to search. The variance in list lengths can be observed by the fact that the median latency is only about half as high as the 95th latency. Therefore, for some queries, response times may be up to two times higher, for example, when many long lists need to be searched.

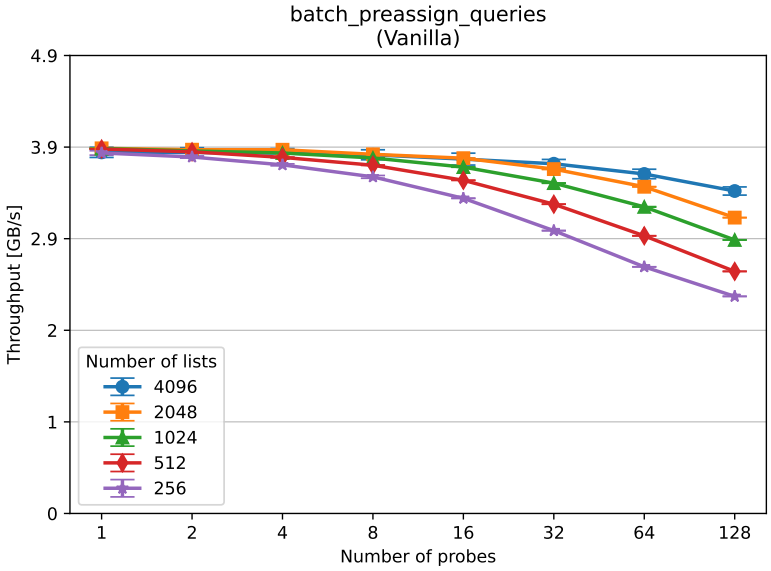


Figure 5.10

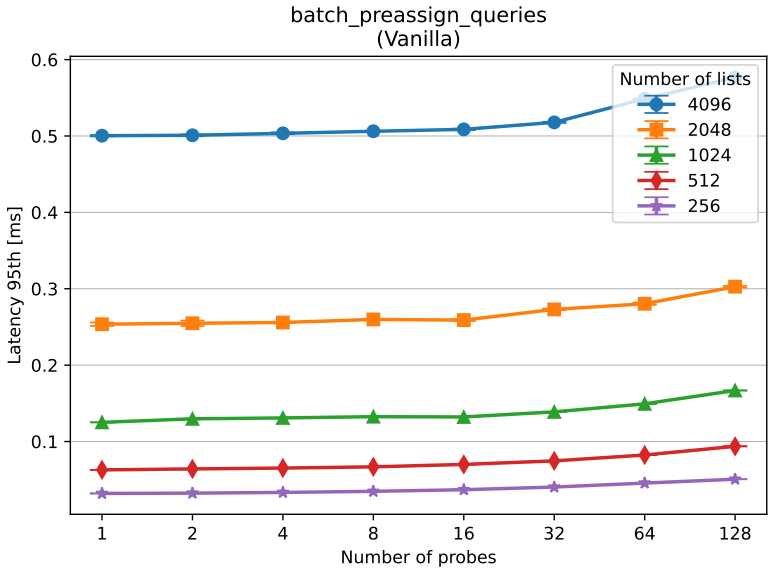


Figure 5.11

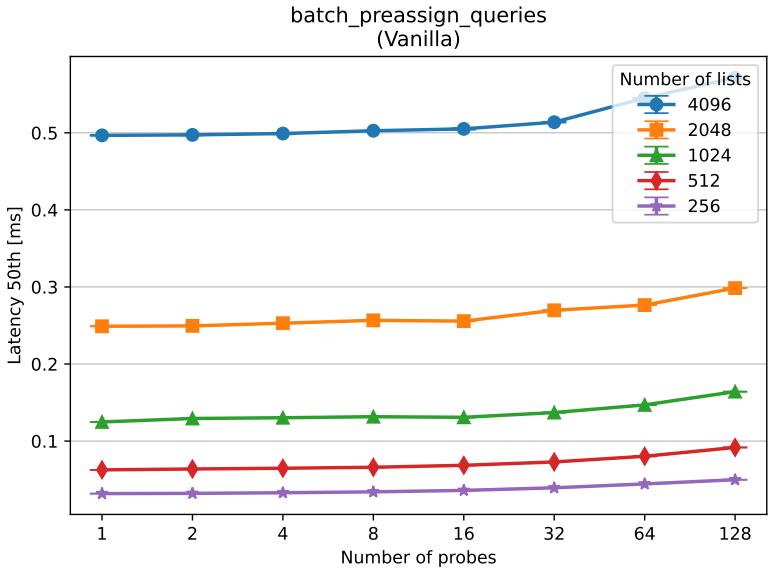


Figure 5.12

SIMD (no OpenMP)

- **Preassign Throughput** (figure 5.16): With SIMD, we first notice a significant increase in throughput compared to the vanilla version, now reaching up to 18 GB/s compared to 4 GB/s. The speedup across the different parameters lies between 2.5x and 4x (figure 5.17). Interestingly, with an increasing number of probes, the throughput speedup decreases

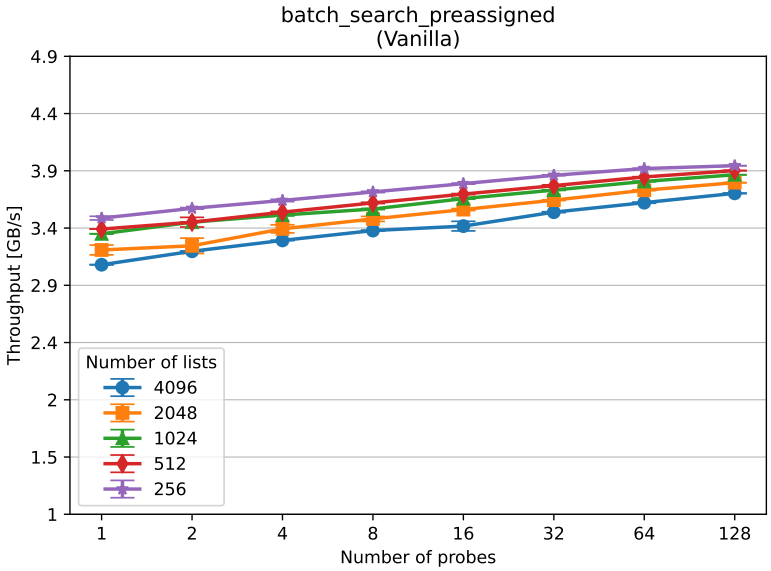


Figure 5.13

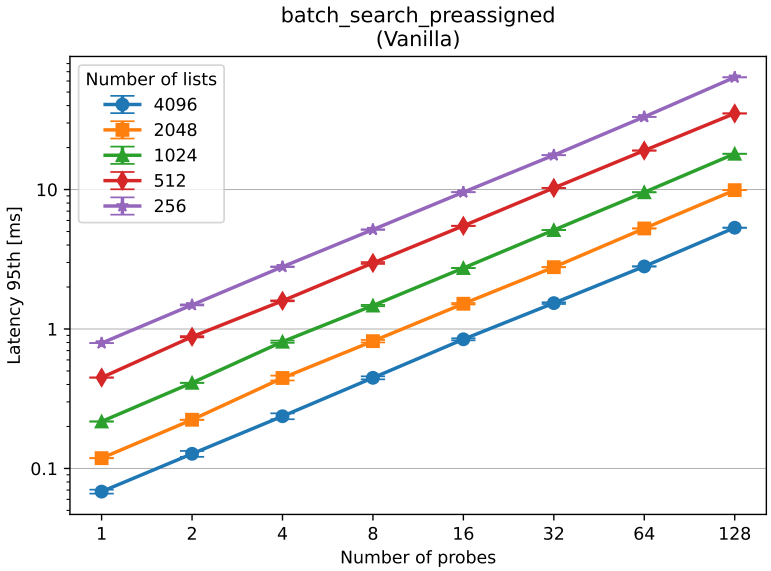


Figure 5.14

from 4x to about 3x. Similarly to the vanilla case, this is likely caused by large overheads of heap maintenance operations required when many centroid candidates need to be tracked, reducing the relative impact of SIMD acceleration on the distance computations. As we have seen before, the throughput increases with increasing the number of lists due to an increase in search space.

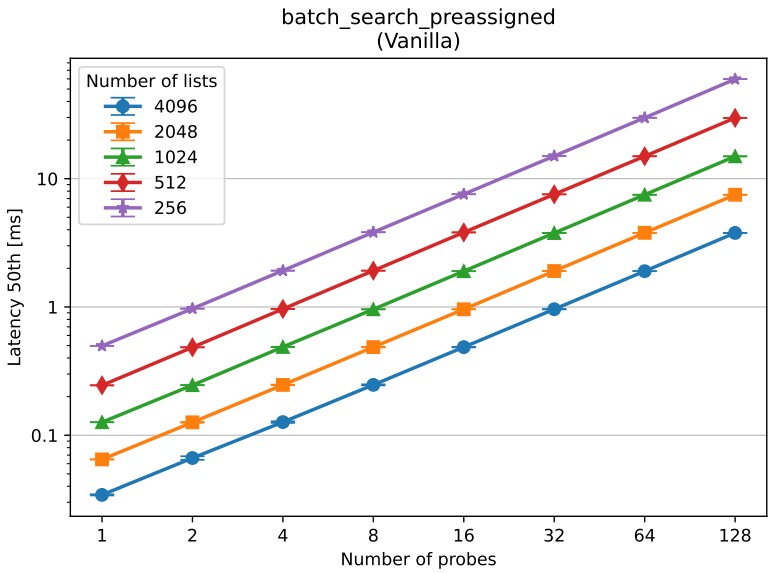


Figure 5.15

- **Preassign Latency** (figures 5.18, 5.19): The latency trends are the same as in the vanilla version, but both median and 95th percentile latencies are now significantly lower, dropping by a factor between 2.5x and 4.5x, similarly to the throughput (figure 5.20). These speedups roughly coincide with the expected performance gains when using SIMD instructions, showing that SIMD acceleration speedups are expected to be high for tasks like large-scale distance computations.
- **Search Throughput** (figure 5.21): The search throughput also surpasses 10 GB/s for a high number of probes, varying between 8 and 11 GB/s. The trend that the throughput increases when increasing the number of probes is also present and slightly more pronounced. With SIMD, we achieve a consistent speedup of 2.8x compared to the vanilla version (figure 5.22). Other than in the centroids search where the number of probes relates to the heap size, for the search through the lists on the storage node we have fixed k and therefore do not notice significant changes in speedup as the number of probes increases.
- **Search Latency** (figures 5.23, 5.24): The 95th percentile latency now ranges between around 80 us and 7ms, for 1024 lists, which is an improvement of around 2.8x (figure 5.25). We notice an outlier for 64 probes. Initially, we thought it could be caused by interference but since the benchmarks run all probes for a fixed number of lists in sequence, this is very unlikely. We have not found an explanation for this behavior, especially since the speedup recovers when the number of probes is 128.

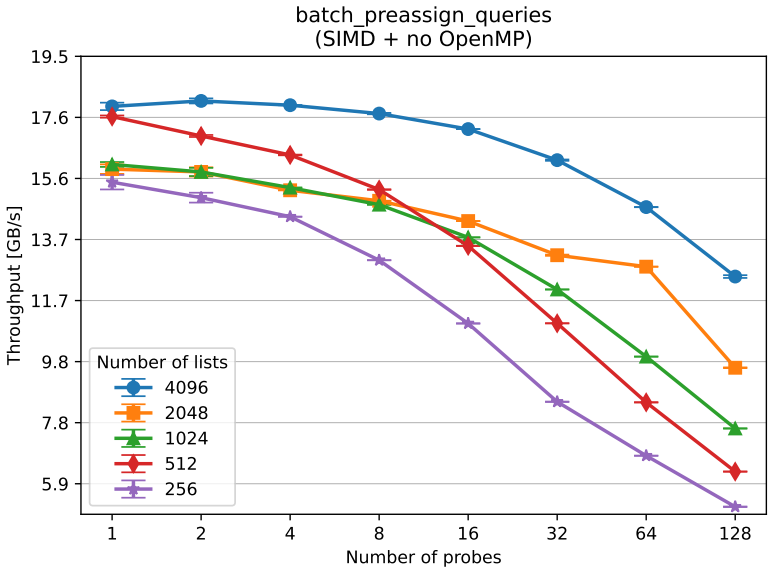


Figure 5.16

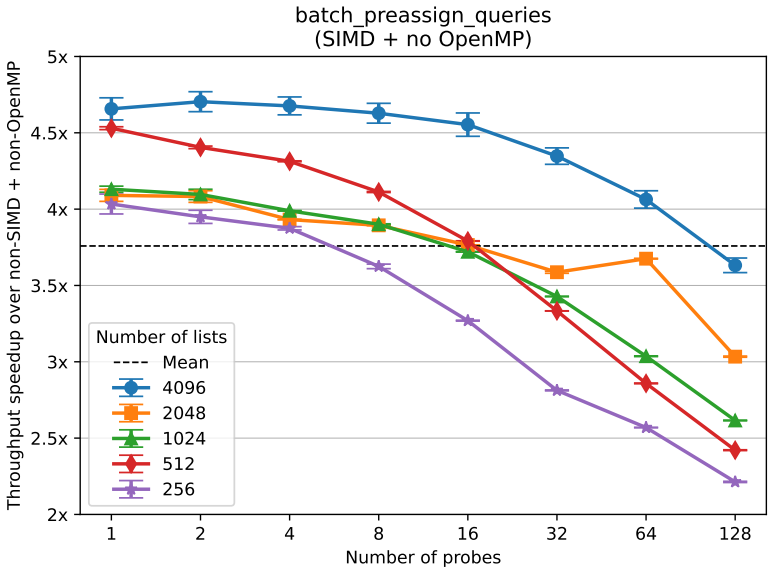


Figure 5.17

OpenMP Scheduling Policy and Number of Threads

- Before we start discussing how the two parallelization modes we implemented, namely OpenMP over queries and OpenMP over queries and lists, affect the performance of the algorithm, we want to look into the effect of the scheduling policies and the number of threads on the system's performance. These parameters can be tuned with OpenMP

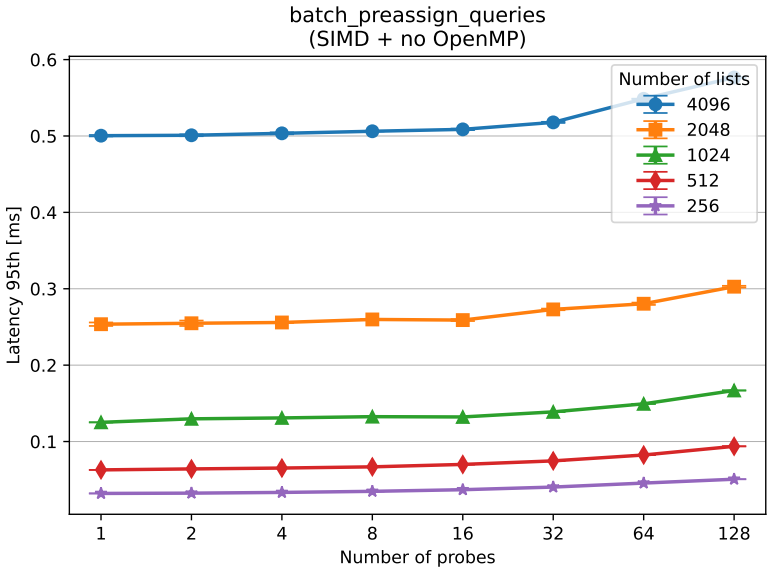


Figure 5.18

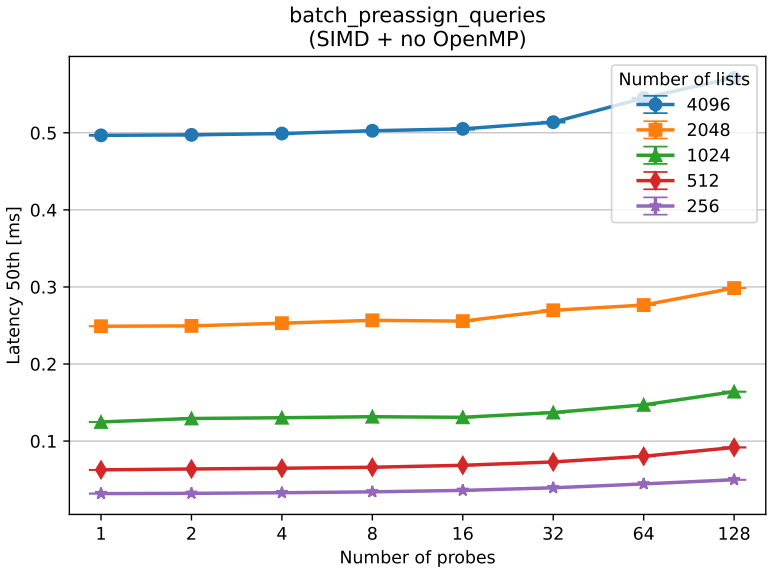


Figure 5.19

at runtime using library functions or by setting environment variables or fixed using compiler directives. For this experiment, we have fixed the parallel mode to be 2, i.e. we parallelize over both queries and lists (see 5.2.1). Also, we only consider the performance of search at the storage node since the performance of the preliminary root node is less relevant for our system.

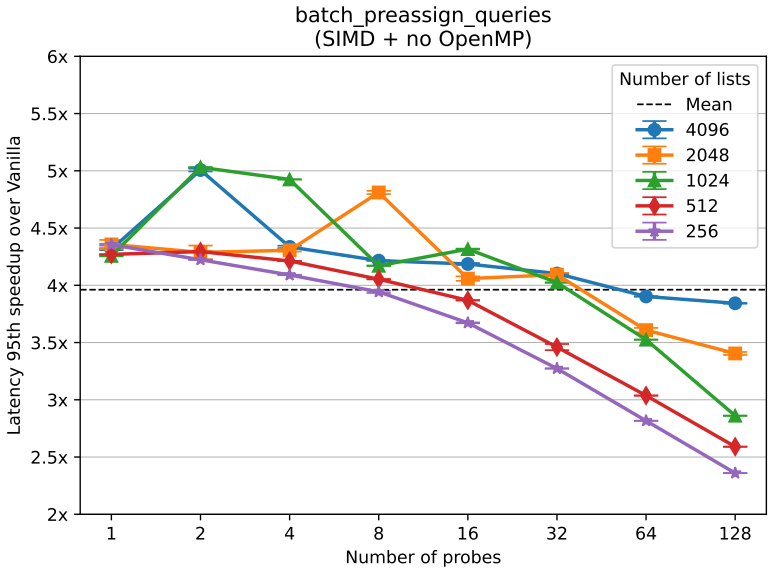


Figure 5.20

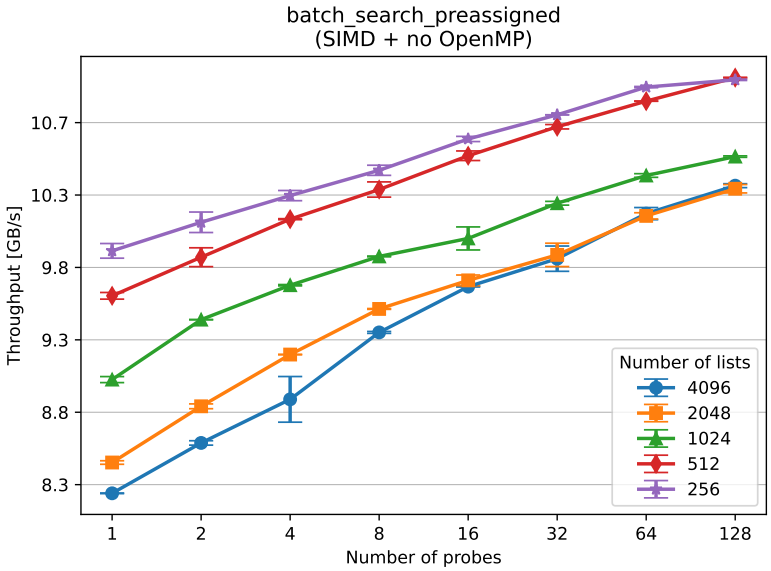


Figure 5.21

- **Throughput** (figure 5.26): The most noticeable change is a dramatic increase in throughput, now reaching up to 50 GB/s. We also see that independent of the scheduling policy, the throughput saturates as expected. Comparing the different scheduling policies, we see that dynamic scheduling performs best, peaking for 32 threads. We assume this is the case because the workload is not evenly distributed,

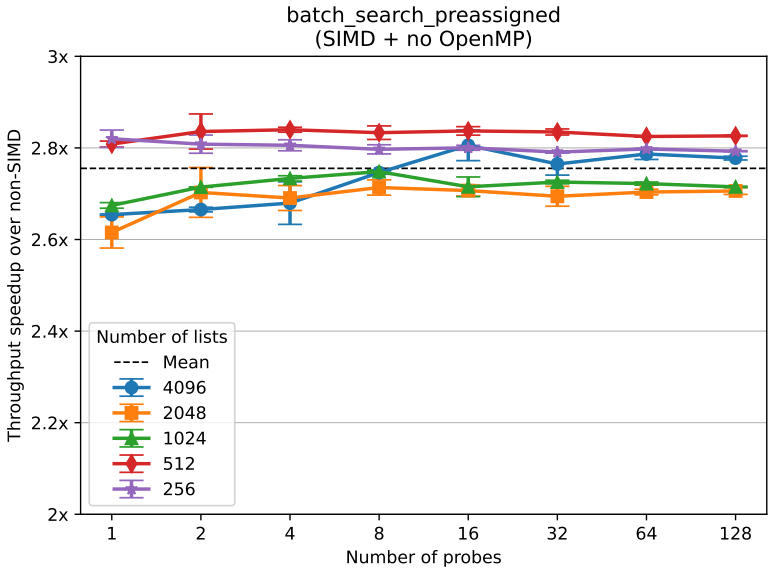


Figure 5.22

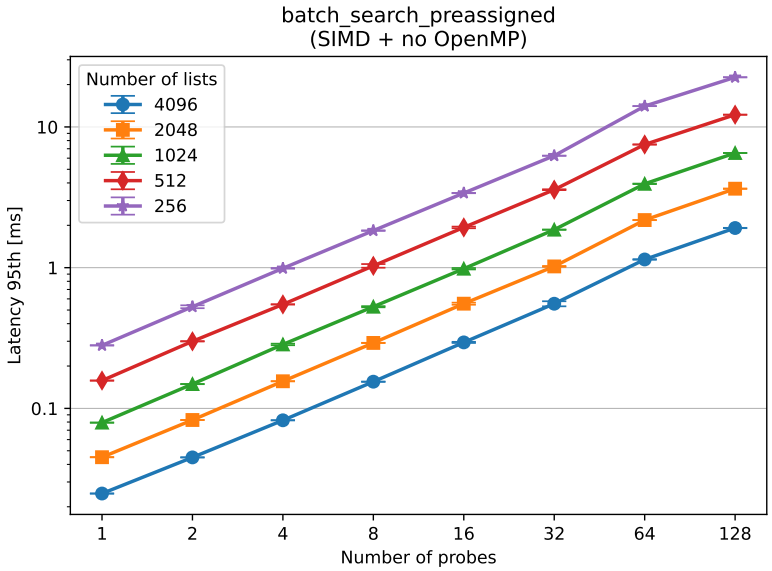


Figure 5.23

meaning the list sizes vary. Dynamic scheduling allows threads to continue processing lists when they complete scanning their list early. Further, the fact that the throughput peaks at 32 threads aligns with the hardware configuration of our machine which has 16 cores each with 2 threads. We also notice that static and guided scheduling perform worse and peak earlier at 4 and 8 threads, respectively and

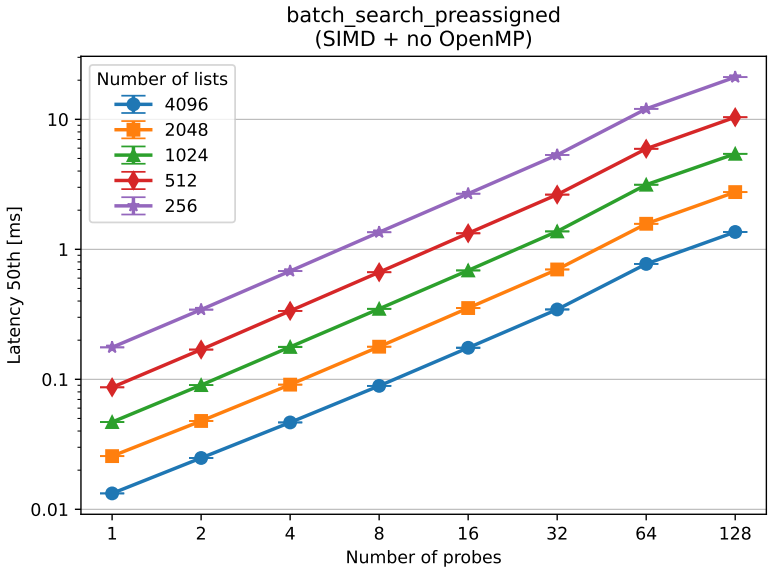


Figure 5.24

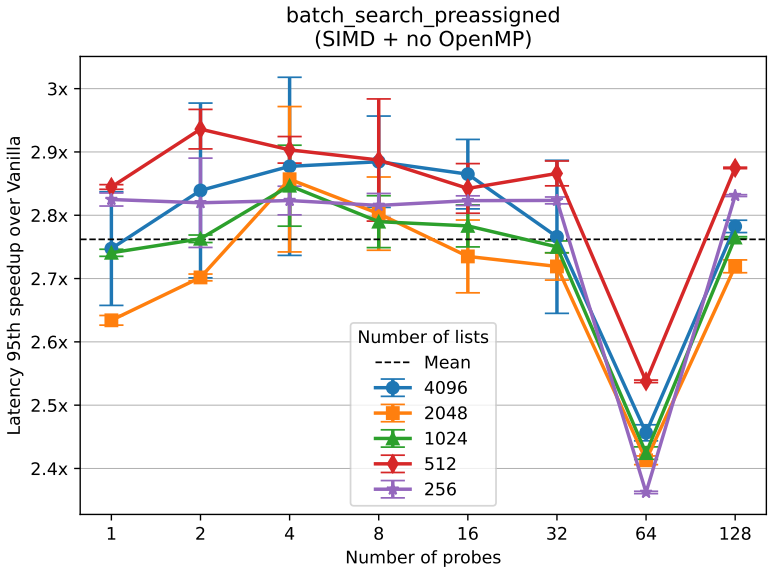


Figure 5.25

their performance drops until they reach 32 threads. We presume that this is caused by the above-mentioned uneven distribution of workload for each thread. Although work is also distributed on demand for the guided schedule, work items are initially set to be too large, likely resulting in idle threads. Similarly, for a static assignment of tasks, threads may become idle while others are still busy processing

their long lists. The reason why the throughput recovers for these two schedules after increasing the thread count beyond 32, could be a change in the scheduling implementation when the thread count surpasses the number of hardware threads. Another explanation could be that individual threads get assigned smaller tasks as the number of threads increases. A high thread count, however, will occur a large latency penalty, as we will see next.

- **Latency** (figures 5.27, 5.28): In this experiment, we also notice a trend in latency which is common when the number of threads is increased. Initially, when using up to 4 threads, we notice a drop in latency, then as we increase the number of threads further and the latency explodes exponentially. This behavior is caused by the high maintenance, scheduling and interference overheads of threads which become more noticeable when we set the thread count too high.
- **Choosing a scheduling policy and the number of threads**: Based on these insights, we can already conclude that the dynamic scheduling policy is superior to the other. Although we have left the configurable chunk size at one, we do not expect the performance to increase as larger chunks imply more lists to search for an individual thread, leading to a loss in efficiency. We choose to set a service level agreement to limit the 95th latency under 1 ms. Based on figure 5.27, this qualifies thread counts between 2 and 32. Figure 5.29 shows the achievable throughput for this range. Since we aim to maximize the throughput under the given latency constraint, we choose the dynamic scheduling policy with 32 threads which achieves the highest throughput among all the other configurations within the chosen latency bound.

OpenMP over Queries

- **Preassign Throughput** (figure 5.30): As we already noticed in the discussion about optimization of the scheduling policies, we now play in a different performance league thanks to multithreading enabled by OpenMP. For the centroids search, we reach a throughput of over 250 GB/s. However, this experiment only measures the performance of the best-case configuration, meaning we benchmarked the centroids search parallelized over the queries, which only needs to access a single list of centroids that reside on the heap in a compact form and which suffers from little thread interference. Although our current implementation of the root index will not be part of the final system due to poor scaling for a large number of lists, it may still serve as a reference for what performance gains can be achieved in the best scenario. As we can see in figure 5.31, we achieve a remarkable average speedup of about 20x over the SIMD version. This showcases how OpenMP

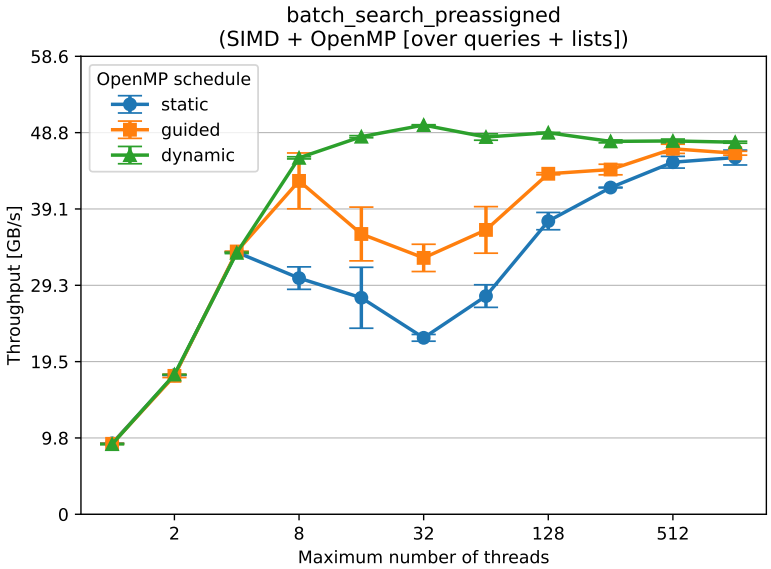


Figure 5.26

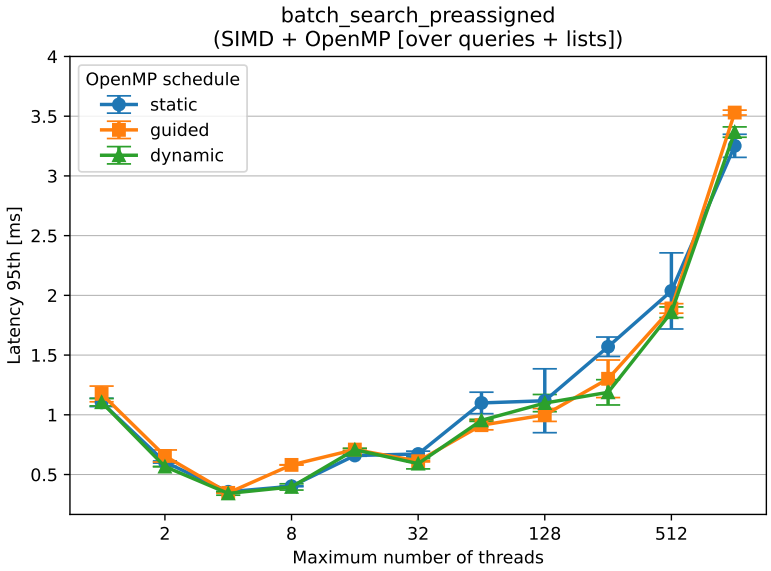


Figure 5.27

can efficiently exploit the available hardware resources and achieve a great performance boost when the conditions are favorable. Lastly, we want to mention the single outlier for 4096 lists which only achieves a throughput improvement between 16x and 18x when the number of probes is less than 64. We assume that this is caused by the fact that the number of lists is directly proportional to the number of centroids

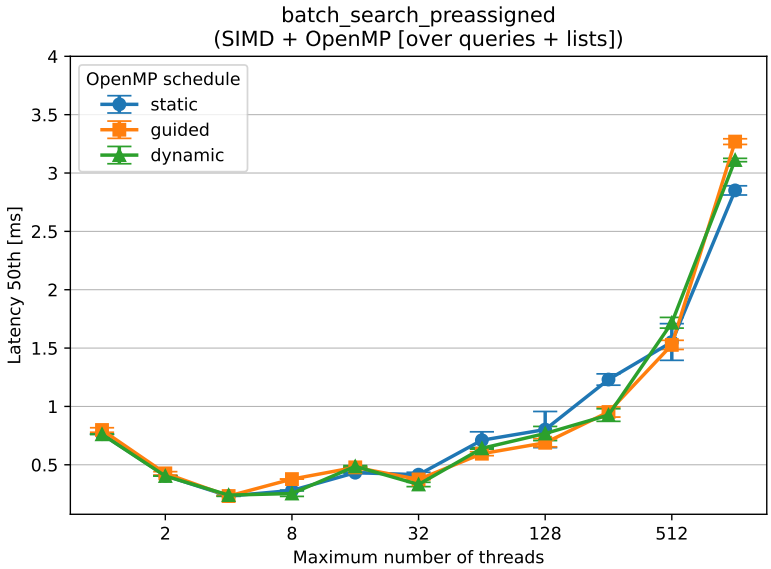


Figure 5.28

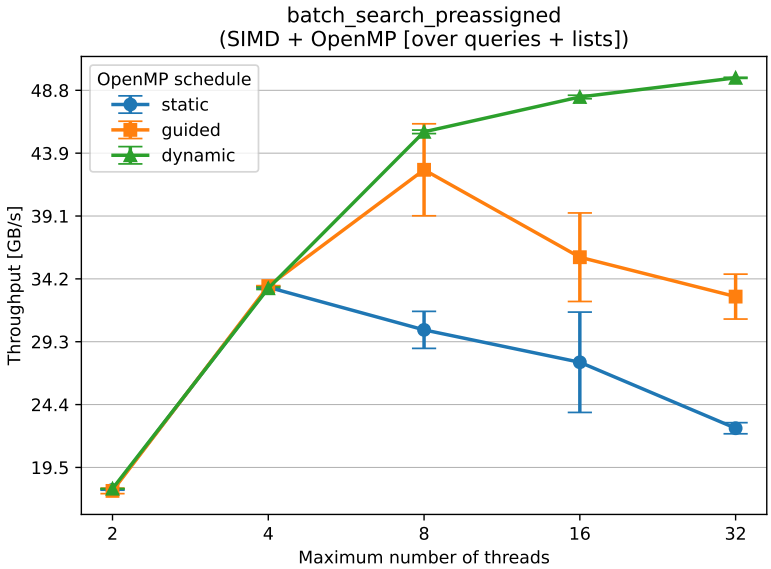


Figure 5.29

and the size of the search space (figure 5.8). Specifically, the number of centroids that need to be searched no longer fit in the lower-level cache, causing more cache evictions while processing the set of queries.

- **Preassign Latency** (figures 5.32, 5.33): When examining the impact on latency when we parallelize the centroids search over queries, we see the drawbacks of introducing more multithreading to a latency-critical

system. Although we observe a degradation of latency compared to the SIMD version, leading to a slow-down of 0.75x (figure 5.34), we the absolute latency is still low with most configurations achieving below 100 us of 95th latency. The degradation compared to the SIMD version is likely due to the overhead in the scheduling and managing threads, which becomes more evident in a single query scenario.

- **Search Throughput** (figure 5.35): Although the increase is not as dramatic as we have seen for the centroids search, we still manage to achieve a throughput of around 5x (figure 5.36) over the SIMD version. The lower speedup compared to the centroid search is likely due to the worse locality of the data which is distributed all over the memory-mapped file and an overall much higher working set (figure 5.9). As a result, the operations are more influenced by the memory bandwidth which is shared among the threads, leading to less parallelization potential.

- **Search Latency** (figures 5.37, 5.38): Similarly to the throughput case, introducing threads also leads to worse search latency, but the degradation is much more pronounced due to the same reasons we mentioned above, performing more than 5x worse, when comparing it to the SIMD version (figure 5.39). We want to highlight another trend that becomes obvious. For the case where the number of lists is high, meaning a smaller working set size, we see that the latency is less affected by introducing threads to the system. This is likely due to the above-mentioned fact that the memory bandwidth for a smaller working set is less saturated, allowing the threads to run more efficiently which partially compensates for the overhead they introduce. We will discuss in the next section how parallelization over both queries and list can help us achieve lower latencies while preserving the throughput gains possible with parallelization over queries.

OpenMP over Queries and Lists

- Since the root node only needs to search a single list of centroids, there is no obvious room for parallelization over the lists in addition to the parallelization over the queries. Therefore, we will only discuss the performance of the search at the storage node when parallelization over queries and lists.

- **Search Throughput** (figure 5.40): Even though parallelizing over the lists required the introduction of critical regions and more thread interference, we see that the throughput is not significantly lower compared to the case where we only parallelize over the queries, approaching a throughout of 50 GB/s. From figure 5.41, we can conclude that we can

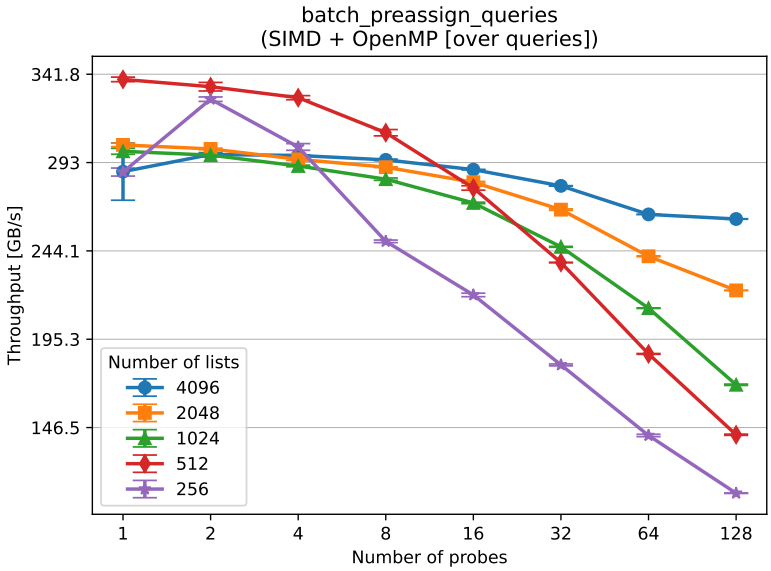


Figure 5.30

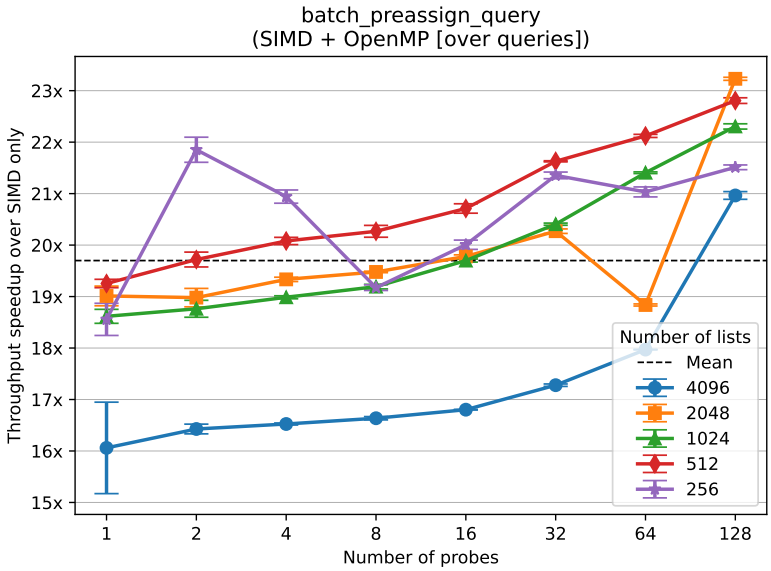


Figure 5.31

maintain close to the same throughput as the version where we only parallelize over the queries.

- **Search Latency** (figures 5.42, 5.43): The reason why we implemented parallelization over both queries and lists becomes more obvious when considering the latency improvements compared to only parallelizing over the queries. Although we can not achieve as low of a latency as

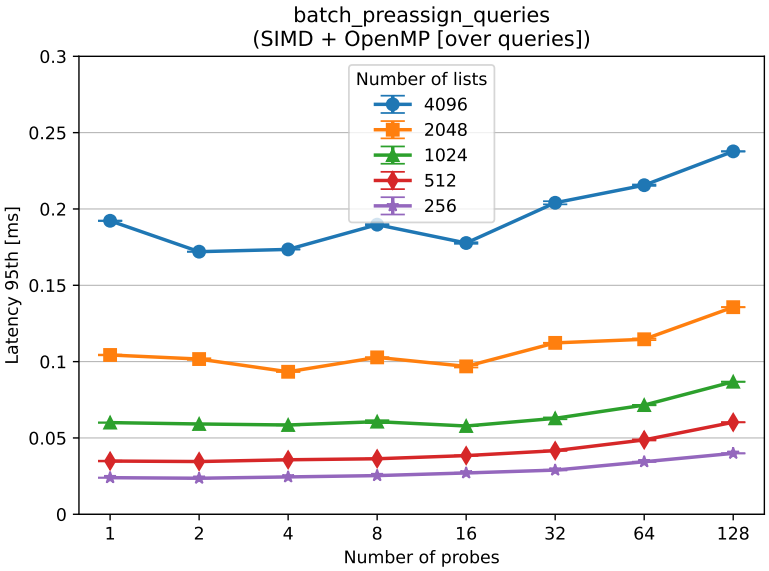


Figure 5.32

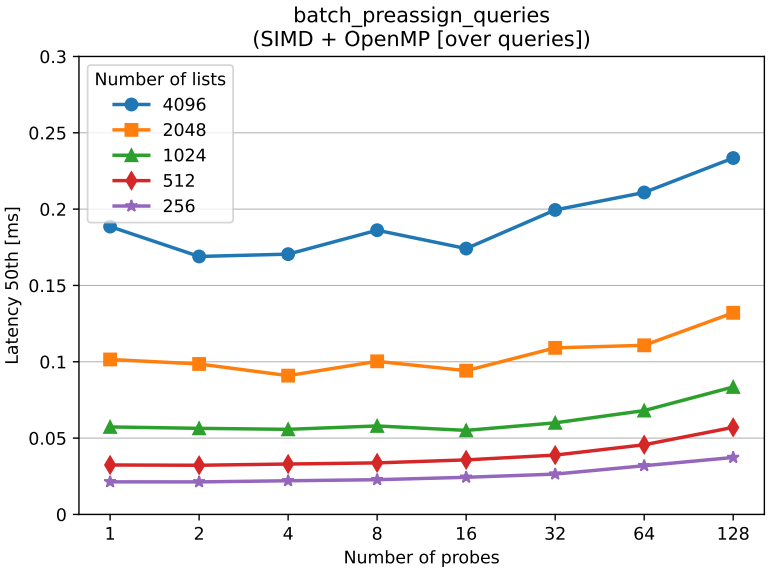


Figure 5.33

did without any multithreading, we can outperform the variant which is only parallelized over queries by a factor ranging between 1x and 5x (figure 5.44). A noticeable trend is that the benefits become greater the higher the number of probes is, especially when surpassing 32 probes. Since this means that more lists per query need to be searched, we effectively parallelize the search time for a single query. This compares

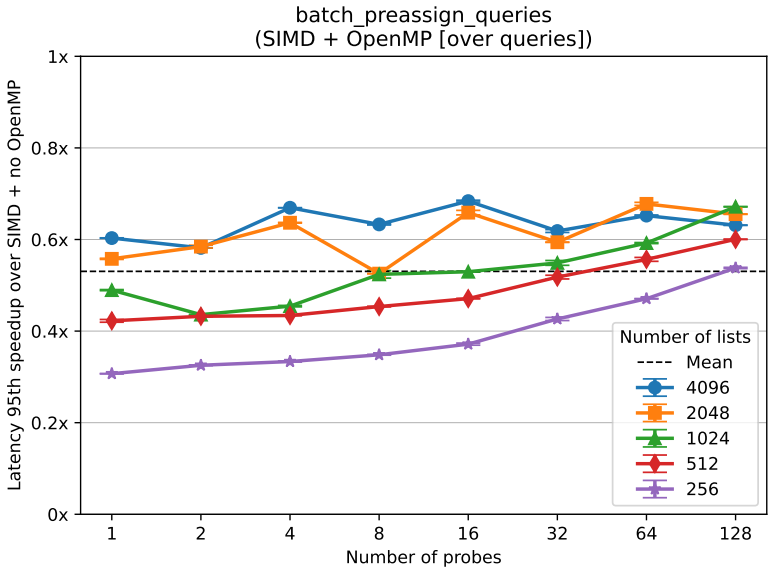


Figure 5.34

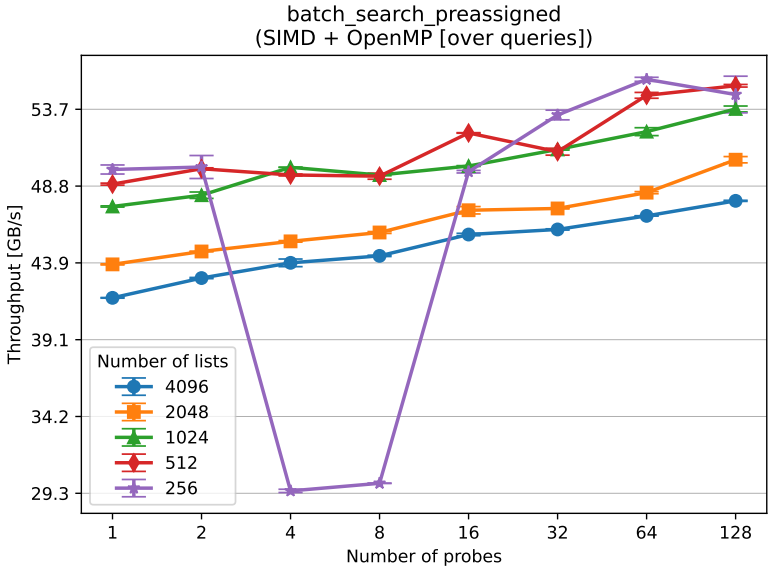


Figure 5.35

to the case where we only parallelize over queries, which has no benefit for the latency of a single thread. Precisely, introduces only overhead leading to a large latency penalty.

Number of Results

- Finally, we would like to briefly discuss the effect of the number of

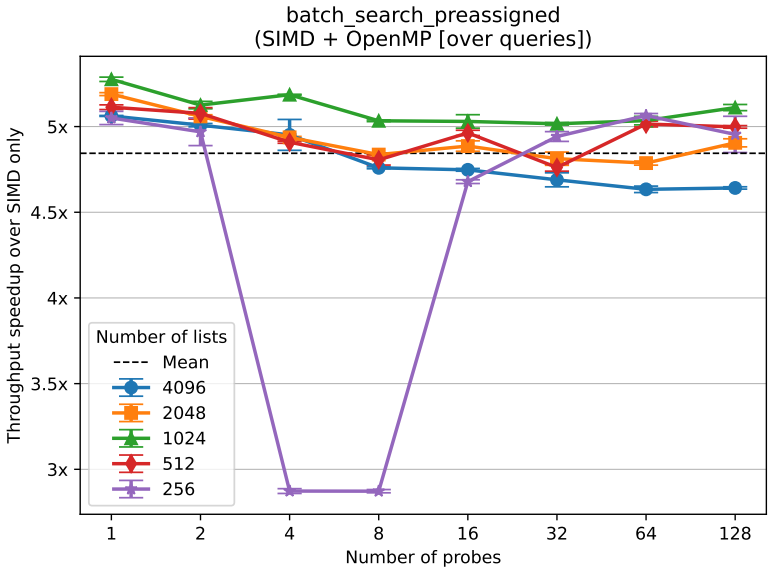


Figure 5.36

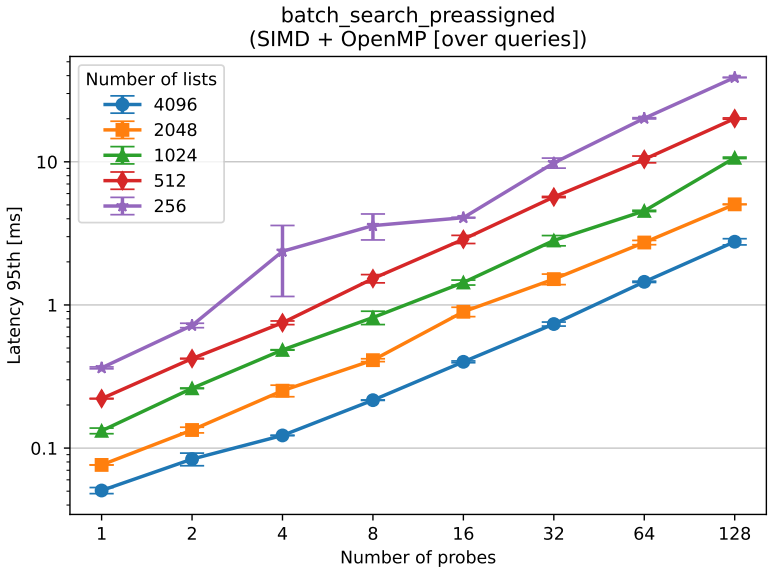


Figure 5.37

results to return k on the performance of our system. For all previous experiments, we have fixed k to be 10 (see 5.2.1), which is a commonly used value in the literature. We will also only discuss the case of search preassigned parallelized over lists and queries, as it is the most relevant configuration.

- **Throughput** (figure 5.45): we see that the throughput is not significantly

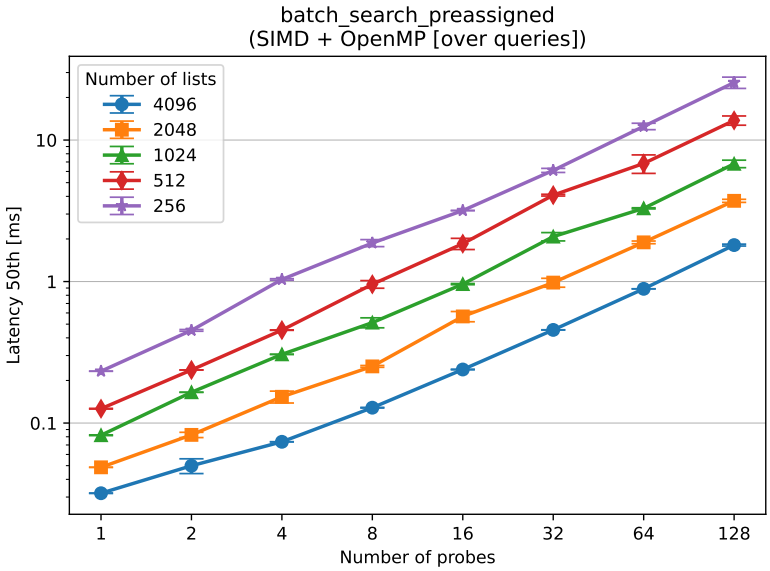


Figure 5.38

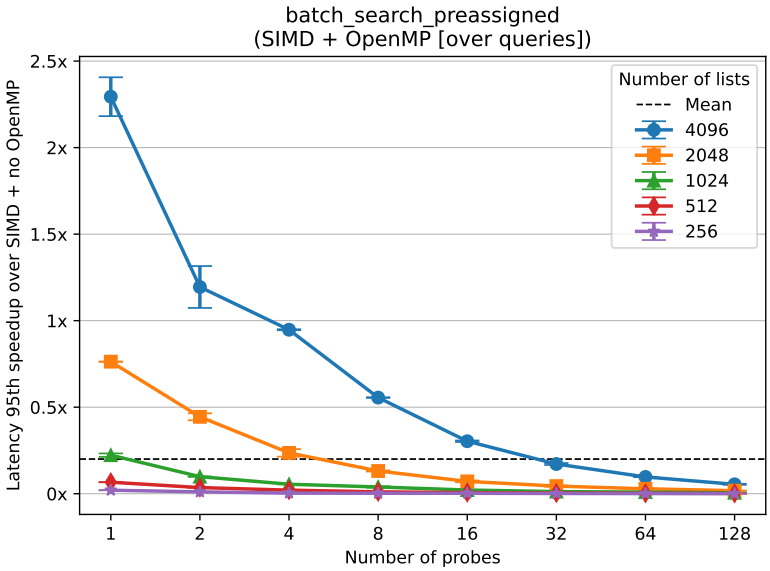


Figure 5.39

affected by the number of results to return. An exception to this is the case for k equal 128, where it severely drops from 50 GB/s down to a range of 30-40 GB/s. Similarly to what we have when varying the number of probes for the centroids search, this behavior is likely caused by the fact that the heaps, which are used to store the nearest neighbor candidates, can no longer be held in the cache and cause thrashing.

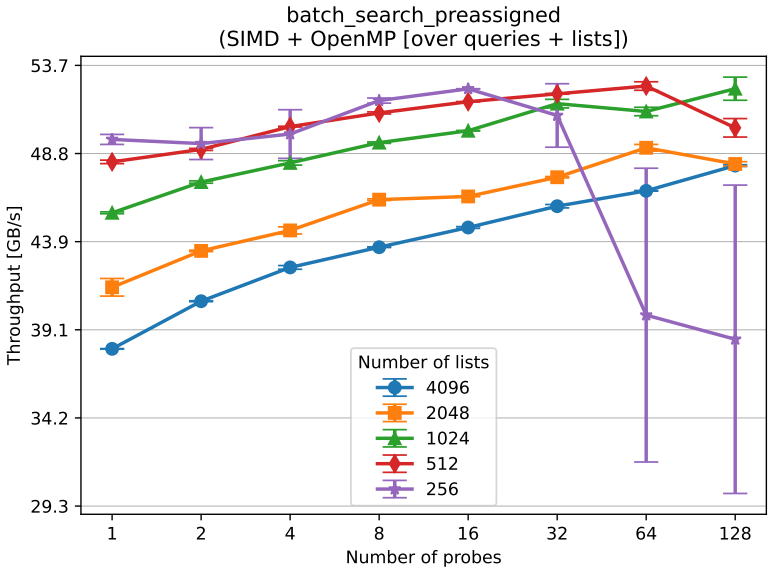


Figure 5.40

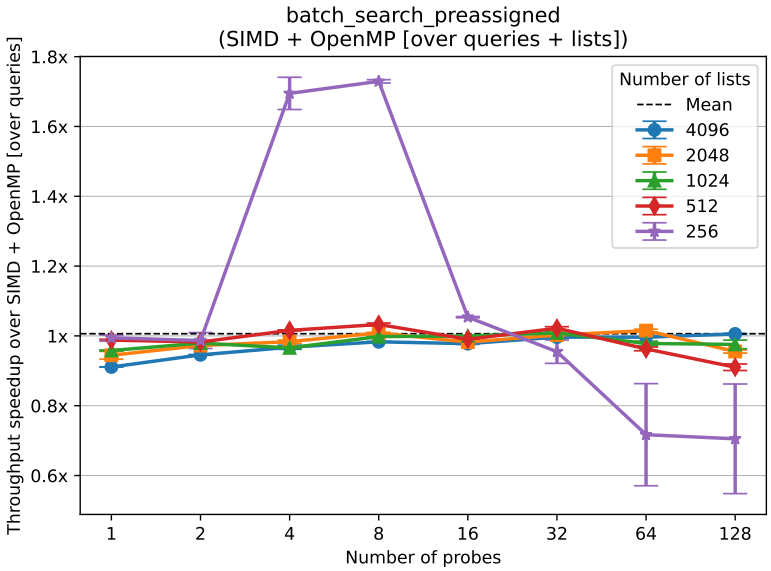


Figure 5.41

We also notice that the decrease in throughput is more pronounced for a lower number of probes. We assume that this is the case because a low number of probes implies a smaller search space which in turn could lead to the heap management overheads being more pronounced relative to the short search time in the case of few probes.

- **Latency** (figures 5.46, 5.47): From the latency plots, we can notice that

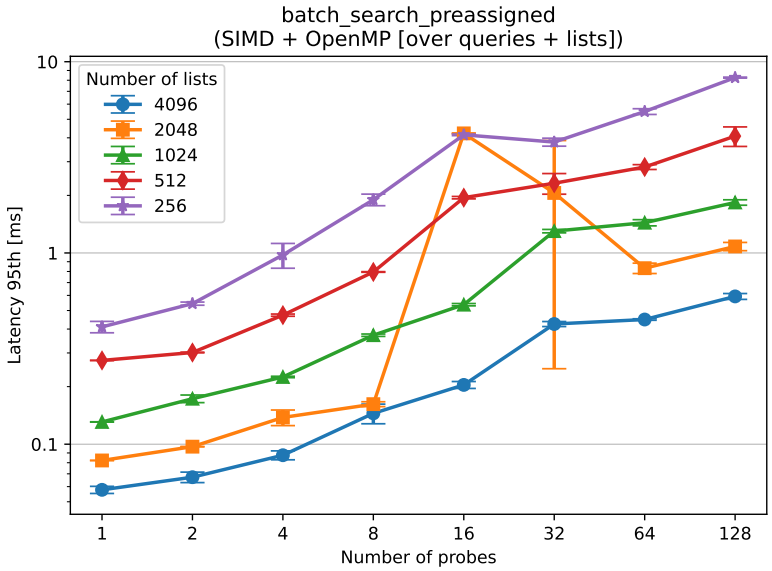


Figure 5.42

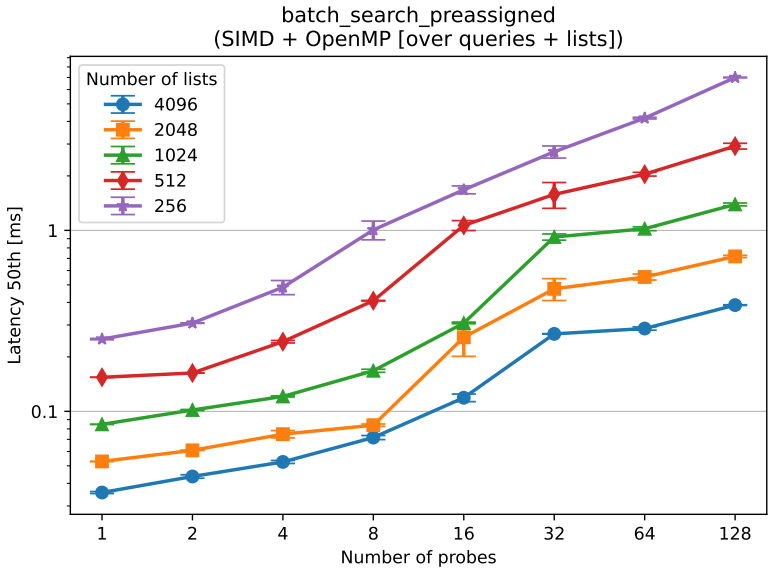


Figure 5.43

the latency is largely unaffected by the number of results to return. The reason why we do not see any decline in latency could be that we only ran the experiments for a single query to measure its latency. Therefore, we do not see the full effect of the heap management overheads at high loads of the system, compared to the throughput experiments where we ran the experiments over all queries of the dataset.

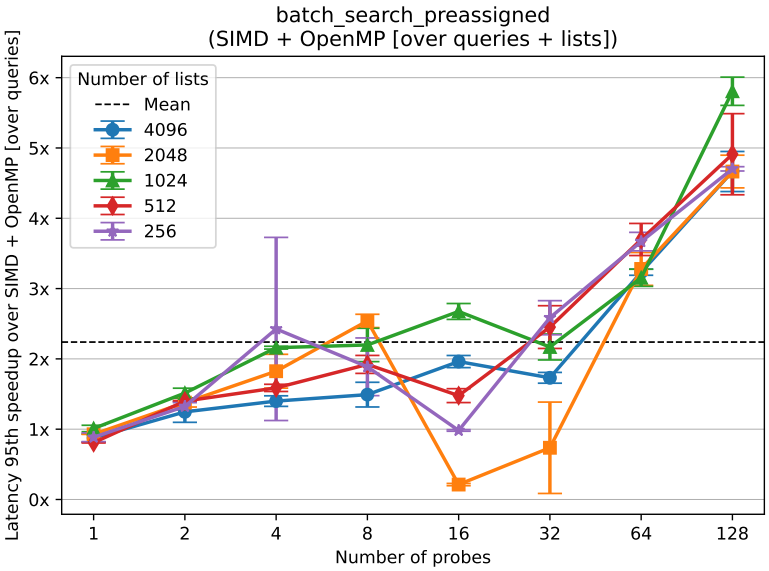


Figure 5.44

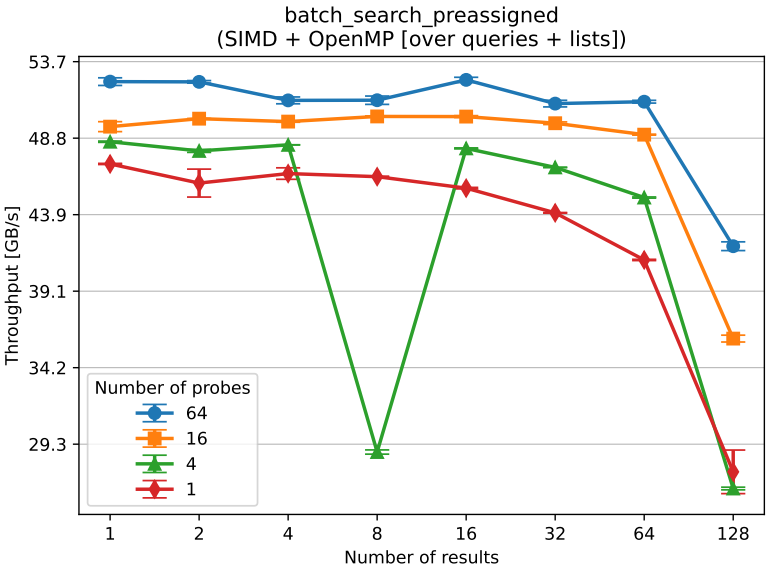


Figure 5.45

Overall Search Performance and Baseline Comparison

- In this section, we would like to summarize the overall results of the search performance evaluation by providing additional figures which show the absolute performance as well as the total speedup we could achieve by combining both the SIMD and `[OpenMP]` improvements. We will only look at the case of search preassigned parallelized over

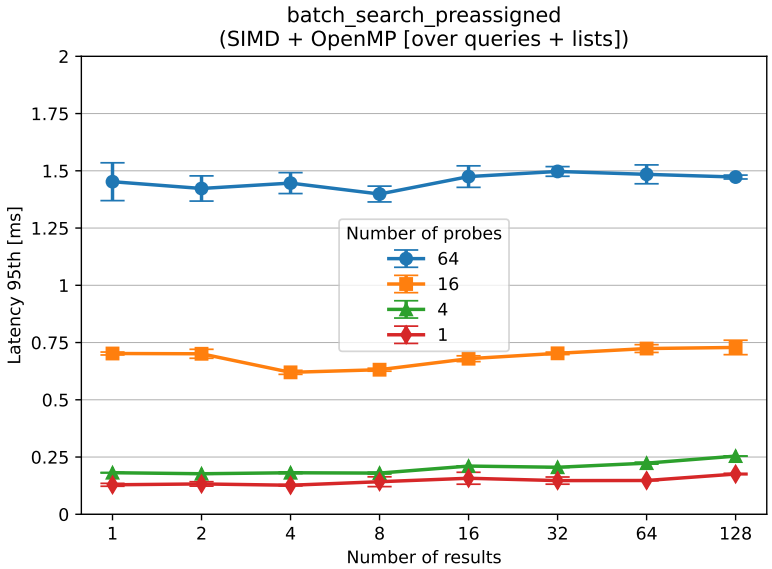


Figure 5.46

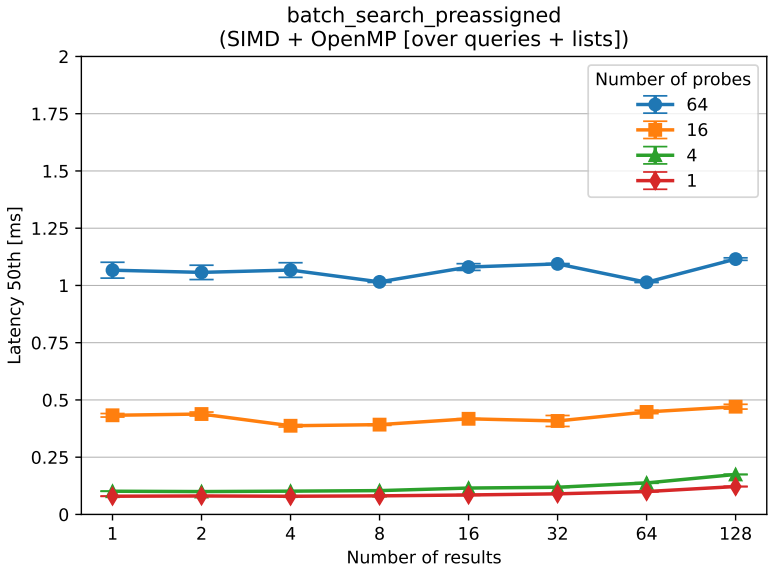


Figure 5.47

lists and queries, as this is the most relevant case for search on the storage node and outperforms the parallelizing over queries in latency while achieving almost the same throughput. We want to note that the runtime of the centroids search when parallelized over queries, is negligible compared to the runtime of the lists search (see figures 5.30, 5.32), which is why we do not consider it in the following discussion.

- **Throughput**: Firstly, we would like to look at the search throughput in terms of queries per second. It is a common metric in information retrieval systems and allows for easier comparisons with other systems. we can see in figure 5.48 that, for 1024 lists and 64 probes, we can achieve a throughput of around 2k queries per second over 128 queries per second in the Vanilla version (figure 5.49), which is speedup of 14x over the vanilla version (figure 5.50).
- **Latency**: We have optimized our system for low latency, achieving a 95th percentile latency of 1.4 ms, for 1024 lists and 64 probes (figure 5.42), compared to our vanilla version which achieves only 9.6 ms. Overall, we achieved an overage latency speedup between 3x and 9x over the vanilla version (figure 5.51).
- **Baseline Comparison**: We also run benchmarks against `FAISS`'s implementation of on-disk inverted index search and came to the following results. Although our implementation does not compete with `FAISS` in terms of throughput, which achieves 5x more throughput for their throughput-optimized mode, we outperforms theirs in terms of latency by at least 3x across all their modes, increasing to 5x when the number of probes surpasses 32. The increase in speedup is because we can exploit more parallelism over the lists for a larger number of probes (see figure 5.51). We have summarized these results for 1024 lists and 128 probes in the table below (see table 5.3).

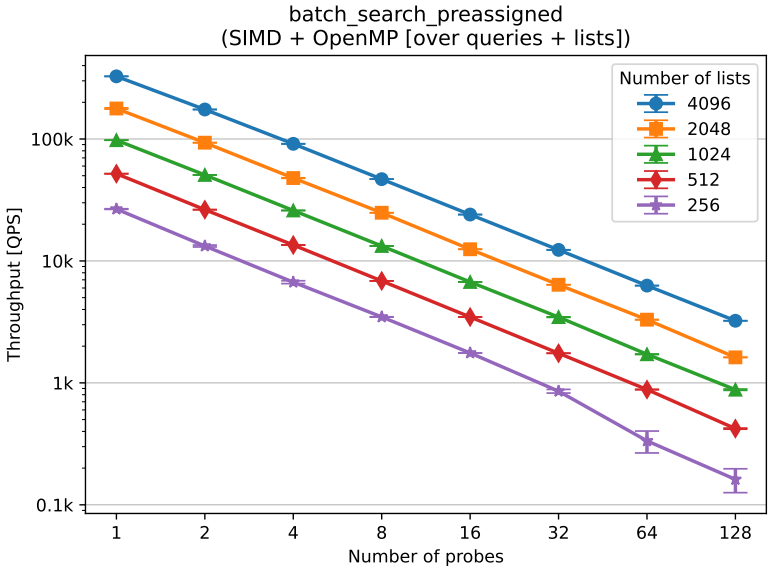


Figure 5.48

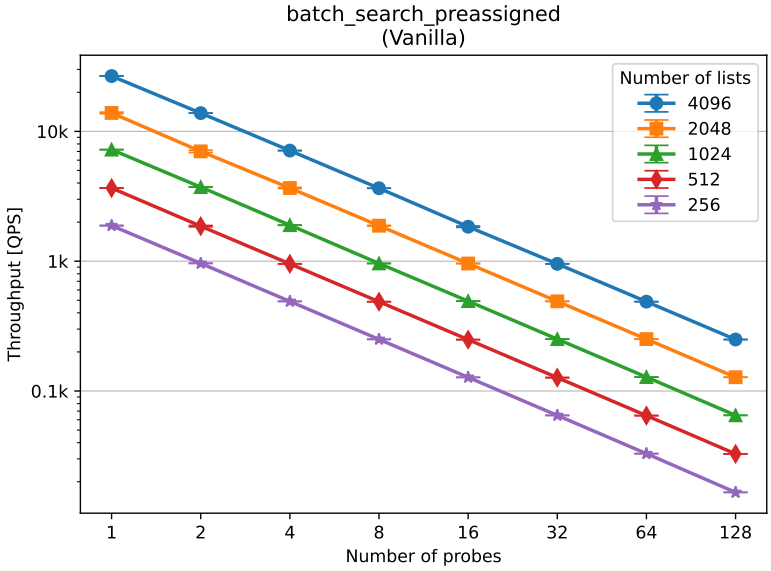


Figure 5.49

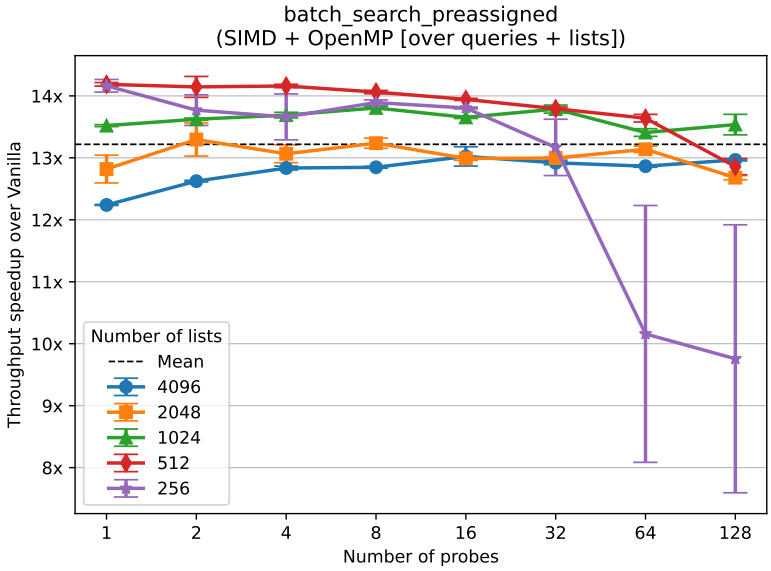


Figure 5.50

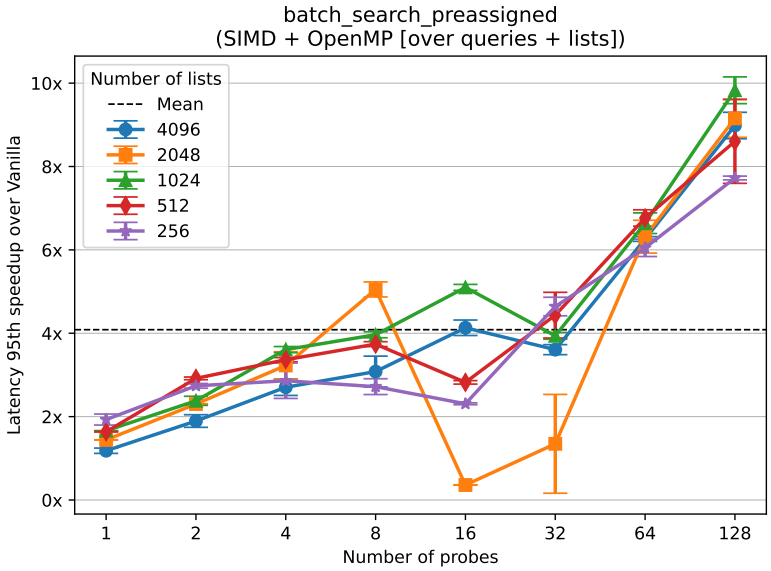


Figure 5.51

System	Throughput	95th Percentile Latency	Median Latency
FAISS (mode 0)	5k QPS	11.6 ms	9.2 ms
FAISS (mode 1)	2k QPS	9.8 ms	7.0 ms
FAISS (mode 2)	2k QPS	9.2 ms	6.3 ms
Ours	1k QPS	1.8 ms	1.4 ms

Table 5.3: Baseline comparison with `FAISS` on `SIFT1M` for `n_lists`=1024, `n_probe`=128, `k`=10. mode 0 (default): parallelize over queries. mode 1: parallelize over lists. mode 2: parallelize over queries and lists. Note that, according to our measurements, `FAISS`'s implementation of their latency-optimized modes only shows significant effects when the number of probes is larger than or equal to 128. We refer to section 6.1 for more details on the baseline comparison.

Chapter 6

Limitations and Future Work

6.1 Limitations

Root Node As outlined in 1.2, we only implemented a scan-based index which does scale well for a high number of lists (see 6.2). Further, when considering the overall distributed design, one could argue that a single root node will not be able to handle all client requests, and would be costly due to high resource requirements. Although this critique is justified, we argue that one could reduce the burden of the root node by partitioning its index into multiple nodes and adding another layer that accumulates the results and load balances between the root node instances.

Memory Management With our current implementation, we have offloaded much of the memory management work, especially the paging of parts of the on-disk file, to the operating system. Since it is unwary of our specific access pattern, (see 6.2).

Insertion Speed Although only relevant during index construction, despite great improvements with the dynamic insertion strategy, insertion performance on hard drives is still low, as it is I/O bound with the random access writes occurring during insertion to the list files (see 6.2).

Performance Baseline We only provided a performance baseline comparison with a single system for a single configuration. Due to time constraints, we were not able to add more extensive plots, e.g. comparing throughput and latency speedups between the systems while varying the number of probes. We will add these mission results in the future and make them available on our repository.

Benchmarking We only chose to benchmark run each benchmark with three samples, excluding another warmup run. Despite this, we observed low errors for most measurements. Further, due to the overall high runtime in the order of hours for index construction of large datasets and a large space of parameters to benchmark for the search, running more samples would lead to an impractically long runtime. Another issue in the latency measurements is that they were not measured under the full load of the system. Real-life scenarios will have a higher load on the system, which might lead to higher latencies.

Correctness Tests Although we used efficient constraint random testing to verify the performance of our system which allowed us to greatly improve the system's robustness, the parameter ranges are still limited and may not cover all edge cases. Further, to test the correctness of the indices, we have only conducted recall tests that compare the ground truth of the first nearest neighbor. Although this should already be enough to verify the searching behavior, one could extend these tests to make use of the remaining available ground truth data.

Thread Safety To be able to support updating the lists while querying, we need to ensure thread safety. However, so far we have not implemented any locking mechanisms, restricting us from extending lists while querying.

Dataset Size Although bulk insertion was evaluated up to datasets of 1 billion, the search is only evaluated for 1 million. We made this decision because at query time only a small fraction of the dataset is accessed. Especially when we will have implemented a buffer pool manager, frequently used lists will already be cached in memory and the performance should not be affected much by the size of the dataset stored on disk.

6.2 Future Work

Graph Index and Scaling Beyond A Single Server The next step in extending our current work would be implementing a graph-based index for the root node based on HNSW instead of our current scan-based approach. Further, one would explore HNSW-related parameters on the performance of the system, e.g. the maximum number of links per node or construction parameters like the maximum number of levels in the graph. Further, we would need to extend the interface of the storage and root node to be able to communicate over the network, e.g. using gRPC. Finally, we would need to implement the necessary features at the root node to be able to handle client requests, distribute them to the storage nodes, gather the results and handle redundancy.

Buffer Pool Manager Despite its ease of use, relying on the operating system's `mmap` to access the on-disk file for storing the lists of vectors also has some downsides. Although the operating system may decide to page in sections of the file as a form of caching, it is unaware of the access pattern specific to the application. Implementing a custom buffer pool manager as found in modern database systems would outperform the paging by the operating systems and give better performance guarantees for query execution.

Support for Storing In-Memory Datastructures on Disk So far, only the list data itself is preserved when shutting down the system. One could make use of accessible C++ libraries such as `BOOST`, to also preserve the metadata of the lists and free slots by storing our in-memory data structures on disk.

SPIMI-like Diskfriendly Insertion Algorithm As we have demonstrated in our experiments, the insertion strategy has a large impact on the performance of the insertion. We have shown that the random access pattern which occurs for current implementations exhibits poor performance on hard drives. To improve the performance, we could implement a SPIMI-like algorithm which would allow us to write the lists sequentially to the disk. This would be enabled by constructing intermediary indices in memory where random writes are less expensive, writing each index to disk where the individual lists are read from memory ordered by their ids. This would only require random reads in memory, as well as in-memory sorting of list id-pointer pairs. Finally, we would then merge the intermediary indices in a final index on disk in a single pass. This approach would build on other extensions to our implementation, such as being able to load and store the in-memory data structures on disk.

Exploring More Optimization Potential We believe that there is still more parallelization potential to be explored when it comes to the search. For example, when parallelizing over both queries and lists, we have not explored the effect of parallelizing the preparation of work items for the threads or the extraction of the results. Further, we have not explored the effect of fine-grained locking, e.g. only locking the critical section for threads trying to merge the same heaps for the same query but allowing other threads to continue. Further, one could make use of threading to prefetch lists into memory. Finally, tools like `perf` or `valgrind` could be further used to find bottlenecks and potential memory leaks to improve both the performance and the robustness of the system.

List Management Extensions Our current implementation of the lists could be extended to support the deletion of lists and vectors, which would also

make implementing the ability to shrink the memory-mapped region more sensible. Deletion of lists could be implemented either by direct deallocations which would incur significant overhead or through the use of bitmaps to mark deleted lists or vectors. To reduce fragmentation when supporting the deletion of lists or entries, the more sophisticated merging of free slots and optimization algorithms could improve locality in the long term. Alternatives to that are simply rebuilding the index after a sufficient amount of updates incurring fragmentation have been performed.

Distance Metrics and Further Evaluation To measure similarity, we currently only support performing euclidean distance computations. However, other metrics such as cosine similarity or inner product could also be implemented to make our system attractive for more use cases. In terms of evaluating the system, measuring disk operations or fragmentation could give more insights into the behavior of the system during index construction. Further, we could also extend the scope of our evaluations, for example by measuring the performance of updating entries. However, this was out of the scope of our work.

Chapter 7

Conclusion

To summarize, we have achieved our goal of building a system implementation for large-scale approximate nearest neighbor search in the out-of-memory, single-server setting. We designed and developed the storage node of the distributed system which is responsible for the management of the lists of vectors that reside on disk as well as providing an interface for the root node of the system for efficient low-latency nearest neighbor search. Further, we implemented a scan-based root index that selects the lists to probe based on the centroids of each dataset partition obtained from clustering.

During our extensive evaluation of the system, we showed that we could achieve significant performance optimization across index construction and query processing. Firstly, by implementing a superior on-pass insertion strategy we could speed up the insertion process by 3-5x, depending on the dataset size. Based on our experiments about the memory access pattern which occurs during insertion, we showed that there is more optimization potential, which could be exploited to speed up the insertion by another 3-5x and proposed an algorithm to make use of it.

Secondly, we showed that SIMD and OpenMP can be effectively used to speed up large-scale searches in our setting. SIMD enabled speedups of 2.8x for both latency and throughput, while OpenMP achieved another 5x for throughput and between 1x and 3x for latency, depending on the number of probes. Combined, we achieved speedups of 14x for throughput and improved latencies by 3-9x.

Further, we have shown the performance effects of parallelizing over the search queries as well as the lists and compared the different scheduling policies while varying the number of threads. Finally, we compared our implementation with `FAISS`, a state-of-the-art system for similarity search,

and showed that although they outperform us in terms of throughput by a factor of 5x, we can achieve significantly lower latencies, achieving an improvement of 3-6x over their system.

Chapter 8

Acknowledgements

First of all, I would like to thank my supervisor, Wenqi Jiang, for his great guidance and support throughout and for allowing me to be part of this project on the cutting edge of approximate nearest neighbor search. Secondly, I want to thank Prof. Gustavo Alonso and the entire Systems group for providing me with this opportunity and welcoming me openly into the group. Last but not least, I would also like to thank my friends and family for their support and encouragement throughout the project.

Chapter 9

Appendix

9.1 Project Structure

For your convenience, we have compiled a file tree of the most relevant project files (4.1). They also include references to the section describing the relevant functionalities. What follows are listings of most source files of this project divided into the following categories:

- System implementation
- Testing and benchmarking
- Evaluation
- Preprocessing
- Prototype

The full source code is available at https://github.com/TimonFercho/ann-distributed-key-value-store.

9.2 System implementation

- include/storage-node/StorageLists.hpp
- src/storage-node/StorageLists.cpp
- include/storage-node/StorageIndex.hpp
- src/storage-node/StorageIndex.cpp
- include/root-node/RootIndex.hpp
- src/root-node/RootIndex.cpp
- include/Query.hpp
- src/Query.cpp
- include/L2Space.hpp
- src/L2Space.cpp
- include/types.hpp

```
1  #pragma once
2
3  #include <unordered_map>
4  #include <vector>
5  #include <string>
6
7  #include "types.hpp"
8
9  #define PROT_READ 0x1
10 #define PROT_WRITE 0x2
11 #define FLAG_READ "r"
12 #define FLAG_READ_WRITE "r+"
13
14 #ifndef MIN_TOTAL_SIZE_BYTES
15 #define MIN_TOTAL_SIZE_BYTES 32
16 #endif
17 #ifndef MIN_N_ENTRIES_PER_LIST
18 #define MIN_N_ENTRIES_PER_LIST 1
19 #endif
20 #ifndef MAX_BUFFER_SIZE
21 #define MAX_BUFFER_SIZE 10000
22 #endif
23 #ifndef DYNAMIC_INSERTION
24 #define DYNAMIC_INSERTION 1
25 #endif
26
27 namespace ann_dkvs
28 {
29   class StorageLists
```

```
30  {
31  private:
32    /**
33     * Contains meta-data about an inverted list in the
          memory-mapped file.
34     *
35     * - offset: offset in bytes relative to base_ptr
36     * - allocated_entries: maximum number of entries
37     * that the list can hold without having to be
          extended.
38     * - used_entries: number of entries currently
39     *  in use and containing valid data (vectors and
          vector ids)
40     */
41    struct InvertedList
42    {
43      size_t offset;
44      len_t allocated_entries;
45      len_t used_entries;
46    };
47
48    /**
49     * Contains meta-data about a free slot of memory in
          the memory-mapped file
50     * which can be used to allocate new inverted lists
          or extend existing ones.
51     *
52     * - offset: offset in bytes relative to base_ptr
53     * - size: size in bytes of the free slot
54     */
55    struct Slot
56    {
57      size_t offset;
58      size_t size;
59    };
60
61    /**
62     * A type that represents a map from list ids to
          inverted lists.
63     */
64    typedef std::unordered_map<list_id_t, InvertedList>
        list_id_list_map_t;
65
66    /**
67     * A type that represents a map from list ids to the
          number of entries
68     * to be allocated. Used to pre-allocate inverted
          lists
69     * in the implementation of bulk_insert_entries().
```

```
70          */
71         typedef std::unordered_map<list_id_t, len_t>
               list_id_counts_map_t;
72
73         /**
74          * A type that represents an iterator over the slots
75          * in the free_slots vector used to allocate and free
               slots.
76          */
77         typedef std::vector<StorageLists::Slot>::iterator
               slot_it_t;
78
79         /**
80          * Specifies the location of the memory-mapped file
81          */
82         const std::string filename;
83
84         /**
85          * Specifies the dimension of the vectors stored in
               the inverted lists
86          */
87         const size_t vector_dim;
88
89         /**
90          * Specifies the size of the vectors stored in the
               inverted lists
91          * in bytes.
92          */
93         const size_t vector_size;
94
95         /**
96          * Specifies the size of the mappping region in bytes.
97          */
98         size_t total_size;
99
100        /**
101         * Holds a pointer to the base of the memory-mapped
               region.
102         */
103        uint8_t *base_ptr;
104
105        /**
106         * In-memory data structure that maps list ids to
               inverted list objects.
107         */
108        list_id_list_map_t id_to_list_map;
109
110        /**
111         * In-memory data structure that holds a list of free
```

```
112         slots
113          * in the memory-mapped file.
114          */
115         std::vector<Slot> free_slots;
116
117         /**
118          * Memory-maps the file used to store the inverted
                lists
119          * containing the vectors and vector ids on disk
120          * with read-write permissions.
121          *
122          * @throws std::runtime_error if the file cannot be
                opened or mapped.
123          */
124         void mmap_region();
125
126         /**
127          * Returns a pointer to the first vector within the
                memory region
128          * associated with the given inverted list.
129          *
130          * All vectors of a list are stored contiguously in
                memory.
131          *
132          * @param list A pointer to the inverted list
133          *             for which the vectors are being
                requested.
134          * @return A pointer to the first vector.
135          */
136         vector_el_t *get_vectors_by_list(const InvertedList
            *list) const;
137
138         /**
139          * Returns a pointer to the first vector id within
                the memory region
140          * associated with the given inverted list.
141          *
142          * All vector ids of a list are stored contiguously
                in memory.
143          *
144          * @param list A pointer to the inverted list
145          *             for which the vector ids are being
                requested.
146          * @return A pointer to the first vector id.
147          */
148         vector_id_t *get_ids_by_list(const InvertedList
            *list) const;
149
150         /**
```

```
150         * Returns total size by the given amount of vector
                ids
151         * when they are stored contiguously in memory.
152         *
153         * @param n_entries The number of vector ids.
154         * @return The total size in bytes.
155         */
156        size_t get_ids_size(const len_t n_entries) const;
157
158        /**
159         * Returns total size by the given amount of vectors
160         * when they are stored contiguously in memory.
161         *
162         * @param n_entries The number of vectors.
163         * @return The total size in bytes.
164         */
165        size_t get_vectors_size(const len_t n_entries) const;
166
167        /**
168         * Returns total size by the given amount of list ids
169         * when they are stored contiguously in memory.
170         *
171         * @param n_entries The number of list ids.
172         * @return The total size in bytes.
173         */
174        size_t get_list_ids_size(const len_t n_entries) const;
175
176        /**
177         * Returns total allocated size of the given inverted
                list in bytes.
178         *
179         * @param list A pointer to the inverted list.
180         * @return The total size in bytes.
181         */
182        size_t get_total_list_size(const InvertedList *list)
              const;
183
184        /**
185         * Returns a slot reprensenting the memory region
                associated
186         * with the given inverted list.
187         *
188         * Used when the inverted list is to be freed.
189         *
190         * @param list A pointer to the inverted list.
191         * @return A slot of the location and size of the
                inverted list.
192         */
193        Slot list_to_slot(const InvertedList *list);
```

```
194
195     /**
196      * Performs the necessary steps to grow the
             memory-mapped region
197      * to the given size.
198      *
199      * Adjusts the size of the file on disk and modifies
             the
200      * list of free slots accordingly.
201      *
202      * @param new_size The new size of the memory-mapped
             region.
203      * @throws std::runtime_error if new_size < total_size
204      *          or if there is an error creating, opening
             or resizing the file
205      *          or during un- or remapping of the file.
206      */
207     void resize_region(const size_t new_size);
208
209     /**
210      * Allocates a new inverted list with the given
             number of entries.
211      *
212      * The number of entries being allocated is always a
             power of two
213      * and at least as large as the given used number of
             entries.
214      *
215      * @param n_entries The number of entries to be
             allocated.
216      * @return An inverted list object representing the
             allocated memory region.
217      */
218     InvertedList alloc_list(const len_t n_entries);
219
220     /**
221      * Allocates a new slot of the given size.
222      *
223      * The memory region is grown if necessary. If a slot
             larger than the given
224      * size is found, the inverted list is placed at the
             beginning of the slot
225      * and the remaining space is added to the list of
             free slots.
226      *
227      * @param size The size of the slot to be allocated.
228      * @return The offset of the allocated slot relative
             to the base pointer.
229      */
```

```
230     size_t alloc_slot(const size_t size);
231
232     /**
233      * Creates a free slot at the location and with the
             size of the given slot.
234      *
235      * The slot is added to the list of free slots. If
             the slot is adjacent
236      * to one or two other slots, the slots are merged.
237      *
238      * @param slot A pointer to the slot to be freed.
239      */
240     void free_slot(const Slot *slot);
241
242     /**
243      * Finds the next power of two that is greater than
             or equal
244      * to the given number.
245      *
246      * @param n The number to be rounded up.
247      * @return The next power of two that is greater than
             or equal to n.
248      */
249     size_t round_up_to_next_power_of_two(const size_t n)
           const;
250
251     /**
252      * Checks if the list of free slots has a slot that
             extends to the end
253      * of the memory-mapped region.
254      *
255      * @return True if there is a free slot at the end of
             the region.
256      */
257     bool has_free_slot_at_end() const;
258
259     /**
260      * If a file is already memory-mapped, it unmaps it.
261      * If not, it creates a new file without
             memory-mapping it
262      * in case the file does not already exist.
263      */
264     void ensure_file_created_and_region_unmapped() const;
265
266     /**
267      * Resizes the file to the given size.
268      *
269      * If the size is smaller than
270      * the current size, the file is truncated.
```

```
271         * If the size is larger than the current size,
272         * the file is extended and padded with zeros.
273         *
274         * @param size The new size of the file.
275         */
276        void resize_file(const size_t size);
277
278        /**
279         * Checks if the given inverted list needs to be
               reallocated.
280         *
281         * Reallocation is necessary unless the list uses
               between 50% and 100%
282         * of the allocated entries.
283         *
284         * @param list A pointer to the inverted list.
285         * @param new_length Number of entries the list
               should be able to hold.
286         * @return True if the list needs to be reallocated.
287         */
288        bool does_list_need_reallocation(const InvertedList
              *list, const len_t new_length) const;
289
290        /**
291         * Copy the entries of one inverted list to another.
292         *
293         * Only as many entries as the smaller list holds are
               copied.
294         *
295         * @param dst A pointer to the destination inverted
               list.
296         * @param src A pointer to the source inverted list.
297         */
298        void copy_shared_data(const InvertedList *dst, const
              InvertedList *src) const;
299
300        /**
301         * Iterates over the list of free slots and finds the
               first slot
302         * that is large enough to hold data of the given
               size.
303         *
304         * @param size The minimum size of the slot to be
               found in bytes.
305         * @return An iterator to the slot.
306         */
307        slot_it_t find_large_enough_slot_index(const size_t
              size);
308
```

```
309     /**
310      * Grows the memory-mapped region until it is large
             enough to hold
311      * at least the given number of bytes.
312      *
313      * The size of the region is always a power of two
             and has at least
314      * a minimum size of MIN_TOTAL_SIZE_BYTES bytes.
315      *
316      * @param size The minimum size of the region to in
             bytes.
317      */
318     void grow_region_until_enough_space(size_t size);
319
320     /**
321      * Finds the next slot to the right of the given slot.
322      *
323      * If the given slot is the last slot, the end
             iterator is returned.
324      *
325      * @param slot A pointer to the slot.
326      * @return An iterator to the next slot to the right.
327      */
328     slot_it_t find_next_slot_to_right(const Slot *slot);
329
330     /**
331      * Finds the next slot to the left of the given slot.
332      *
333      * If the given slot is the first slot, the end
             iterator is returned.
334      *
335      * @param next_slot_right An iterator to the slot to
             the right
336      *                        of the slot to be found.
337      * @return An iterator to the next slot to the left.
338      */
339     slot_it_t find_next_slot_to_left(const slot_it_t
           next_slot_right);
340
341     /**
342      * Checks if the given slots are adjacent.
343      *
344      * Two slots are adjacent if the right slot starts
345      * at the end of the left slot.
346      *
347      * @param slot_left A pointer to the left slot.
348      * @param slot_right A pointer to the right slot.
349      * @return True if the slots are adjacent.
350      */
```

```
351     bool are_slots_adjacent(const Slot *slot_left, const
            Slot *slot_right) const;
352
353     /**
354      * Converts an iterator to a slot to a pointer to the
             slot.
355      *
356      * If the iterator is the end iterator, a null
             pointer is returned.
357      *
358      * @param it An iterator to a slot.
359      * @return A pointer to the slot.
360      */
361     Slot *to_slot(const slot_it_t it) const;
362
363     /**
364      * Grows the memory-mapped region until it is large
             enough to hold
365      * at least the given number of entries.
366      *
367      * @param n_entries The minimum number of entries
368      *                  the region should be able to hold.
369      */
370     void reserve_space(const len_t n_entries);
371
372     /**
373      * Preallocates inverted lists given a file of list
             ids.
374      *
375      * @param entries_left A pointer to the map that
             should be populated
376      *                     with the number of entries of
             each list.
377      * @param list_ids_filename The name of the file
             containing the list ids.
378      * @param n_entries The number of entries in the file.
379      */
380     void bulk_create_lists(list_id_counts_map_t
            &entries_left, const std::string
            &list_ids_filename, const len_t n_entries);
381
382     /**
383      * Creates a file stream for the given file.
384      *
385      * @param filename The name of the file.
386      * @return A file stream for the file.
387      */
388     std::ifstream open_filestream(const std::string
            &filename) const;
```

```
389
390   public:
391     /**
392      * Creates a new storage lists object.
393      *
394      * @param vector_dim The dimension of the vectors.
395      * @param filename The name of the file to be used
             for storage.
396      */
397     StorageLists(const len_t vector_dim, const
           std::string &filename);
398
399     /**
400      * Destroys the storage lists object.
401      *
402      * Unmaps the memory-mapped region.
403      */
404     ~StorageLists();
405
406     /**
407      * Returns the number of inverted lists that are
             currently stored.
408      *
409      * @return The number of lists.
410      */
411     len_t get_length() const;
412
413     /**
414      * Returns the total size of the memory-mapped region.
415      *
416      * @return The total size of the region in bytes.
417      */
418     size_t get_total_size() const;
419
420     /**
421      * Returns the total number of bytes that are
             currently free across
422      * all slots.
423      *
424      * @return The number of free bytes.
425      */
426     size_t get_free_space() const;
427
428     /**
429      * Returns the size of the largest continuous free
             space.
430      *
431      * @return The size of the largest continuous free
             space in bytes.
```

```
432         */
433         size_t get_largest_continuous_free_space() const;
434
435         /**
436          * Returns the total size of the vectors that can be
                stored.
437          *
438          * @return The size of a single vector in bytes.
439          */
440         size_t get_vector_size() const;
441
442         /**
443          * Returns the dimension of the vectors that can be
                stored.
444          *
445          * @return The dimension of the vectors.
446          */
447         len_t get_vector_dim() const;
448
449         /**
450          * Returns the name of the file used to store the
                inverted lists.
451          *
452          * @return The name of the file.
453          */
454         std::string get_filename() const;
455
456         /**
457          * Returns a pointer to the vectors of the given list.
458          *
459          * @param list_id The id of the list.
460          * @return A pointer to the first vector of the list.
461          * @throws std::invalid_argument If the list does not
                exist.
462          */
463         const vector_el_t *get_vectors(const list_id_t
                list_id) const;
464
465         /**
466          * Returns a pointer to the ids of the given list.
467          *
468          * @param list_id The id of the list.
469          * @return A pointer to the first id of the list.
470          * @throws std::invalid_argument If the list does not
                exist.
471          */
472         const vector_id_t *get_ids(const list_id_t list_id)
                const;
473
```

```
474     /**
475      * Returns the number of entries that are in used in
             the given list.
476      *
477      * @param list_id The id of the list.
478      * @return The number of used entries.
479      * @throws std::invalid_argument If the list does not
             exist.
480      */
481     len_t get_list_length(const list_id_t list_id) const;
482
483     /**
484      * Resizes the given list to the given number of
             entries.
485      *
486      * @param list_id The id of the list.
487      * @param n_entries The new length of the list.
488      * @throws std::invalid_argument If the list does not
             exist.
489      * @throws std::out_of_range If the target length is
             0.
490      */
491     void resize_list(const list_id_t list_id, const len_t
        n_entries);
492
493     /**
494      * Inserts the given entries into the given list.
495      *
496      * @param list_id The id of the list.
497      * @param vectors A pointer to the first vector to
             insert.
498      * @param ids A pointer to the first id to insert.
499      * @param n_entries The number of entries to insert.
500      */
501     void insert_entries(const list_id_t list_id, const
        vector_el_t *vectors, const vector_id_t *ids,
        const len_t n_entries);
502
503     /**
504      * Updates the given entries in the given list.
505      *
506      * @param list_id The id of the list.
507      * @param vectors A pointer to the first vector to
             update.
508      * @param ids A pointer to the first id to update.
509      * @param n_entries The number of entries to update.
510      * @param offset The number of entries to skip before
             starting to update.
511      * @throws std::invalid_argument If the list does not
```

```
512         exist.
          * @throws std::out_of_range If the entries to be
              updated are out of bounds.
513       */
514       void update_entries(const list_id_t list_id, const
              vector_el_t *vectors, const vector_id_t *ids,
              const len_t n_entries, const size_t offset) const;
515
516       /**
517        * Creates a new inverted list
518        * and allocates space for the given number of
              entries.
519        *
520        * @param list_id The id of the list.
521        * @param n_entries The number of entries to allocate.
522        * @throws std::invalid_argument If the list already
              exists.
523        * @throws std::out_of_range If the number of entries
              is 0.
524        */
525       void create_list(
526           const list_id_t list_id,
527           const len_t n_entries);
528
529       /**
530        * Inserts the given entries into inverted lists with
              the given ids
531        * where vectors, vector ids and list ids are stored
              in separate files.
532        *
533        * The binary files are expected to be little-endian
              where values
534        * are expected to be stored consecutively in the
              following format:
535        * - vectors: vector_dim * sizeof(vector_el_t) bytes
              per vector,
536        *             the vector dimension is to be specified
              in the constructor.
537        * - vector ids: sizeof(vector_id_t) bytes per id
538        * - list ids: sizeof(list_id_t) bytes per id
539        *
540        * @param vectors_filename The name of the file
              containing the vectors.
541        * @param vector_ids_filename The name of the file
              containing
542        *                               the vector ids.
543        * @param list_ids_filename The name of the file
              containing
544        *                            the list ids.
```

```
545       * @param n_entries The number of entries to insert.
546       */
547      void bulk_insert_entries(const std::string
             &vectors_filename, const std::string
             &vector_ids_filename, const std::string
             &list_ids_filename, const len_t n_entries);
548    };
549  }
```

Listing 9.1: include/storage-node/StorageLists.hpp

```
1   #include <sys/mman.h>
2   #include <string.h>
3   #include <string>
4   #include <iostream>
5   #include <unistd.h>
6   #include <fstream>
7
8   #include "StorageLists.hpp"
9
10  namespace ann_dkvs
11  {
12    void StorageLists::mmap_region()
13    {
14      FILE *f = fopen(filename.c_str(), "r+");
15      if (f == nullptr)
16      {
17        throw std::runtime_error("Could not open file " +
              filename);
18      }
19      base_ptr = (uint8_t *)mmap(
20          nullptr,
21          total_size,
22          PROT_READ | PROT_WRITE,
23          MAP_SHARED,
24          fileno(f),
25          0);
26      if (base_ptr == MAP_FAILED)
27      {
28        throw std::runtime_error("Could not mmap file " +
              filename);
29      }
30      fclose(f);
31    }
32
33    vector_el_t *StorageLists::get_vectors_by_list(const
        InvertedList *list) const
34    {
35      return (vector_el_t *)(base_ptr + list->offset);
36    }
```

```
37
38  vector_id_t *StorageLists::get_ids_by_list(const
        InvertedList *list) const
39  {
40    vector_el_t *vector_ptr = get_vectors_by_list(list);
41    vector_id_t *id_ptr = (vector_id_t
          *)((size_t)vector_ptr + vector_size *
          list->allocated_entries);
42    return id_ptr;
43  }
44
45  size_t StorageLists::get_vectors_size(const len_t
        n_entries) const
46  {
47    return n_entries * vector_size;
48  }
49
50  size_t StorageLists::get_ids_size(const len_t
        n_entries) const
51  {
52    return n_entries * sizeof(vector_id_t);
53  }
54
55  size_t StorageLists::get_list_ids_size(const len_t
        n_entries) const
56  {
57    return n_entries * sizeof(list_id_t);
58  }
59
60  size_t StorageLists::get_total_list_size(const
        InvertedList *list) const
61  {
62    len_t n_entries = list->allocated_entries;
63    return get_vectors_size(n_entries) +
          get_ids_size(n_entries);
64  }
65
66  size_t StorageLists::get_free_space() const
67  {
68    size_t free_space = 0;
69    for (auto slot : free_slots)
70    {
71      free_space += slot.size;
72    }
73    return free_space;
74  }
75
76  size_t
        StorageLists::get_largest_continuous_free_space()
```

```
    const
{
  size_t max_free_space = 0;
  for (auto slot : free_slots)
  {
    if (slot.size > max_free_space)
    {
      max_free_space = slot.size;
    }
  }
  return max_free_space;
}

StorageLists::StorageLists(const len_t vector_dim,
    const std::string &filename) : filename(filename),
    vector_dim(vector_dim), vector_size(vector_dim *
    sizeof(vector_el_t)), total_size(0)
{
  if (vector_dim == 0)
  {
    throw std::out_of_range("Vector dimension must be
        greater than 0");
  }
}

StorageLists::~StorageLists()
{
  if (base_ptr != nullptr)
  {
    munmap(base_ptr, total_size);
  }
}

len_t StorageLists::get_length() const
{
  return id_to_list_map.size();
}

size_t StorageLists::get_total_size() const
{
  return total_size;
}

size_t StorageLists::get_vector_size() const
{
  return vector_size;
}

len_t StorageLists::get_vector_dim() const
```

```
121   {
122     return vector_dim;
123   }
124
125   std::string StorageLists::get_filename() const
126   {
127     return filename;
128   }
129
130   bool StorageLists::has_free_slot_at_end() const
131   {
132     if (free_slots.size() == 0)
133       return false;
134     Slot last_slot = free_slots.back();
135     return last_slot.offset + last_slot.size ==
          total_size;
136   }
137
138   void
       StorageLists::ensure_file_created_and_region_unmapped()
       const
139   {
140     if (total_size == 0)
141     {
142       FILE *f = fopen(filename.c_str(), "r+");
143       if (f == nullptr)
144       {
145         f = fopen(filename.c_str(), "w+");
146         if (f == nullptr)
147         {
148           throw std::runtime_error("Could not create file
              " + filename);
149         }
150       }
151       fclose(f);
152     }
153     else
154     {
155       munmap(base_ptr, total_size);
156     }
157   }
158
159   void StorageLists::resize_file(const size_t size)
160   {
161     FILE *f = fopen(filename.c_str(), "r+");
162     if (f == nullptr)
163     {
164       throw std::runtime_error("Could not open file " +
          filename);
```

```
165     }
166     if (ftruncate(fileno(f), size) == -1)
167     {
168       throw std::runtime_error("Could not resize file " +
            filename);
169     }
170     fclose(f);
171   }
172
173   void StorageLists::resize_region(const size_t new_size)
174   {
175     if (new_size < total_size)
176     {
177       throw std::runtime_error("Cannot shrink region");
178     }
179     if (new_size == total_size)
180     {
181       return;
182     }
183     ensure_file_created_and_region_unmapped();
184     size_t size_to_grow = new_size - total_size;
185     if (has_free_slot_at_end())
186     {
187       free_slots.back().size += size_to_grow;
188     }
189     else
190     {
191       Slot new_slot;
192       new_slot.offset = total_size;
193       new_slot.size = size_to_grow;
194       free_slots.push_back(new_slot);
195     }
196     total_size = new_size;
197     resize_file(total_size);
198     mmap_region();
199   }
200
201   bool StorageLists::does_list_need_reallocation(const
        InvertedList *list, const len_t n_entries) const
202   {
203     return n_entries <= list->allocated_entries / 2 ||
          n_entries > list->allocated_entries;
204   }
205
206   void StorageLists::copy_shared_data(const InvertedList
        *dst, const InvertedList *src) const
207   {
208     len_t n_entries_to_copy = std::min(dst->used_entries,
          src->used_entries);
```

```
209     if (n_entries_to_copy == 0)
210     {
211       return;
212     }
213     memcpy(get_vectors_by_list(dst),
            get_vectors_by_list(src),
            get_vectors_size(n_entries_to_copy));
214     memcpy(get_ids_by_list(dst), get_ids_by_list(src),
            get_ids_size(n_entries_to_copy));
215   }
216
217   StorageLists::Slot StorageLists::list_to_slot(const
        InvertedList *list)
218   {
219     Slot slot;
220     slot.offset = list->offset;
221     slot.size = get_total_list_size(list);
222     return slot;
223   }
224
225   void StorageLists::resize_list(const list_id_t list_id,
        const len_t n_entries)
226   {
227     list_id_list_map_t::iterator list_it =
            id_to_list_map.find(list_id);
228     if (list_it == id_to_list_map.end())
229     {
230       throw std::invalid_argument("List not found");
231     }
232     if (n_entries == 0)
233     {
234       throw std::out_of_range("Cannot resize list to 0
            entries");
235     }
236     InvertedList *list = &list_it->second;
237     if (!does_list_need_reallocation(list, n_entries))
238     {
239       list->used_entries = n_entries;
240       return;
241     }
242     Slot slot = list_to_slot(list);
243     free_slot(&slot);
244     InvertedList new_list;
245     new_list = alloc_list(n_entries);
246     if (new_list.offset == list->offset)
247     {
248       memmove(get_ids_by_list(&new_list),
            get_ids_by_list(list),
            get_ids_size(list->used_entries));
```

```
249     }
250     else
251     {
252       copy_shared_data(&new_list, list);
253     }
254     id_to_list_map[list_id] = new_list;
255   }
256
257   StorageLists::InvertedList
        StorageLists::alloc_list(const len_t n_entries)
258   {
259     InvertedList list;
260     list.used_entries = n_entries;
261     const len_t min_list_length =
          (len_t)(MIN_N_ENTRIES_PER_LIST);
262     len_t allocated_entries = std::max(n_entries,
          min_list_length);
263     list.allocated_entries =
          round_up_to_next_power_of_two(allocated_entries);
264     size_t list_size = get_total_list_size(&list);
265     list.offset = alloc_slot(list_size);
266     return list;
267   }
268
269   StorageLists::slot_it_t
        StorageLists::find_large_enough_slot_index(const
        size_t size)
270   {
271     for (auto it = free_slots.begin(); it !=
          free_slots.end(); it++)
272     {
273       if (it->size >= size)
274       {
275         return it;
276       }
277     }
278     return free_slots.end();
279   }
280
281   void
        StorageLists::grow_region_until_enough_space(size_t
        size)
282   {
283     size_t new_size = total_size == 0 ?
          MIN_TOTAL_SIZE_BYTES : total_size;
284     if (has_free_slot_at_end())
285     {
286       size -= free_slots.back().size;
287     }
```

```
288     while (new_size - total_size < size)
289     {
290       new_size *= 2;
291     }
292     resize_region(new_size);
293   }
294
295   size_t StorageLists::alloc_slot(const size_t size)
296   {
297     auto slot_it = find_large_enough_slot_index(size);
298     if (slot_it == free_slots.end())
299     {
300       grow_region_until_enough_space(size);
301       slot_it = find_large_enough_slot_index(size);
302     }
303     Slot *slot = &(*slot_it);
304     size_t offset = slot->offset;
305     if (slot->size == size)
306     {
307       free_slots.erase(slot_it);
308     }
309     else
310     {
311       slot->offset += size;
312       slot->size -= size;
313     }
314     return offset;
315   }
316
317   StorageLists::slot_it_t
        StorageLists::find_next_slot_to_right(const Slot
        *slot)
318   {
319     for (auto it = free_slots.begin(); it !=
          free_slots.end(); it++)
320     {
321       if (it->offset > slot->offset)
322       {
323         return it;
324       }
325     }
326     return free_slots.end();
327   }
328
329   StorageLists::slot_it_t
        StorageLists::find_next_slot_to_left(
330      const slot_it_t next_slot_right)
331   {
332     if (next_slot_right == free_slots.begin())
```

```
333     {
334       return free_slots.end();
335     }
336     return prev(next_slot_right);
337   }
338
339   bool StorageLists::are_slots_adjacent(
340       const Slot *slot_left, const Slot *slot_right) const
341   {
342     if (slot_left == nullptr || slot_right == nullptr)
343     {
344       return false;
345     }
346     return slot_left->offset + slot_left->size ==
          slot_right->offset;
347   }
348
349   StorageLists::Slot *StorageLists::to_slot(const
       slot_it_t it) const
350   {
351     if (it == free_slots.end())
352     {
353       return nullptr;
354     }
355     return &(*it);
356   }
357
358   void StorageLists::free_slot(const Slot *slot)
359   {
360     slot_it_t slot_right_it =
          find_next_slot_to_right(slot);
361     slot_it_t slot_left_it =
          find_next_slot_to_left(slot_right_it);
362     Slot *slot_right = to_slot(slot_right_it);
363     Slot *slot_left = to_slot(slot_left_it);
364     bool has_adjacent_slot_left =
          are_slots_adjacent(slot_left, slot);
365     bool has_adjacent_slot_right =
          are_slots_adjacent(slot, slot_right);
366     if (has_adjacent_slot_left && has_adjacent_slot_right)
367     {
368       slot_left->size += slot->size + slot_right->size;
369       free_slots.erase(slot_right_it);
370     }
371     else if (has_adjacent_slot_left &&
          !has_adjacent_slot_right)
372     {
373       slot_left->size += slot->size;
374     }
```

```
375     else if (!has_adjacent_slot_left &&
            has_adjacent_slot_right)
376     {
377       slot_right->offset -= slot->size;
378       slot_right->size += slot->size;
379     }
380     else
381     {
382       free_slots.insert(slot_right_it, *slot);
383     }
384   }
385
386   const vector_el_t *StorageLists::get_vectors(const
        list_id_t list_id) const
387   {
388     list_id_list_map_t::const_iterator list_it =
            id_to_list_map.find(list_id);
389     if (list_it == id_to_list_map.end())
390     {
391       throw std::invalid_argument("List not found");
392     }
393     return get_vectors_by_list(&list_it->second);
394   }
395
396   const vector_id_t *StorageLists::get_ids(const
        list_id_t list_id) const
397   {
398     list_id_list_map_t::const_iterator list_it =
            id_to_list_map.find(list_id);
399     if (list_it == id_to_list_map.end())
400     {
401       throw std::invalid_argument("List not found");
402     }
403     return get_ids_by_list(&list_it->second);
404   }
405
406   len_t StorageLists::get_list_length(const list_id_t
        list_id) const
407   {
408     list_id_list_map_t::const_iterator list_it =
            id_to_list_map.find(list_id);
409     if (list_it == id_to_list_map.end())
410     {
411       throw std::invalid_argument("List not found");
412     }
413     return list_it->second.used_entries;
414   }
415
416   size_t
```

```
    StorageLists::round_up_to_next_power_of_two(const
    size_t n) const
{
  size_t power = n;
  power--;
  power |= power >> 1;
  power |= power >> 2;
  power |= power >> 4;
  power |= power >> 8;
  power |= power >> 16;
  power |= power >> 32;
  power++;
  return power;
}

void StorageLists::create_list(
    const list_id_t list_id,
    const len_t n_entries)
{
  if (id_to_list_map.find(list_id) !=
      id_to_list_map.end())
  {
    throw std::invalid_argument("List already exists");
  }
  if (n_entries == 0)
  {
    throw std::out_of_range("List must have at least
        one entry");
  }
  InvertedList list = alloc_list(n_entries);
  id_to_list_map[list_id] = list;
}

void StorageLists::update_entries(
    const list_id_t list_id,
    const vector_el_t *vectors,
    const vector_id_t *ids,
    const len_t n_entries,
    const size_t offset) const
{
  list_id_list_map_t::const_iterator list_it =
      id_to_list_map.find(list_id);
  if (list_it == id_to_list_map.end())
  {
    throw std::invalid_argument("List not found");
  }
  const InvertedList *list = &list_it->second;
  if (offset + n_entries > list->used_entries)
  {
```

```
461       throw std::out_of_range("updating more entries than
              list has");
462     }
463     vector_el_t *list_vectors = get_vectors_by_list(list);
464     vector_id_t *list_ids = get_ids_by_list(list);
465     memcpy(list_vectors + offset * vector_dim, vectors,
            get_vectors_size(n_entries));
466     memcpy(list_ids + offset, ids,
            get_ids_size(n_entries));
467   }
468
469   void StorageLists::insert_entries(
470       const list_id_t list_id,
471       const vector_el_t *vectors,
472       const vector_id_t *ids,
473       const len_t n_entries)
474   {
475     list_id_list_map_t::iterator list_it =
            id_to_list_map.find(list_id);
476     InvertedList *list;
477     if (list_it == id_to_list_map.end())
478     {
479       create_list(list_id, n_entries);
480       list = &id_to_list_map[list_id];
481     }
482     else
483     {
484       list = &list_it->second;
485     }
486     resize_list(list_id, list->used_entries + n_entries);
487     update_entries(list_id, vectors, ids, n_entries,
            list->used_entries - n_entries);
488   }
489
490   void StorageLists::reserve_space(const len_t n_entries)
491   {
492     if (n_entries == 0)
493     {
494       throw std::runtime_error("Cannot reserve 0
              entries");
495     }
496     size_t size_to_reserve = get_vectors_size(n_entries)
            + get_ids_size(n_entries);
497     grow_region_until_enough_space(size_to_reserve);
498   }
499
500   std::ifstream StorageLists::open_filestream(const
        std::string &filename) const
501   {
```

```
502     std::ifstream filestream(filename, std::ios::in |
            std::ios::binary);
503     if (!filestream.is_open())
504     {
505       throw std::runtime_error("Could not open file " +
              filename);
506     }
507     return filestream;
508   }
509
510   void StorageLists::bulk_create_lists(
511       list_id_counts_map_t &lists_counts,
512       const std::string &list_ids_filename,
513       const len_t n_entries)
514   {
515     std::ifstream list_ids_file =
            open_filestream(list_ids_filename);
516     list_id_t list_id;
517     len_t n_entries_read = 0;
518     while (list_ids_file.read((char *)&list_id,
            sizeof(list_id_t)))
519     {
520       lists_counts[list_id]++;
521       n_entries_read++;
522     }
523     list_ids_file.close();
524     if (list_ids_file.bad())
525     {
526       throw std::runtime_error("Error reading list ids
              file");
527     }
528     if (n_entries_read != n_entries)
529     {
530       throw std::runtime_error("Number of entries in list
              ids file does not match n_entries");
531     }
532
533     for (auto list_it : lists_counts)
534     {
535       create_list(list_it.first, list_it.second);
536     }
537   }
538
539   void StorageLists::bulk_insert_entries(
540       const std::string &vectors_filename,
541       const std::string &ids_filename,
542       const std::string &list_ids_filename,
543       const len_t n_entries)
544   {
```

```
545     if (total_size != 0)
546     {
547       throw std::runtime_error("bulk_insert_entries() can
            only be called on an empty inverted lists
            object");
548     }
549
550 #if DYNAMIC_INSERTION == 0
551     reserve_space(n_entries);
552     list_id_counts_map_t entries_left;
553     bulk_create_lists(entries_left, list_ids_filename,
          n_entries);
554 #endif
555
556     std::ifstream vectors_file =
          open_filestream(vectors_filename);
557     std::ifstream ids_file =
          open_filestream(ids_filename);
558     std::ifstream list_ids_file =
          open_filestream(list_ids_filename);
559
560     const len_t max_buffer_size =
          (len_t)(MAX_BUFFER_SIZE);
561     const len_t buffer_size = std::min(n_entries,
          max_buffer_size);
562
563     vector_el_t *vectors = (vector_el_t
          *)malloc(get_vectors_size(buffer_size));
564     vector_id_t *vector_ids = (vector_id_t
          *)malloc(get_ids_size(buffer_size));
565     list_id_t *list_ids = (list_id_t
          *)malloc(get_list_ids_size(buffer_size));
566
567     len_t n_entries_read = 0;
568
569     while (n_entries_read < n_entries)
570     {
571       len_t n_entries_to_read = std::min(buffer_size,
            n_entries - n_entries_read);
572
573       if (!vectors_file.read((char *)vectors,
            get_vectors_size(n_entries_to_read)))
574       {
575         throw std::runtime_error("Error reading vectors
              file");
576       }
577       if (!ids_file.read((char *)vector_ids,
            get_ids_size(n_entries_to_read)))
578       {
```

```
579           throw std::runtime_error("Error reading ids
                 file");
580         }
581         if (!list_ids_file.read((char *)list_ids,
                get_list_ids_size(n_entries_to_read)))
582         {
583           throw std::runtime_error("Error reading list ids
                 file");
584         }
585
586         for (len_t i = 0; i < n_entries_to_read; i++)
587         {
588           vector_el_t *vector = &vectors[i * vector_dim];
589           vector_id_t *vector_id = &vector_ids[i];
590           list_id_t list_id = list_ids[i];
591
592 #if DYNAMIC_INSERTION == 1
593           insert_entries(list_id, vector, vector_id, 1);
594 #else
595           len_t list_length = get_list_length(list_id);
596           len_t cur_list_offset = list_length -
                 entries_left[list_id];
597           update_entries(list_id, vector, vector_id, 1,
                 cur_list_offset);
598           entries_left[list_id]--;
599 #endif
600         }
601
602         n_entries_read += n_entries_to_read;
603       }
604
605       free(vectors);
606       free(vector_ids);
607       free(list_ids);
608
609       if (vectors_file.fail() || ids_file.fail() ||
             list_ids_file.fail())
610       {
611         throw std::runtime_error("Error reading files");
612       }
613
614       vectors_file.close();
615       ids_file.close();
616       list_ids_file.close();
617     }
618 }
```

Listing 9.2: src/storage-node/StorageLists.cpp

```
1 #pragma once
```

```
2
3  #include <string>
4  #include <queue>
5
6  #include "StorageLists.hpp"
7  #include "L2Space.hpp"
8  #include "Query.hpp"
9
10 namespace ann_dkvs
11 {
12   /**
13    * @brief A class that implements a comparison function
          used for a max heap.
14    */
15   class VectorDistanceIdMaxHeapCompare
16   {
17   public:
18     bool operator()(const QueryResult &parent, const
          QueryResult &child) const
19     {
20       bool swapParentWithChild = parent < child;
21       return swapParentWithChild;
22     }
23   };
24   /**
25    * Data structure used to store the results of a query.
26    */
27   typedef std::priority_queue<QueryResult,
        std::vector<QueryResult>,
        VectorDistanceIdMaxHeapCompare> heap_t;
28
29   /**
30    * Internal data structure representing a work iterm
          for a thread.
31    */
32   typedef std::pair<len_t, list_id_t> QueryListPair;
33
34   /**
35    * Internal data structure representing a list of work
          items
36    * which are distributed among the available threads.
37    */
38   typedef std::vector<QueryListPair> QueryListPairs;
39
40   class StorageIndex
41   {
42
43   private:
44     /**
```

```
45          * Pointer to a storage lists object
46          * used to access the data.
47          */
48         const StorageLists *lists;
49
50         /**
51          * Distance function used to compute the distance
52          * between query vector and the vectors within a list.
53          */
54         const distance_func_t distance_func;
55
56         /**
57          * Converts a heap of results into a QueryResults
                object,
58          * i.e. a vector of QueryResult objects.
59          *
60          * @param candidates A heap of query results.
61          * @return A vector of query results.
62          */
63         QueryResults extract_results(heap_t &candidates)
               const;
64
65         /**
66          * Searches a single list for the nearest neighbors
                of a query.
67          *
68          * @param query A pointer to a query object.
69          * @param list_id The id of the list to search.
70          * @param candidates A reference to a heap of query
                results used to store the query results.
71          */
72         void search_preassigned_list(
73             const Query *query,
74             const list_id_t list_id,
75             heap_t &candidates) const;
76
77         /**
78          * Creates a list of work items for a batch of
                queries.
79          * Each work item is a pair of a query id and a list
                id.
80          * The work items are distributed among the available
                threads.
81          *
82          * @param queries A batch of queries.
83          * @return A vector of work items.
84          */
85         QueryListPairs get_work_items(const QueryBatch
               &queries) const;
```

```
86
87      /**
88       * Adds a given query result to the heap of candidate
             results.
89       *
90       * Only updates the heap if the candidate result is
             closer
91       * than the furthest result in the heap or if the
             heap is not full.
92       *
93       * @param query A pointer to a query object.
94       * @param candidate A query result,
95       *                  i.e. a pair of a distance and a
             vector id.
96       * @param candidates A reference to a heap of query
             results used to store the query results.
97       */
98      void add_candidate(const Query *query, const
           QueryResult &candidate, heap_t &candidates) const;
99
100   public:
101     /**
102      * Creates a new storage index object.
103      *
104      * @param lists A pointer to a storage lists object
105      */
106     StorageIndex(const StorageLists *lists);
107
108     /**
109      * Searches all lists of a query selected for probing
110      * to find the query's nearest neighbors.
111      *
112      * @param query A pointer to a query object.
113      * @return A vector of query results.
114      */
115     QueryResults search_preassigned(const Query *query)
           const;
116
117     /**
118      * Searches all lists of a batch of queries selected
             for probing
119      * to find the queries' nearest neighbors.
120      *
121      * @param queries A batch of queries, i.e. a vector
             of query objects.
122      * @return A batch of query results,
123      *           i.e. a vector of vectors of query results.
124      */
125     QueryResultsBatch batch_search_preassigned(const
```

```
        QueryBatch &queries) const;
126   };
127 }
```

Listing 9.3: include/storage-node/StorageIndex.hpp

```
1  #include "StorageIndex.hpp"
2  #include "L2Space.hpp"
3
4  namespace ann_dkvs
5  {
6
7    QueryResults StorageIndex::extract_results(heap_t
       &candidates) const
8    {
9      len_t n_results = candidates.size();
10     QueryResults results(n_results);
11     for (size_t i = 0; i < n_results; i++)
12     {
13       results[n_results - i - 1] = candidates.top();
14       candidates.pop();
15     }
16     return results;
17   }
18
19   void StorageIndex::add_candidate(const Query *query,
       const QueryResult &result, heap_t &candidates) const
20   {
21     if (candidates.size() < query->get_n_results())
22     {
23       candidates.push(result);
24     }
25     else if (result < candidates.top())
26     {
27       candidates.pop();
28       candidates.push(result);
29     }
30   }
31
32   void StorageIndex::search_preassigned_list(
33       const Query *query,
34       const list_id_t list_id,
35       heap_t &candidates) const
36   {
37     const vector_el_t *vectors =
         lists->get_vectors(list_id);
38     const vector_id_t *ids = lists->get_ids(list_id);
39     size_t list_size = lists->get_list_length(list_id);
40     size_t vector_dim = lists->get_vector_dim();
41     for (size_t j = 0; j < list_size; j++)
```

```
42      {
43        const vector_el_t *vector = &vectors[j *
              vector_dim];
44        float distance = distance_func(vector,
              query->get_query_vector(), &vector_dim);
45        const vector_id_t vector_id = ids[j];
46        QueryResult result = {distance, vector_id};
47        add_candidate(query, result, candidates);
48      }
49    }
50
51    StorageIndex::StorageIndex(const StorageLists *lists)
52        : lists(lists),
              distance_func(L2Space(lists->get_vector_dim()).get_distance_func())
53    {
54    }
55
56    QueryResults StorageIndex::search_preassigned(const
          Query *query) const
57    {
58      heap_t candidates;
59      for (len_t i = 0; i < query->get_n_probe(); i++)
60      {
61        list_id_t list_id = query->get_list_to_probe(i);
62        search_preassigned_list(query, list_id, candidates);
63      }
64      return extract_results(candidates);
65    }
66
67    QueryListPairs StorageIndex::get_work_items(const
          QueryBatch &queries) const
68    {
69      QueryListPairs work_items;
70      for (len_t i = 0; i < queries.size(); i++)
71      {
72        for (len_t j = 0; j < queries[i]->get_n_probe();
              j++)
73        {
74          list_id_t list_id =
                queries[i]->get_list_to_probe(j);
75          work_items.push_back({i, list_id});
76        }
77      }
78      return work_items;
79    }
80
81    QueryResultsBatch
          StorageIndex::batch_search_preassigned(const
          QueryBatch &queries) const
```

```
82    {
83      QueryResultsBatch results(queries.size());
84
85  #if PMODE == 0 || PMODE == 1
86  #if PMODE != 0
87  #pragma omp parallel for schedule(runtime)
88  #endif
89      for (len_t i = 0; i < queries.size(); i++)
90      {
91        results[i] = search_preassigned(queries[i]);
92      }
93  #elif PMODE == 2
94      std::vector<heap_t> candidate_lists(queries.size());
95
96      QueryListPairs work_items = get_work_items(queries);
97
98  #pragma omp parallel for schedule(runtime)
99      for (len_t i = 0; i < work_items.size(); i++)
100     {
101       len_t query_index = work_items[i].first;
102       const Query *query = queries[query_index];
103       list_id_t list_id = work_items[i].second;
104       heap_t local_candidates;
105       search_preassigned_list(query, list_id,
            local_candidates);
106 #pragma omp critical
107       {
108         while (local_candidates.size() > 0)
109         {
110           QueryResult result = local_candidates.top();
111           local_candidates.pop();
112           add_candidate(query, result,
                candidate_lists[query_index]);
113         }
114       }
115     }
116 #pragma omp single
117     for (len_t j = 0; j < queries.size(); j++)
118     {
119       results[j] = extract_results(candidate_lists[j]);
120     }
121 #endif
122     return results;
123   }
124 }
```

Listing 9.4: src/storage-node/StorageIndex.cpp

```
1   #pragma once
2
```

```cpp
#include <vector>
#include <queue>

#include "types.hpp"
#include "Query.hpp"

namespace ann_dkvs
{
  /**
   * @brief A struct that represents a centroid candidate containing
   * the distance to the query vector and the list id.
   */
  struct CentroidsResult
  {
    distance_t distance;
    list_id_t list_id;
    friend bool operator<(const CentroidsResult &a, const CentroidsResult &b)
    {
      bool closer = a.distance < b.distance ||
                    (a.distance == b.distance &&
                       a.list_id < b.list_id);
      return closer;
    }
  };

  /**
   * @brief A class that implements a comparison function used for a max heap.
   */
  class CentroidsDistanceIdMaxHeapCompare
  {
  public:
    bool operator()(const CentroidsResult &parent, const CentroidsResult &child)
    {
      bool swapParentWithChild = parent < child;
      return swapParentWithChild;
    }
  };

  /**
   * Data structure used to store the centroid candidates.
   */
  typedef std::priority_queue<CentroidsResult,
     std::vector<CentroidsResult>,
     CentroidsDistanceIdMaxHeapCompare> centroids_heap_t;

```

```
45  class RootIndex
46  {
47  private:
48    /**
49     * Dimension of the centroid vectors.
50     */
51    len_t vector_dim;
52
53    /**
54     * Pointer to the centroid vectors.
55     */
56    vector_el_t *centroids;
57
58    /**
59     * Number of centroid vectors.
60     */
61    len_t n_centroids;
62
63    /**
64     * Given a query and the results of the nearest
          centroid search,
65     * this function sets the lists to be searched for
          the query.
66     *
67     * @param query A pointer to the query object.
68     * @param nearest_centroids A heap of the nearest
          centroid candidates.
69     */
70    void allocate_list_ids(Query *query, centroids_heap_t
        *nearest_centroids);
71
72    /**
73     * Given a query and a centroid candidate, this
          function adds the candidate
74     * to the heap of candidates.
75     *
76     * The heap is only updated if the candidate is
          closer than the furthest
77     * candidate in the heap or if the heap is not full.
78     *
79     * @param query A pointer to the query object.
80     * @param candidate A centroid candidate.
81     * @param candidates A heap of the nearest centroid
          candidates.
82     */
83    void add_candidate(const Query *query, const
        CentroidsResult &candidate, centroids_heap_t
        &candidates);
84
```

```
85    public:
86      /**
87       * Creates a new root index object.
88       *
89       * Allocates memory on the heap for the centroid
             vectors.
90       *
91       * @param vector_dim Dimension of the centroid
             vectors.
92       * @param centroids Pointer to the centroid vectors.
93       * @param n_centroids Number of centroid vectors.
94       */
95      RootIndex(len_t vector_dim, vector_el_t *centroids,
           len_t n_centroids);
96
97      /**
98       * Destroys the root index object.
99       *
100      * Frees the memory allocated for the centroid
           vectors.
101      */
102     ~RootIndex();
103
104     /**
105      * Finds the nearest centroids of the query
106      * and sets the list ids to be searched for the query.
107      *
108      * @param query A pointer to the query object.
109      */
110     void preassign_query(Query *query);
111
112     /**
113      * Finds the nearest centroids of a list of queries
114      * and sets the list ids to be searched.
115      *
116      * @param queries A query batch object.
117      */
118     void batch_preassign_queries(QueryBatch queries);
119   };
120 } // namespace ann_dkvs
```

Listing 9.5: include/root-node/RootIndex.hpp

```
1  #include <vector>
2  #include <cassert>
3  #include <iostream>
4  #include <cstring>
5
6  #include "../include/L2Space.hpp"
7  #include "../include/root-node/RootIndex.hpp"
```

```
8
9   namespace ann_dkvs
10  {
11    RootIndex::RootIndex(len_t vector_dim, vector_el_t
          *centroids, len_t n_centroids)
12        : vector_dim(vector_dim), centroids(centroids),
              n_centroids(n_centroids)
13    {
14      this->centroids = (vector_el_t *)malloc(n_centroids *
            vector_dim * sizeof(vector_el_t));
15      memcpy(this->centroids, centroids, n_centroids *
            vector_dim * sizeof(vector_el_t));
16    }
17
18    RootIndex::~RootIndex()
19    {
20      free(centroids);
21    }
22
23    void RootIndex::add_candidate(const Query *query, const
          CentroidsResult &result, centroids_heap_t
          &candidates)
24    {
25      if (candidates.size() < query->get_n_probe())
26      {
27        candidates.push(result);
28      }
29      else if (result < candidates.top())
30      {
31        candidates.pop();
32        candidates.push(result);
33      }
34    }
35
36    void RootIndex::allocate_list_ids(Query *query,
          centroids_heap_t *nearest_centroids)
37    {
38      for (size_t query_id = 0; query_id <
            query->get_n_probe(); query_id++)
39      {
40        size_t insertion_index = query->get_n_probe() -
              query_id - 1;
41        list_id_t list_id =
              nearest_centroids->top().list_id;
42        query->set_list_to_probe(insertion_index, list_id);
43        nearest_centroids->pop();
44      }
45    }
46
```

```
47   void RootIndex::preassign_query(Query *query)
48   {
49     centroids_heap_t candidates;
50     distance_func_t distance_func =
         L2Space(vector_dim).get_distance_func();
51
52     for (list_id_t list_id = 0; list_id <
         (list_id_t)n_centroids; list_id++)
53     {
54       vector_el_t *centroid = &centroids[list_id *
           vector_dim];
55       float distance = distance_func(centroid,
           query->get_query_vector(), &vector_dim);
56       const CentroidsResult result = {.distance =
           distance, .list_id = list_id};
57       add_candidate(query, result, candidates);
58     }
59     allocate_list_ids(query, &candidates);
60   }
61
62   void RootIndex::batch_preassign_queries(QueryBatch
       queries)
63   {
64 #if PMODE != 0
65 #pragma omp parallel for schedule(runtime)
66 #endif
67     for (len_t i = 0; i < queries.size(); i++)
68     {
69       preassign_query(queries[i]);
70     }
71   }
72 }
```

Listing 9.6: src/root-node/RootIndex.cpp

```
1  #pragma once
2
3  #include <vector>
4
5  #include "types.hpp"
6
7  namespace ann_dkvs
8  {
9    class Query
10   {
11   private:
12     vector_el_t *query_vector;
13     list_id_t *list_to_probe;
14     const len_t n_results;
15     const len_t n_probes;
```

```
16      bool free_list_to_probe = false;
17
18    public:
19      Query(vector_el_t *query_vector, const len_t
            n_results, const len_t n_probes);
20      Query(vector_el_t *query_vector, list_id_t
            *list_to_probe, const len_t n_results, const len_t
            n_probes);
21      vector_el_t *get_query_vector() const;
22      list_id_t get_list_to_probe(const len_t i) const;
23      len_t get_n_results() const;
24      len_t get_n_probe() const;
25      void set_list_to_probe(const len_t offset, const
            list_id_t list_id) const;
26      ~Query();
27    };
28
29    typedef std::vector<Query *> QueryBatch;
30
31    struct QueryResult
32    {
33      distance_t distance;
34      vector_id_t vector_id;
35      friend bool operator<(const QueryResult &a, const
            QueryResult &b)
36      {
37        bool closer = a.distance < b.distance ||
38                      (a.distance == b.distance &&
                           a.vector_id < b.vector_id);
39        return closer;
40      }
41    };
42
43    typedef std::vector<QueryResult> QueryResults;
44    typedef std::vector<QueryResults> QueryResultsBatch;
45  } // namespace ann_dkvs
```

Listing 9.7: include/Query.hpp

```
1   #include "Query.hpp"
2
3   namespace ann_dkvs
4   {
5     Query::Query(vector_el_t *query_vector, const len_t
          n_results, const len_t n_probes) :
          query_vector(query_vector), list_to_probe(new
          list_id_t[n_probes]), n_results(n_results),
          n_probes(n_probes), free_list_to_probe(true) {}
6     Query::Query(vector_el_t *query_vector, list_id_t
          *list_to_probe, const len_t n_results, const len_t
```

```
      n_probes)
7       : query_vector(query_vector),
            list_to_probe(list_to_probe),
            n_results(n_results), n_probes(n_probes) {}
8   vector_el_t *Query::get_query_vector() const { return
      query_vector; }
9   list_id_t Query::get_list_to_probe(const len_t i) const
10  {
11    return list_to_probe[i];
12  }
13  len_t Query::get_n_results() const
14  {
15    return n_results;
16  }
17  len_t Query::get_n_probe() const
18  {
19    return n_probes;
20  }
21  void Query::set_list_to_probe(const len_t offset, const
      list_id_t list_id) const
22  {
23    assert(offset < n_probes);
24    list_to_probe[offset] = list_id;
25  }
26  Query::~Query()
27  {
28    if (free_list_to_probe)
29    {
30      delete[] list_to_probe;
31    }
32  }
33 }
```

Listing 9.8: src/Query.cpp

```
1  #pragma once
2
3  #ifdef __AVX__
4  #include <immintrin.h>
5  #endif
6
7  #include "types.hpp"
8
9  #define PORTABLE_ALIGN32 __attribute__((aligned(32)))
10
11 namespace ann_dkvs
12 {
13   static inline float L2Sqr(
14       const void *pVect1v,
15       const void *pVect2v,
```

```
16        const void *qty_ptr)
17    {
18      float *pVect1 = (float *)pVect1v;
19      float *pVect2 = (float *)pVect2v;
20      size_t qty = *((size_t *)qty_ptr);
21
22      float res = 0;
23      for (size_t i = 0; i < qty; i++)
24      {
25        float t = *pVect1 - *pVect2;
26        pVect1++;
27        pVect2++;
28        res += t * t;
29      }
30      return (res);
31    }
32
33  #ifdef __AVX__
34    static inline float L2SqrSIMD16ExtAVX(const void
          *pVect1v, const void *pVect2v, const void *qty_ptr)
35    {
36      float *pVect1 = (float *)pVect1v;
37      float *pVect2 = (float *)pVect2v;
38      size_t qty = *((size_t *)qty_ptr);
39      float PORTABLE_ALIGN32 TmpRes[8];
40      size_t qty16 = qty >> 4;
41
42      const float *pEnd1 = pVect1 + (qty16 << 4);
43
44      __m256 diff, v1, v2;
45      __m256 sum = _mm256_set1_ps(0);
46
47      while (pVect1 < pEnd1)
48      {
49        v1 = _mm256_loadu_ps(pVect1);
50        pVect1 += 8;
51        v2 = _mm256_loadu_ps(pVect2);
52        pVect2 += 8;
53        diff = _mm256_sub_ps(v1, v2);
54        sum = _mm256_add_ps(sum, _mm256_mul_ps(diff, diff));
55
56        v1 = _mm256_loadu_ps(pVect1);
57        pVect1 += 8;
58        v2 = _mm256_loadu_ps(pVect2);
59        pVect2 += 8;
60        diff = _mm256_sub_ps(v1, v2);
61        sum = _mm256_add_ps(sum, _mm256_mul_ps(diff, diff));
62      }
63
```

```
64      _mm256_store_ps(TmpRes, sum);
65      return TmpRes[0] + TmpRes[1] + TmpRes[2] + TmpRes[3]
            + TmpRes[4] + TmpRes[5] + TmpRes[6] + TmpRes[7];
66    }
67
68    static inline float L2SqrSIMD16ExtResiduals(const void
          *pVect1v, const void *pVect2v, const void *qty_ptr)
69    {
70      size_t qty = *((size_t *)qty_ptr);
71      size_t qty16 = qty >> 4 << 4;
72      float res = L2SqrSIMD16ExtAVX(pVect1v, pVect2v,
            &qty16);
73      float *pVect1 = (float *)pVect1v + qty16;
74      float *pVect2 = (float *)pVect2v + qty16;
75
76      size_t qty_left = qty - qty16;
77      float res_tail = L2Sqr(pVect1, pVect2, &qty_left);
78      return (res + res_tail);
79    }
80
81  #endif
82
83    using distance_func_t = distance_t (*)(const void *,
          const void *, const void *);
84    class L2Space
85    {
86    private:
87      size_t vector_dim;
88      distance_func_t distance_func;
89
90    public:
91      L2Space(size_t vector_dim);
92      distance_func_t get_distance_func() const;
93      size_t get_vector_dim() const;
94    };
95
96  } // namespace ann_dkvs
```

Listing 9.9: include/L2Space.hpp

```
1   #include "L2Space.hpp"
2
3   namespace ann_dkvs
4   {
5
6     L2Space::L2Space(size_t vector_dim) :
          vector_dim(vector_dim)
7     {
8       distance_func = L2Sqr;
9
```

```
#ifdef __AVX__
    if (vector_dim % 16 == 0)
    {
      distance_func = L2SqrSIMD16ExtAVX;
    }
    else if (vector_dim > 16)
    {
      distance_func = L2SqrSIMD16ExtResiduals;
    }
#endif
  }

  distance_func_t L2Space::get_distance_func() const
  {
    return distance_func;
  }

  len_t L2Space::get_vector_dim() const
  {
    return vector_dim;
  }
}
```

Listing 9.10: src/L2Space.cpp

```
#pragma once

#include <vector>
#include <cassert>

namespace ann_dkvs
{
  // 1 byte
  typedef unsigned char uint8_t;
  // 4 bytes
  typedef float vector_el_t;
  typedef float distance_t;
  // 8 bytes
  typedef unsigned long len_t;
  typedef unsigned long size_t;
  typedef signed long int64_t;
  typedef int64_t list_id_t;
  typedef int64_t vector_id_t;
}
```

Listing 9.11: include/types.hpp

9.3 Testing and benchmarking

- tests/StorageListsTest.cpp
- tests/IndexTest.cpp
- include/tests/StorageListsTestUtils.hpp
- include/tests/TestConfig.hpp
- tests/StorageListsTestUtils.cpp

```
#include <cmath>
#include <iostream>
#include <limits>
#include <stdlib.h>
#include <fstream>
#include <sys/mman.h>

#include "../lib/catch.hpp"

#include "../include/tests/StorageListsTestUtils.hpp"

#ifndef TEST_VECTOR_DIM
#define TEST_VECTOR_DIM 128
#endif
#ifndef TEST_N_LISTS
#define TEST_N_LISTS 1024
#endif

SCENARIO("StorageLists(): an StorageLists object can be
    constructed", "[.StorageLists]")
{
  GIVEN("a nonzero vector dimension")
  {
    len_t vector_dim = gen_vector_dim({0});

    CHECK(vector_dim > 0);

    WHEN("an StorageLists object is created")
    {
      StorageLists lists =
          get_inverted_lists_object(vector_dim);

      THEN("the number of lists is 0")
      {
        REQUIRE(lists.get_length() == 0);
      }

      THEN("the vector dimension is correct")
      {
```

```
38          REQUIRE(lists.get_vector_dim() == vector_dim);
39        }
40
41        THEN("the filename is correct")
42        {
43          REQUIRE(lists.get_filename() == join(TMP_DIR,
                get_lists_filename()));
44        }
45
46        THEN("the vector size is correct")
47        {
48          REQUIRE(lists.get_vector_size() == vector_dim *
                sizeof(vector_el_t));
49        }
50
51        THEN("the total size is correct")
52        {
53          REQUIRE(lists.get_total_size() == 0);
54        }
55      }
56    }
57    GIVEN("the vector dimension 0")
58    {
59      len_t vector_dim = 0;
60
61      WHEN("an StorageLists object is created")
62      {
63        THEN("an exception is thrown")
64        {
65          REQUIRE_THROWS_AS(get_inverted_lists_object(vector_dim),
                std::out_of_range);
66        }
67      }
68    }
69  }
70
71  SCENARIO("create_list(): inverted lists can be created",
        "[.StorageLists]")
72  {
73
74    GIVEN("an StorageLists object storing vectors of some
          dimension")
75    {
76      len_t vector_dim = gen_vector_dim({0});
77      StorageLists lists =
            get_inverted_lists_object(vector_dim);
78
79      WHEN("a list of length 0 is created")
80      {
```

```
81        len_t list_id = gen_list_id();
82        len_t list_length = 0;
83        THEN("an exception is thrown")
84        {
85          REQUIRE_THROWS_AS(lists.create_list(list_id,
              list_length), std::out_of_range);
86        }
87      }
88
89      WHEN("a list of nonzero length is created")
90      {
91        auto list_ids = gen_list_ids(2);
92        auto list_lengths = gen_list_lengths(2, {0});
93        CHECK(list_ids[0] != list_ids[1]);
94        CHECK(list_lengths[0] > 0);
95        CHECK(list_lengths[1] > 0);
96
97        lists.create_list(list_ids[0], list_lengths[0]);
98
99        THEN("the number of lists is 1")
100       {
101         REQUIRE(lists.get_length() == 1);
102       }
103
104       THEN("the list length is correct")
105       {
106         REQUIRE(lists.get_list_length(list_ids[0]) ==
              list_lengths[0]);
107       }
108
109       THEN("the total size is correct")
110       {
111         size_t used_space = get_list_size(vector_dim,
              list_lengths[0]);
112         REQUIRE(lists.get_total_size() ==
              get_total_size(used_space));
113       }
114
115       AND_WHEN("another list of nonzero length with the
            same index is created")
116       {
117
118         THEN("an exception is thrown")
119         {
120           REQUIRE_THROWS_AS(lists.create_list(list_ids[0],
                list_lengths[1]), std::invalid_argument);
121
122           THEN("the number of lists is still 1")
123           {
```

```
124             REQUIRE(lists.get_length() == 1);
125           }
126
127           THEN("the first list has the correct length")
128           {
129             REQUIRE(lists.get_list_length(list_ids[0]) ==
                    list_lengths[0]);
130           }
131
132           THEN("the total size is still correct")
133           {
134             size_t used_space = get_list_size(vector_dim,
                    list_lengths[0]);
135             REQUIRE(lists.get_total_size() ==
                    get_total_size(used_space));
136           }
137         }
138       }
139
140       AND_WHEN("another list of nonzero length with a
             different index is created")
141       {
142         lists.create_list(list_ids[1], list_lengths[1]);
143
144         THEN("the number of lists is 2")
145         {
146           REQUIRE(lists.get_length() == 2);
147         }
148
149         THEN("the first list has the correct length")
150         {
151           REQUIRE(lists.get_list_length(list_ids[0]) ==
                  list_lengths[0]);
152         }
153
154         THEN("the second list has the correct length")
155         {
156           REQUIRE(lists.get_list_length(list_ids[1]) ==
                  list_lengths[1]);
157         }
158
159         THEN("the total size is correct")
160         {
161           size_t used_space = get_list_size(vector_dim,
                  list_lengths[0]) + get_list_size(vector_dim,
                  list_lengths[1]);
162           REQUIRE(lists.get_total_size() ==
                  get_total_size(used_space));
163         }
```

```
164
165         THEN("the free space is correct")
166         {
167           size_t used_space = get_list_size(vector_dim,
                list_lengths[0]) + get_list_size(vector_dim,
                list_lengths[1]);
168           size_t total_size = get_total_size(used_space);
169           REQUIRE(lists.get_free_space() == total_size -
                used_space);
170         }
171       }
172     }
173     WHEN("a list of size 0 is created")
174     {
175       list_id_t list_id = 1;
176       len_t list_length = 0;
177
178       THEN("an exception is thrown")
179       {
180         REQUIRE_THROWS_AS(lists.create_list(list_id,
              list_length), std::out_of_range);
181       }
182     }
183
184     WHEN("a list of length 0 is created")
185     {
186       list_id_t list_id = 1;
187       len_t list_length = 0;
188
189       THEN("an exception is thrown")
190       {
191         REQUIRE_THROWS_AS(lists.create_list(list_id,
              list_length), std::out_of_range);
192       }
193     }
194   }
195   GIVEN("an StorageLists object storing vectors of
        dimension 1")
196   {
197     len_t vector_dim = 1;
198     StorageLists lists =
          get_inverted_lists_object(vector_dim);
199
200     WHEN("a list of size < 32B is created")
201     {
202       list_id_t list_id = 1;
203       len_t list_length = 1;
204       lists.create_list(list_id, list_length);
205
```

```
206         size_t list_size = get_list_size(vector_dim,
                list_length);
207
208         CHECK(list_size < 32);
209
210         THEN("the total size is 32B")
211         {
212           REQUIRE(lists.get_total_size() == 32);
213         }
214
215         THEN("the number of lists is 1")
216         {
217           REQUIRE(lists.get_length() == 1);
218         }
219
220         THEN("the list length is correct")
221         {
222           REQUIRE(lists.get_list_length(list_id) ==
                list_length);
223         }
224       }
225     }
226   }
227
228   SCENARIO("get_free_space(): the free space of an
        StorageLists object is as expected", "[.StorageLists]")
229   {
230
231     GIVEN('an StorageLists object storing vectors of some
          dimension')
232     {
233       len_t vector_dim = gen_vector_dim({0});
234       StorageLists lists =
            get_inverted_lists_object(vector_dim);
235
236       WHEN("no lists have been created")
237       {
238         THEN("the free space is 0")
239         {
240           REQUIRE(lists.get_free_space() == 0);
241         }
242       }
243     }
244
245     GIVEN("an StorageLists object storing vectors of some
          dimension > 2")
246     {
247       len_t vector_dim =
            gen_vector_dim(std::vector<len_t>({0, 1, 2}));
```

```
248     StorageLists lists =
            get_inverted_lists_object(vector_dim);
249
250     WHEN("a list is created which has a size > 32B")
251     {
252       list_id_t list_id = gen_list_id();
253       len_t list_length = gen_list_length({0});
254       lists.create_list(list_id, list_length);
255
256       THEN("the free space is the difference between the
              list size and the next power of two")
257       {
258         len_t list_size = get_list_size(vector_dim,
                list_length);
259         len_t next_power_of_two =
                round_up_to_next_power_of_two(list_size);
260         REQUIRE(lists.get_free_space() ==
                next_power_of_two - list_size);
261       }
262     }
263   }
264
265   GIVEN("an StorageLists object storing vectors of
          dimension 2")
266   {
267     len_t vector_dim = 2;
268     StorageLists lists =
            get_inverted_lists_object(vector_dim);
269
270     WHEN("a list is created with size < 32B")
271     {
272       list_id_t list_id = gen_list_id();
273       len_t list_length = 1;
274       lists.create_list(list_id, list_length);
275
276       len_t list_size = get_list_size(vector_dim,
              list_length);
277
278       CHECK(list_size < 32);
279
280       THEN("the free space is 32 - list size")
281       {
282         REQUIRE(lists.get_free_space() == 32 - list_size);
283       }
284     }
285   }
286 }
287
288 SCENARIO("update_entries(): multiple entries of a list
```

```
    can be updated", "[.StorageLists]")
{
  GIVEN("an StorageLists object and two lists of 128D
      vectors and corresponding ids")
  {
    len_t vector_dim = 128;
    StorageLists lists =
        get_inverted_lists_object(vector_dim);

    auto data = gen_vectors(128);
    len_t list_length = get_vector_length(data,
        vector_dim);
    vector_el_t *vectors = to_ptr(vector_el_t,
        data.first);
    vector_id_t *ids = to_ptr(vector_id_t, data.second);

    REQUIRE(list_length == std::min(data.first.size() /
        vector_dim, data.second.size()));

    auto data2 = gen_vectors(128);
    len_t list_length2 = get_vector_length(data2,
        vector_dim);
    vector_el_t *vectors2 = to_ptr(vector_el_t,
        data2.first);
    vector_id_t *ids2 = to_ptr(vector_id_t, data2.second);

    REQUIRE(list_length2 == std::min(data2.first.size() /
        vector_dim, data2.second.size()));

    WHEN("an inverted list is created")
    {
      list_id_t list_id = gen_list_id();
      lists.create_list(list_id, list_length);
      size_t total_size_before_update =
          lists.get_total_size();

      AND_WHEN("all entries of the list are updated with
          the vectors and ids of the first list")
      {
        lists.update_entries(list_id, vectors, ids, 0,
            list_length);

        THEN("all vectors are updated")
        {
          const vector_el_t *list_vectors =
              lists.get_vectors(list_id);
          for (len_t i = 0; i < list_length * vector_dim;
              i++)
          {
```

```
324             REQUIRE(list_vectors[i] == vectors[i]);
325           }
326         }
327
328         THEN("all ids are updated")
329         {
330           const list_id_t *list_ids =
                lists.get_ids(list_id);
331           for (len_t i = 0; i < list_length; i++)
332           {
333             REQUIRE(list_ids[i] == ids[i]);
334           }
335         }
336
337         THEN("the list length is unaffected")
338         {
339           REQUIRE(lists.get_list_length(list_id) ==
                list_length);
340         }
341
342         THEN("the total size is unaffected")
343         {
344           REQUIRE(lists.get_total_size() ==
                total_size_before_update);
345         }
346
347         AND_WHEN("only the first few entries are updated
              with the second list")
348         {
349           len_t update_length = random_range(1,
                std::min(list_length, list_length2));
350
351           REQUIRE(update_length <= list_length);
352           REQUIRE(update_length <= list_length2);
353
354           lists.update_entries(list_id, vectors2, ids2,
                0, update_length);
355
356           THEN("only the first few entries are updated
                with the vectors from the second list")
357           {
358             const vector_el_t *list_vectors =
                  lists.get_vectors(list_id);
359
360             are_vectors_equal(list_vectors, vectors2,
                  vector_dim, update_length);
361
362             are_vectors_equal(list_vectors +
                  update_length * vector_dim, vectors +
```

```
                update_length * vector_dim, vector_dim,
                list_length - update_length);
363         }
364
365         THEN("only the first few entries are updated
                with the ids from the second list")
366         {
367           const list_id_t *list_ids =
                lists.get_ids(list_id);
368
369           are_ids_equal(list_ids, ids2, update_length);
370
371           are_ids_equal(list_ids + update_length, ids +
                update_length, list_length -
                update_length);
372         }
373
374         THEN("the list length is unaffected")
375         {
376           REQUIRE(lists.get_list_length(list_id) ==
                list_length);
377         }
378
379         THEN("the total size is unaffected")
380         {
381           REQUIRE(lists.get_total_size() ==
                total_size_before_update);
382         }
383       }
384
385       AND_WHEN("only the last few entries are updated
              with the second list")
386       {
387         len_t update_length = random_range(1,
              std::min(list_length, list_length2));
388
389         REQUIRE(update_length <= list_length);
390         REQUIRE(update_length <= list_length2);
391
392         lists.update_entries(list_id, vectors2, ids2,
              list_length - update_length, update_length);
393
394         THEN("only the last few entries are updated
                with the vectors from the second list")
395         {
396           const vector_el_t *list_vectors =
                lists.get_vectors(list_id);
397
398           are_vectors_equal(list_vectors, vectors,
```

```
399                 vector_dim, list_length - update_length);
400
                  are_vectors_equal(list_vectors + (list_length
                     - update_length) * vector_dim, vectors2,
                     vector_dim, update_length);
401             }
402
403             THEN("only the last few entries are updated
                   with the ids from the second list")
404             {
405               const list_id_t *list_ids =
                     lists.get_ids(list_id);
406
407               are_ids_equal(list_ids, ids, list_length -
                     update_length);
408
409               are_ids_equal(list_ids + (list_length -
                     update_length), ids2, update_length);
410             }
411
412             THEN("the list length is unaffected")
413             {
414               REQUIRE(lists.get_list_length(list_id) ==
                     list_length);
415             }
416
417             THEN("the total size is unaffected")
418             {
419               REQUIRE(lists.get_total_size() ==
                     total_size_before_update);
420             }
421           }
422         }
423
424         AND_WHEN("the list is updated with more entries
                than it has")
425         {
426           len_t update_length = list_length + 1;
427
428           THEN("an exception is thrown")
429           {
430             REQUIRE_THROWS_AS(lists.update_entries(list_id,
                   vectors, ids, 0, update_length),
                   std::out_of_range);
431           }
432         }
433
434         AND_WHEN("a list is updated which does not exist")
435         {
```

```
436         list_id_t list_id2 = list_id + 1;
437         len_t update_length = 5;
438         THEN("an exception is thrown")
439         {
440           REQUIRE_THROWS_AS(lists.update_entries(list_id2,
                vectors, ids, 0, update_length),
                std::invalid_argument);
441         }
442       }
443     }
444   }
445 }
446
447 SCENARIO("insert_entries(): entries can be appended to an
      inverted list", "[.StorageLists]")
448 {
449   GIVEN("an StorageLists object and two lists of 1D
        vectors and corresponding ids")
450   {
451     len_t vector_dim = 128;
452     StorageLists lists =
          get_inverted_lists_object(vector_dim);
453
454     auto data = gen_vectors(128);
455     len_t list1_length = get_vector_length(data,
          vector_dim);
456     vector_el_t *vectors = to_ptr(vector_el_t,
          data.first);
457     vector_id_t *ids = to_ptr(vector_id_t, data.second);
458
459     auto data2 = gen_vectors(128);
460     len_t list2_length = get_vector_length(data2,
          vector_dim);
461     vector_el_t *vectors2 = to_ptr(vector_el_t,
          data2.first);
462     vector_id_t *ids2 = to_ptr(vector_id_t, data2.second);
463
464     len_t total_list_length = list1_length + list2_length;
465
466     WHEN("an inverted list is created")
467     {
468       list_id_t list_id = gen_list_id();
469       lists.create_list(list_id, list1_length);
470       size_t total_size_before_update =
            lists.get_total_size();
471
472       AND_WHEN("all entries of the list are updated with
            the vectors and ids of the first list")
473       {
```

```
474         lists.update_entries(list_id, vectors, ids, 0,
                list1_length);
475
476         THEN("all vectors are updated")
477         {
478           const vector_el_t *list_vectors =
                lists.get_vectors(list_id);
479           are_vectors_equal(list_vectors, vectors,
                vector_dim, list1_length);
480         }
481
482         THEN("all ids are updated")
483         {
484           const list_id_t *list_ids =
                lists.get_ids(list_id);
485           are_ids_equal(list_ids, ids, list1_length);
486         }
487
488         THEN("the list length is unaffected")
489         {
490           REQUIRE(lists.get_list_length(list_id) ==
                list1_length);
491         }
492
493         THEN("the total size is unaffected")
494         {
495           REQUIRE(lists.get_total_size() ==
                total_size_before_update);
496         }
497
498         AND_WHEN("the list is appended with the entries
                of the second list")
499         {
500           lists.insert_entries(list_id, vectors2, ids2,
                list2_length);
501
502           const vector_el_t *list_vectors =
                lists.get_vectors(list_id);
503
504           THEN("all vectors of the first list are still
                present")
505           {
506             are_vectors_equal(list_vectors, vectors,
                  vector_dim, list1_length);
507           }
508
509           THEN("all vectors of the second list are
                appended")
510           {
```

```
511             are_vectors_equal(list_vectors + list1_length
                    * vector_dim, vectors2, vector_dim,
                    list2_length);
512           }
513
514           const list_id_t *list_ids =
                  lists.get_ids(list_id);
515
516           THEN("all ids of the first list are still
                  present")
517           {
518             are_ids_equal(list_ids, ids, list1_length);
519           }
520
521           THEN("all ids of the second list are correctly
                  appended")
522           {
523             are_ids_equal(list_ids + list1_length, ids2,
                    list2_length);
524           }
525
526           THEN("the list length is updated")
527           {
528             REQUIRE(lists.get_list_length(list_id) ==
                    total_list_length);
529           }
530
531           THEN("the total size is updated")
532           {
533             size_t list_size_after_append =
                    get_list_size(vector_dim,
                    total_list_length);
534             size_t total_size_after_append =
                    get_total_size(list_size_after_append);
535             REQUIRE(lists.get_total_size() ==
                    total_size_after_append);
536           }
537         }
538       }
539     }
540   }
541 }
542
543 SCENARIO("resize_list(): an inverted list can be
      resized", "[.StorageLists]")
544 {
545   GIVEN("an StorageLists object and a list of 1D vectors
        and corresponding ids")
546   {
```

```
547     len_t vector_dim = 1;
548     StorageLists lists =
            get_inverted_lists_object(vector_dim);
549
550     auto data = gen_vectors(1);
551     len_t list_length = get_vector_length(data,
            vector_dim);
552     vector_el_t *vectors = to_ptr(vector_el_t,
            data.first);
553     vector_id_t *ids = to_ptr(vector_id_t, data.second);
554
555     WHEN("an inverted list is created")
556     {
557       list_id_t list_id = 1;
558       lists.create_list(list_id, list_length);
559
560       AND_WHEN("all entries of the list are updated with
                the vectors and ids of the first list")
561       {
562         lists.update_entries(list_id, vectors, ids, 0,
                list_length);
563         size_t total_size_before_resizing =
                lists.get_total_size();
564
565         if (list_length >= 2)
566         {
567           AND_WHEN("the list is resized to a smaller
                    size")
568           {
569             len_t new_list_length = random_range(1,
                        list_length - 1);
570
571             REQUIRE(new_list_length < list_length);
572             lists.resize_list(list_id, new_list_length);
573
574             const vector_el_t *list_vectors =
                        lists.get_vectors(list_id);
575             THEN("all vectors of the first list are
                        unchanged")
576             {
577               for (len_t i = 0; i < new_list_length; i++)
578               {
579                 REQUIRE(list_vectors[i] == vectors[i]);
580               }
581             }
582             const list_id_t *list_ids =
                        lists.get_ids(list_id);
583             THEN("all ids of the first list are
                        unchanged")
```

```
584             {
585               for (len_t i = 0; i < new_list_length; i++)
586               {
587                 REQUIRE(list_ids[i] == ids[i]);
588               }
589             }
590             THEN("the list length is updated")
591             {
592               REQUIRE(lists.get_list_length(list_id) ==
                      new_list_length);
593             }
594
595             THEN("the total size is unaffected")
596             {
597               REQUIRE(lists.get_total_size() ==
                      total_size_before_resizing);
598             }
599           }
600         }
601         if (list_length < MAX_LIST_LENGTH)
602         {
603           AND_THEN("the list is resized to a larger size")
604           {
605             len_t new_list_length =
                    random_range(list_length + 1,
                    MAX_LIST_LENGTH);
606             REQUIRE(new_list_length > list_length);
607             lists.resize_list(list_id, new_list_length);
608
609             const vector_el_t *list_vectors =
                    lists.get_vectors(list_id);
610             THEN("all vectors of the first list are
                    unchanged")
611             {
612               for (len_t i = 0; i < list_length; i++)
613               {
614                 REQUIRE(list_vectors[i] == vectors[i]);
615               }
616             }
617             const list_id_t *list_ids =
                    lists.get_ids(list_id);
618             THEN("all ids of the first list are
                    unchanged")
619             {
620               for (len_t i = 0; i < list_length; i++)
621               {
622                 REQUIRE(list_ids[i] == ids[i]);
623               }
624             }
```

```
625             THEN("the list length is updated")
626             {
627               REQUIRE(lists.get_list_length(list_id) ==
                      new_list_length);
628             }
629
630             THEN("the total size is updated")
631             {
632               size_t list_size_after_resize =
                      get_list_size(vector_dim,
                      new_list_length);
633               size_t total_size_after_resize =
                      get_total_size(list_size_after_resize);
634               REQUIRE(lists.get_total_size() ==
                      total_size_after_resize);
635             }
636           }
637         }
638         AND_WHEN("the list is resized to the same size")
639         {
640           len_t new_list_length = list_length;
641           lists.resize_list(list_id, new_list_length);
642
643           const vector_el_t *list_vectors =
                  lists.get_vectors(list_id);
644           THEN("all vectors of the first list are
                  unchanged")
645           {
646             for (len_t i = 0; i < list_length; i++)
647             {
648               REQUIRE(list_vectors[i] == vectors[i]);
649             }
650           }
651           const list_id_t *list_ids =
                  lists.get_ids(list_id);
652           THEN("all ids of the first list are unchanged")
653           {
654             for (len_t i = 0; i < list_length; i++)
655             {
656               REQUIRE(list_ids[i] == ids[i]);
657             }
658           }
659           THEN("the list length is updated")
660           {
661             REQUIRE(lists.get_list_length(list_id) ==
                      new_list_length);
662           }
663
664           THEN("the total size is unaffected")
```

```
665             {
666               REQUIRE(lists.get_total_size() ==
                      total_size_before_resizing);
667             }
668           }
669           AND_WHEN("the list is resized to a size of 0")
670           {
671             len_t new_list_length = 0;
672
673             THEN("an expected exception is thrown")
674             {
675               REQUIRE_THROWS_AS(lists.resize_list(list_id,
                      new_list_length), std::out_of_range);
676             }
677           }
678         }
679         AND_WHEN("a list is resized which does not exist")
680         {
681           len_t new_list_length = 10;
682           list_id_t non_existing_list_id = 2;
683           THEN("an exception is thrown")
684           {
685             REQUIRE_THROWS_AS(lists.resize_list(non_existing_list_id,
                    new_list_length), std::invalid_argument);
686           }
687         }
688       }
689     }
690   }
691
692   SCENARIO("get_list_length(): the length of an inverted
         list can be retrieved", "[.StorageLists]")
693   {
694     GIVEN("an StorageLists object of 128D vectors")
695     {
696       len_t vector_dim = 128;
697       StorageLists lists =
             get_inverted_lists_object(vector_dim);
698
699       WHEN("an inverted list is created")
700       {
701         len_t list_length = gen_list_length();
702         auto list_ids = gen_list_ids(2);
703         lists.create_list(list_ids[0], list_length);
704
705         THEN("the list length is correct")
706         {
707           REQUIRE(lists.get_list_length(list_ids[0]) ==
                  list_length);
```

```
708       }
709
710       AND_WHEN("the length of a list is retrieved which
              does not exist")
711       {
712         THEN("an exception is thrown")
713         {
714           REQUIRE_THROWS_AS(lists.get_list_length(list_ids[1]),
                  std::invalid_argument);
715         }
716       }
717     }
718   }
719 }
720
721 SCENARIO("get_vectors(): the vectors of an inverted list
        can be retrieved", "[.StorageLists]")
722 {
723   GIVEN("an StorageLists object and a list of 128D
          vectors and corresponding ids")
724   {
725     len_t vector_dim = 128;
726     StorageLists lists =
            get_inverted_lists_object(vector_dim);
727
728     auto data = gen_vectors(128);
729     len_t list_length = get_vector_length(data,
            vector_dim);
730     vector_el_t *vectors = to_ptr(vector_el_t,
            data.first);
731     vector_id_t *ids = to_ptr(vector_id_t, data.second);
732
733     WHEN("an inverted list is created")
734     {
735       auto list_ids = gen_list_ids(2);
736       lists.create_list(list_ids[0], list_length);
737
738       AND_WHEN("all entries of the list are updated with
              the vectors and ids of the first list")
739       {
740         lists.update_entries(list_ids[0], vectors, ids,
                0, list_length);
741
742         const vector_el_t *list_vectors =
                lists.get_vectors(list_ids[0]);
743         THEN("all vectors of the first list are correctly
                retrieved")
744         {
745           for (len_t i = 0; i < list_length; i++)
```

```
746             {
747               REQUIRE(list_vectors[i] == vectors[i]);
748             }
749           }
750
751           AND_WHEN("the vectors of a list are retrieved
                  which does not exist")
752           {
753             THEN("an exception is thrown")
754             {
755               REQUIRE_THROWS_AS(lists.get_vectors(list_ids[1]),
                      std::invalid_argument);
756             }
757           }
758         }
759       }
760     }
761   }
762
763   SCENARIO("get_ids(): the ids of an inverted list can be
        retrieved", "[.StorageLists]")
764   {
765     GIVEN("an StorageLists object and a list of 128D
          vectors and corresponding ids")
766     {
767       len_t vector_dim = 128;
768       StorageLists lists =
            get_inverted_lists_object(vector_dim);
769
770       auto data = gen_vectors(128);
771       len_t list_length = get_vector_length(data,
            vector_dim);
772       vector_el_t *vectors = to_ptr(vector_el_t,
            data.first);
773       vector_id_t *ids = to_ptr(vector_id_t, data.second);
774
775       WHEN("an inverted list is created")
776       {
777         auto list_ids = gen_list_ids(2);
778         lists.create_list(list_ids[0], list_length);
779
780         AND_WHEN("all entries of the list are updated with
              the vectors and ids of the first list")
781         {
782           lists.update_entries(list_ids[0], vectors, ids,
                0, list_length);
783
784           const list_id_t *actual_ids =
                lists.get_ids(list_ids[0]);
```

```
785         THEN("all ids of the first list are correctly
                retrieved")
786         {
787           for (len_t i = 0; i < list_length; i++)
788           {
789             REQUIRE(actual_ids[i] == ids[i]);
790           }
791         }
792
793         AND_WHEN("the ids of a list are retrieved which
                does not exist")
794         {
795           THEN("an exception is thrown")
796           {
797             REQUIRE_THROWS_AS(lists.get_ids(list_ids[1]),
                   std::invalid_argument);
798           }
799         }
800       }
801     }
802   }
803 }
804
805 auto test_bulk_insert_entries =
806     [](
807         len_t n_entries,
808         len_t vector_dim,
809         vector_el_t *vectors,
810         vector_id_t *ids,
811         list_id_t *list_ids,
812         std::string vectors_filepath,
813         std::string vector_ids_filepath,
814         std::string list_ids_filepath)
815 {
816   GIVEN("an StorageLists object and a list of 128D
       vectors and corresponding ids")
817   {
818     StorageLists lists =
          get_inverted_lists_object(vector_dim);
819     WHEN("the entries are bulk inserted")
820     {
821       lists.bulk_insert_entries(vectors_filepath,
             vector_ids_filepath, list_ids_filepath,
             n_entries);
822
823       std::unordered_map<list_id_t,
             std::vector<vector_el_t *>>
824           list_vectors_map;
825       std::unordered_map<list_id_t,
```

```
          std::vector<list_id_t>> list_ids_map;
826       for (len_t i = 0; i < n_entries; i++)
827       {
828         list_vectors_map[list_ids[i]].push_back(&vectors[i
              * vector_dim]);
829         list_ids_map[list_ids[i]].push_back(ids[i]);
830       }
831
832       THEN("the inverted lists exist and have the correct
            length")
833       {
834         for (auto list : list_vectors_map)
835         {
836           REQUIRE(lists.get_list_length(list.first) ==
                list.second.size());
837         }
838       }
839       THEN("the inverted lists have the correct vectors")
840       {
841         for (auto list : list_vectors_map)
842         {
843           len_t list_length = list.second.size();
844           const vector_el_t *actual_vectors =
                lists.get_vectors(list.first);
845           for (len_t i = 0; i < list_length; i++)
846           {
847             for (len_t j = 0; j < vector_dim; j++)
848             {
849               REQUIRE(actual_vectors[i * vector_dim + j]
                    == list.second[i][j]);
850             }
851           }
852         }
853       }
854       THEN("the inverted lists have the correct ids")
855       {
856         for (auto list : list_ids_map)
857         {
858           const list_id_t *actual_ids =
                lists.get_ids(list.first);
859           for (len_t i = 0; i < list.second.size(); i++)
860           {
861             REQUIRE(actual_ids[i] == list.second[i]);
862           }
863         }
864       }
865       THEN("the total size is updated")
866       {
867         size_t total_size = 0;
```

```
868         for (auto list : list_vectors_map)
869         {
870           total_size += get_list_size(vector_dim,
                list.second.size());
871         }
872
873         REQUIRE(lists.get_total_size() ==
              get_total_size(total_size));
874       }
875       AND_WHEN("the entries are bulk inserted again")
876       {
877         THEN("an exception is thrown")
878         {
879           REQUIRE_THROWS_AS(lists.bulk_insert_entries(vectors_filepath,
                vector_ids_filepath, list_ids_filepath,
                n_entries), std::runtime_error);
880         }
881       }
882     }
883   }
884 };
885
886 auto benchmark_bulk_insert_entries =
887     [](
888         len_t n_entries,
889         len_t vector_dim,
890         vector_el_t *vectors,
891         vector_id_t *ids,
892         list_id_t *list_ids,
893         std::string vectors_filepath,
894         std::string vector_ids_filepath,
895         std::string list_ids_filepath)
896 {
897   UNUSED(vectors);
898   UNUSED(ids);
899   UNUSED(list_ids);
900
901   size_t file_size = 0;
902   BENCHMARK_ADVANCED("bulk_insert_entries()")
903   (Catch::Benchmark::Chronometer meter)
904   {
905     StorageLists lists =
          get_inverted_lists_object(vector_dim);
906     meter.measure([&lists, vectors_filepath,
          vector_ids_filepath, list_ids_filepath, n_entries,
          &file_size]
907                   { return
                        lists.bulk_insert_entries(vectors_filepath,
                        vector_ids_filepath,
```

```
                        list_ids_filepath, n_entries); });
908     std::string filename = lists.get_filename();
909     file_size = get_file_size(filename);
910   };
911   WARN("file_size := " << std::to_string(file_size));
912 };
913
914 void bench_bulk_insert_entries_dataset(std::string
      dataset, len_t n_entries, len_t vector_dim, len_t
      n_lists, bool is_dataset_sorted = false)
915 {
916   WARN("n_lists := " << std::to_string(n_lists));
917   GIVEN("the " + dataset + " dataset which has already
        been clusered and  written to files according to the
        format required by bulk_insert_entries()")
918   {
919     std::string dataset_dir = join(SIFT_OUTPUT_DIR,
          dataset);
920     std::string vectors_filepath = join(dataset_dir,
          get_vectors_filename(is_dataset_sorted, n_lists));
921     std::string vector_ids_filepath = join(dataset_dir,
          get_vector_ids_filename(is_dataset_sorted,
          n_lists));
922     std::string list_ids_filepath = join(dataset_dir,
          get_list_ids_filename(n_lists, is_dataset_sorted));
923
924     setup_run_teardown_bulk_insert_entries_dataset(
925         n_entries,
926         vector_dim,
927         vectors_filepath,
928         vector_ids_filepath,
929         list_ids_filepath,
930         benchmark_bulk_insert_entries);
931   }
932 }
933
934 void test_bulk_insert_entries_dataset(std::string
      dataset, len_t n_entries, len_t vector_dim, len_t
      n_lists, bool is_dataset_sorted = false)
935 {
936   WARN("n_lists := " << std::to_string(n_lists));
937   GIVEN("the " + dataset + " dataset which has already
        been clusered and  written to files according to the
        format required by bulk_insert_entries()")
938   {
939     std::string dataset_dir = join(SIFT_OUTPUT_DIR,
          dataset);
940     std::string vectors_filepath = join(dataset_dir,
          get_vectors_filename(is_dataset_sorted, n_lists));
```

```
941     std::string vector_ids_filepath = join(dataset_dir,
            get_vector_ids_filename(is_dataset_sorted,
            n_lists));
942     std::string list_ids_filepath = join(dataset_dir,
            get_list_ids_filename(n_lists, is_dataset_sorted));
943
944     setup_run_teardown_bulk_insert_entries_dataset(
945         n_entries,
946         vector_dim,
947         vectors_filepath,
948         vector_ids_filepath,
949         list_ids_filepath,
950         test_bulk_insert_entries);
951   }
952 }
953
954 SCENARIO("bulk_insert_entries(): randomized testing",
        "[StorageLists][bulk_insert_entries][test][random]")
955 {
956   GIVEN("an StorageLists object, a list of vectors, ids
        and list ids")
957   {
958     len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
959     StorageLists lists =
            get_inverted_lists_object(vector_dim);
960
961     auto data = gen_vectors(128);
962     auto list_ids_data = gen_list_lengths_random_length();
963     len_t n_entries = get_clustered_vectors_length(data,
            list_ids_data, vector_dim);
964     vector_el_t *vectors = to_ptr(vector_el_t,
            data.first);
965     vector_id_t *ids = to_ptr(vector_id_t, data.second);
966     list_id_t *list_ids = to_ptr(list_id_t,
            list_ids_data);
967
968     AND_GIVEN("the vectors, ids and list ids are written
            to files according to the format required by
            bulk_insert_entries()")
969     {
970       std::string vectors_filepath = join(TMP_DIR,
              get_vectors_filename());
971       std::string vector_ids_filepath = join(TMP_DIR,
              VECTOR_IDS_FILENAME);
972       std::string list_ids_filepath = join(TMP_DIR,
              get_list_ids_filename(0));
973       size_t vectors_size = n_entries * vector_dim *
              sizeof(vector_el_t);
974       size_t ids_size = n_entries * sizeof(list_id_t);
```

```
975         size_t list_ids_size = n_entries *
                sizeof(list_id_t);
976
977         write_to_file(vectors_filepath, vectors,
                vectors_size);
978         write_to_file(vector_ids_filepath, ids, ids_size);
979         write_to_file(list_ids_filepath, list_ids,
                list_ids_size);
980
981         test_bulk_insert_entries(
982             n_entries,
983             vector_dim,
984             vectors,
985             ids,
986             list_ids,
987             vectors_filepath,
988             vector_ids_filepath,
989             list_ids_filepath);
990       }
991     }
992 }
993
994 SCENARIO("test bulk_insert_entries with SIFT1M",
        "[StorageLists][bulk_insert_entries][test][SIFT1M]")
995 {
996   len_t n_entries = (len_t)1E6;
997   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
998   len_t n_lists = GENERATE(TEST_N_LISTS);
999   test_bulk_insert_entries_dataset("SIFT1M", n_entries,
        vector_dim, n_lists);
1000 }
1001
1002 SCENARIO("test bulk_insert_entries with sorted SIFT1M",
        "[StorageLists][bulk_insert_entries][test][SIFT1M][sorted]")
1003 {
1004   len_t n_entries = (len_t)1E6;
1005   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
1006   len_t n_lists = GENERATE(TEST_N_LISTS);
1007   test_bulk_insert_entries_dataset("SIFT1M", n_entries,
        vector_dim, n_lists, true);
1008 }
1009
1010 SCENARIO("benchmark bulk_insert_entries with SIFT1M",
        "[StorageLists][bulk_insert_entries][.benchmark][SIFT1M]")
1011 {
1012   len_t n_entries = (len_t)1E6;
1013   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
1014   len_t n_lists = GENERATE(TEST_N_LISTS);
1015   bench_bulk_insert_entries_dataset("SIFT1M", n_entries,
```

```
      vector_dim, n_lists);
}

SCENARIO("benchmark bulk_insert_entries with sorted SIFT1M",
    "[StorageLists][bulk_insert_entries][.benchmark][SIFT1M][sorted]")
{
  len_t n_entries = (len_t)1E6;
  len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
  len_t n_lists = GENERATE(TEST_N_LISTS);
  bench_bulk_insert_entries_dataset("SIFT1M", n_entries,
      vector_dim, n_lists, true);
}

SCENARIO("test bulk_insert_entries with SIFT10M",
    "[StorageLists][bulk_insert_entries][.test][SIFT10M]")
{
  len_t n_entries = (len_t)1E7;
  len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
  len_t n_lists = GENERATE(TEST_N_LISTS);
  test_bulk_insert_entries_dataset("SIFT10M", n_entries,
      vector_dim, n_lists);
}

SCENARIO("benchmark bulk_insert_entries with SIFT10M",
    "[StorageLists][bulk_insert_entries][.benchmark][SIFT10M]")
{
  len_t n_entries = (len_t)1E7;
  len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
  len_t n_lists = GENERATE(TEST_N_LISTS);
  bench_bulk_insert_entries_dataset("SIFT10M", n_entries,
      vector_dim, n_lists);
}

SCENARIO("benchmark bulk_insert_entries with sorted SIFT10M",
    "[StorageLists][bulk_insert_entries][.benchmark][SIFT10M][sorted]")
{
  len_t n_entries = (len_t)1E7;
  len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
  len_t n_lists = GENERATE(TEST_N_LISTS);
  bench_bulk_insert_entries_dataset("SIFT10M", n_entries,
      vector_dim, n_lists, true);
}

SCENARIO("test bulk_insert_entries with SIFT100M",
    "[StorageLists][bulk_insert_entries][.test][SIFT100M]")
{
  len_t n_entries = (len_t)1E8;
```

```
1053   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
1054   len_t n_lists = GENERATE(TEST_N_LISTS);
1055   test_bulk_insert_entries_dataset("SIFT100M", n_entries,
          vector_dim, n_lists);
1056 }
1057
1058 SCENARIO("benchmark bulk_insert_entries with SIFT100M",
       "[StorageLists][bulk_insert_entries][.benchmark][SIFT100M]")
1059 {
1060   len_t n_entries = (len_t)1E8;
1061   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
1062   len_t n_lists = GENERATE(TEST_N_LISTS);
1063   bench_bulk_insert_entries_dataset("SIFT100M",
          n_entries, vector_dim, n_lists);
1064 }
1065
1066 SCENARIO("benchmark bulk_insert_entries with sorted
       SIFT100M",
       "[StorageLists][bulk_insert_entries][.benchmark][SIFT100M][sorted]")
1067 {
1068   len_t n_entries = (len_t)1E8;
1069   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
1070   len_t n_lists = GENERATE(TEST_N_LISTS);
1071   bench_bulk_insert_entries_dataset("SIFT100M",
          n_entries, vector_dim, n_lists, true);
1072 }
1073
1074 SCENARIO("test bulk_insert_entries with SIFT1B",
       "[StorageLists][bulk_insert_entries][.test][SIFT1B][SIFT1000M]")
1075 {
1076   len_t n_entries = (len_t)1E9;
1077   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
1078   len_t n_lists = GENERATE(TEST_N_LISTS);
1079   test_bulk_insert_entries_dataset("SIFT1B", n_entries,
          vector_dim, n_lists);
1080 }
1081
1082 SCENARIO("benchmark bulk_insert_entries with SIFT1B",
       "[StorageLists][bulk_insert_entries][.benchmark][SIFT1B][SIFT1000M]")
1083 {
1084   len_t n_entries = (len_t)1E9;
1085   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
1086   len_t n_lists = GENERATE(TEST_N_LISTS);
1087   bench_bulk_insert_entries_dataset("SIFT1B", n_entries,
          vector_dim, n_lists);
1088 }
```

Listing 9.12: tests/StorageListsTest.cpp

```
1 #include <sys/mman.h>
```

```
2  #include <unordered_set>
3  #include <cmath>
4
5  #include "../lib/catch.hpp"
6
7  #include "../include/tests/StorageListsTestUtils.hpp"
8  #include "../include/storage-node/StorageIndex.hpp"
9  #include "../include/L2Space.hpp"
10 #include "../include/root-node/RootIndex.hpp"
11 #include "../include/Query.hpp"
12
13 #define UNUSED(x) (void)(x)
14 #define N_RESULTS_GROUNDTRUTH 1000
15
16 #ifndef TEST_VECTOR_DIM
17 #define TEST_VECTOR_DIM 128
18 #endif
19 #ifndef TEST_N_SAMPLES
20 #define TEST_N_SAMPLES 3
21 #endif
22 #ifndef TEST_N_LISTS
23 #define TEST_N_LISTS 1024
24 #endif
25 #ifndef TEST_N_PROBES
26 #define TEST_N_PROBES 32
27 #endif
28 #ifndef TEST_N_RESULTS
29 #define TEST_N_RESULTS 1
30 #endif
31 #ifndef TEST_N_THREADS
32 #define TEST_N_THREADS 64
33 #endif
34
35 #ifdef _OPENMP
36 #include <omp.h>
37 #endif
38
39 using namespace ann_dkvs;
40
41 auto alloc_query_as_vector_el = [](uint8_t *query_vector,
      len_t vector_dim)
42 {
43   vector_el_t *query_vector_float = (vector_el_t
       *)malloc(vector_dim * sizeof(vector_el_t));
44   for (len_t i = 0; i < vector_dim; i++)
45   {
46     query_vector_float[i] = (vector_el_t)query_vector[i];
47   }
48   return query_vector_float;
```

```
49 };
50
51 std::string get_centroids_filename(len_t n_lists)
52 {
53   std::string filename = CENTROIDS_FILENAME;
54   std::string seperator = "_";
55   std::string file_ext = FILE_EXT;
56   return filename + seperator + std::to_string(n_lists) +
        file_ext;
57 }
58
59 SCENARIO("search_preassigned(): use index to find top k
    ANN of a query vector",
    "[StorageIndex][search_preassigned][test][hard-coded]")
60 {
61   GIVEN("an StorageLists object and a list of 1D vectors,
       corresponding vector ids and list ids")
62   {
63     len_t vector_dim = 1;
64     StorageLists lists =
          get_inverted_lists_object(vector_dim);
65     len_t list1_length = 2;
66     vector_el_t vectors_list1[] = {2, 4};
67     vector_id_t ids_list1[] = {1, 2};
68
69     len_t list2_length = 2;
70     vector_el_t vectors_list2[] = {1, 3};
71     vector_id_t ids_list2[] = {3, 4};
72
73     list_id_t list_ids[] = {1, 2};
74
75     AND_GIVEN("A query vector, the list ids to search and
          a value of k")
76     {
77       vector_el_t query_vector[] = {0};
78       len_t n_results = 3;
79       list_id_t list_ids_to_probe[] = {1, 2};
80       len_t n_probes = 2;
81       Query query = Query(query_vector,
            list_ids_to_probe, n_results, n_probes);
82
83       WHEN("the StorageLists object is populated with the
            vectors, ids and list ids and used to initialize
            an StorageIndex object")
84       {
85         lists.insert_entries(list_ids[0], vectors_list1,
              ids_list1, list1_length);
86         lists.insert_entries(list_ids[1], vectors_list2,
              ids_list2, list2_length);
```

```
87
88          StorageIndex index(&lists);
89
90          WHEN("the StorageIndex is queried with a query
                vector")
91          {
92            QueryResults results =
                 index.search_preassigned(&query);
93
94            THEN("the results are correct")
95            {
96              CHECK(results[0].distance == 1 * 1);
97              CHECK(results[0].vector_id == 3);
98              CHECK(results[1].distance == 2 * 2);
99              CHECK(results[1].vector_id == 1);
100             CHECK(results[2].distance == 3 * 3);
101             CHECK(results[2].vector_id == 4);
102           }
103         }
104       }
105     }
106   }
107 }
108
109 auto setup_indices_and_run = [](len_t n_probes,
110                                 len_t n_lists,
111                                 len_t n_entries,
112                                 len_t n_query_vectors,
113                                 len_t
                                     n_results_groundtruth,
114                                 len_t vector_dim,
115                                 bool mmap_groundtruth,
116                                 std::string dataset,
117                                 std::string
                                     groundtruth_filename,
118                                 std::function<void(uint8_t
                                     *, uint32_t *,
                                     StorageIndex *,
                                     RootIndex *)> run)
119 {
120   if (n_probes <= n_lists)
121   {
122     std::string dataset_dir = join(SIFT_OUTPUT_DIR,
          dataset);
123     // groundtruth format: sizeof(uint32_t) +
          [n_results_groundtruth * sizeof(uint32_t)]
124     std::string groundtruth_filepath =
          join(SIFT_GROUNDTRUTH_DIR, groundtruth_filename);
125     // vectors format: n_entries * 128 * sizeof(float)
```

```
126     std::string vectors_filepath = join(dataset_dir,
            get_vectors_filename());
127     // vector ids format: n_entries * sizeof(int64_t)
128     std::string vectors_ids_filepath = join(dataset_dir,
            get_vector_ids_filename());
129     // list_ids format: n_entries * sizeof(int64_t)
130     std::string list_ids_filepath = join(dataset_dir,
            get_list_ids_filename(n_lists));
131     // centroids format: n_lists * 128 * sizeof(float)
132     std::string centroids_filepath = join(dataset_dir,
            get_centroids_filename(n_lists));
133     // query_vectors format: n_query_vectors *
            [sizeof(uint32_t) + 128 * sizeof(uint8_t)]
134     std::string query_vectors_filepath =
            SIFT_QUERY_VECTORS_FILEPATH;
135
136     THEN("all required files are present")
137     {
138       REQUIRE(file_exists(vectors_filepath));
139       REQUIRE(file_exists(vectors_ids_filepath));
140       REQUIRE(file_exists(list_ids_filepath));
141       REQUIRE(file_exists(centroids_filepath));
142       REQUIRE(file_exists(query_vectors_filepath));
143       REQUIRE(file_exists(groundtruth_filepath));
144
145       WHEN("the files are mapped to memory")
146       {
147         len_t vectors_size = n_entries * vector_dim *
                sizeof(vector_el_t);
148         len_t ids_size = n_entries * sizeof(list_id_t);
149         len_t list_ids_size = n_entries *
                sizeof(list_id_t);
150         len_t centroids_size = n_lists * vector_dim *
                sizeof(vector_el_t);
151         len_t query_vectors_size = n_query_vectors *
                vector_dim * sizeof(vector_el_t);
152         len_t groundtruth_size = n_query_vectors *
                n_results_groundtruth * sizeof(vector_id_t);
153
154         vector_el_t *vectors = (vector_el_t
                *)mmap_file(vectors_filepath, vectors_size);
155         vector_id_t *vector_ids = (vector_id_t
                *)mmap_file(vectors_ids_filepath, ids_size);
156         list_id_t *list_ids = (list_id_t
                *)mmap_file(list_ids_filepath, list_ids_size);
157         vector_el_t *centroids = (vector_el_t
                *)mmap_file(centroids_filepath,
                centroids_size);
158         uint8_t *query_vectors = (uint8_t
```

```
            *)mmap_file(query_vectors_filepath,
            query_vectors_size);
159         uint32_t *groundtruth = nullptr;
160         if (mmap_groundtruth)
161         {
162           groundtruth = (uint32_t
                *)mmap_file(groundtruth_filepath,
                groundtruth_size);
163         }
164
165         THEN("the files are mapped correctly")
166         {
167           REQUIRE(vectors != nullptr);
168           REQUIRE(vector_ids != nullptr);
169           REQUIRE(list_ids != nullptr);
170           REQUIRE(centroids != nullptr);
171           REQUIRE(query_vectors != nullptr);
172           if (mmap_groundtruth)
173           {
174             REQUIRE(groundtruth != nullptr);
175           }
176         }
177
178         WHEN("the StorageLists object is populated with
                the vectors, ids and list ids and used to
                initialize an StorageIndex object")
179         {
180
181           StorageLists lists =
                get_inverted_lists_object(vector_dim);
182           lists.bulk_insert_entries(vectors_filepath,
                vectors_ids_filepath, list_ids_filepath,
                n_entries);
183           StorageIndex storage_index(&lists);
184           RootIndex root_index(vector_dim, centroids,
                n_lists);
185
186           run(query_vectors, groundtruth, &storage_index,
                &root_index);
187         }
188         munmap(vectors, vectors_size);
189         munmap(vector_ids, ids_size);
190         munmap(list_ids, list_ids_size);
191         munmap(centroids, centroids_size);
192         munmap(query_vectors, query_vectors_size);
193         if (mmap_groundtruth)
194         {
195           munmap(groundtruth, groundtruth_size);
196         }
```

```
197       }
198     }
199   }
200 };
201
202 auto prepare_query = [](uint8_t *query_vectors, len_t
      query_id, len_t vector_dim, len_t n_results, len_t
      n_probes)
203 {
204   uint8_t *query_bytes = &query_vectors[query_id *
        (vector_dim + 4) + 4];
205   vector_el_t *query_vector =
        alloc_query_as_vector_el(query_bytes, vector_dim);
206   Query *query = new Query(query_vector, n_results,
        n_probes);
207   return query;
208 };
209
210 auto prepare_queries = [](uint8_t *query_vectors, len_t
      n_query_vectors, len_t vector_dim, len_t n_results,
      len_t n_probes)
211 {
212   QueryBatch queries;
213   for (len_t query_id = 0; query_id < n_query_vectors;
        query_id++)
214   {
215     Query *query = prepare_query(query_vectors, query_id,
          vector_dim, n_results, n_probes);
216     queries.push_back(query);
217   }
218   return queries;
219 };
220
221 auto free_query = [](Query *query)
222 {
223   free(query->get_query_vector());
224   delete query;
225 };
226
227 auto free_queries = [](QueryBatch queries)
228 {
229   for (len_t query_id = 0; query_id < queries.size();
        query_id++)
230   {
231     free_query(queries[query_id]);
232   }
233 };
234
235 auto get_recall_at_r = [](const QueryResultsBatch
```

```
    &result_batch, const uint32_t *groundtruth, len_t
    n_query_vectors, len_t n_results_groundtruth, len_t
    n_results, len_t r)
{
  REQUIRE(r <= n_results);
  len_t n_correct = 0;
  for (len_t query_id = 0; query_id < n_query_vectors;
      query_id++)
  {
    const QueryResults results = result_batch[query_id];
    vector_id_t groundtruth_first_nearest_neighbor =
        groundtruth[query_id * (n_results_groundtruth + 1)
        + 1];
    for (len_t result_id = 0; result_id < r; result_id++)
    {
      const vector_id_t id = results[result_id].vector_id;
      if (id == groundtruth_first_nearest_neighbor)
      {
        n_correct++;
        break;
      }
    }
  }
  return (float)n_correct / n_query_vectors;
};

auto get_median_95th_99th_percentile = [](const
    std::vector<long> &values)
{
  std::vector<long> sorted_values = values;
  std::sort(sorted_values.begin(), sorted_values.end());
  len_t n_values = sorted_values.size();
  len_t median_index = n_values / 2;
  len_t percentile_95_index = (len_t)(n_values * 0.95);
  len_t percentile_99_index = (len_t)(n_values * 0.99);
  return std::make_tuple(sorted_values[median_index],
      sorted_values[percentile_95_index],
      sorted_values[percentile_99_index]);
};

auto measure_search_preassigned_latency = [](StorageIndex
    *storage_index, RootIndex *root_index, uint8_t
    *query_vectors, len_t n_query_vectors, len_t
    vector_dim, len_t n_results, len_t n_probes)
{
  std::vector<long> latencies(n_query_vectors);
  for (len_t query_id = 0; query_id < n_query_vectors;
      query_id++)
  {
```

```
272     Query *query = prepare_query(query_vectors, query_id,
            vector_dim, n_results, n_probes);
273     QueryBatch queries = {query};
274     root_index->batch_preassign_queries(queries);
275     auto begin =
            std::chrono::high_resolution_clock::now();
276
277     storage_index->batch_search_preassigned(queries);
278
279     auto end = std::chrono::high_resolution_clock::now();
280     auto latency_ns =
            std::chrono::duration_cast<std::chrono::nanoseconds>(end
            - begin).count();
281     latencies[query_id] = latency_ns;
282     free_query(query);
283   }
284   return latencies;
285 };
286
287 auto get_median_95th_99th_percentile_mean_std_latency =
      [](StorageIndex *storage_index, RootIndex *root_index,
      uint8_t *query_vectors, len_t n_query_vectors, len_t
      vector_dim, len_t n_results, len_t n_probes)
288 {
289   std::vector<long> warmup_latencies =
        measure_search_preassigned_latency(storage_index,
        root_index, query_vectors, n_query_vectors,
        vector_dim, n_results, n_probes);
290
291   std::vector<std::tuple<long, long, long>>
        median_95th_99th_runs(TEST_N_SAMPLES);
292
293   for (len_t run = 0; run < TEST_N_SAMPLES; run++)
294   {
295     std::vector<long> run_latencies =
          measure_search_preassigned_latency(storage_index,
          root_index, query_vectors, n_query_vectors,
          vector_dim, n_results, n_probes);
296     median_95th_99th_runs[run] =
          get_median_95th_99th_percentile(run_latencies);
297   }
298
299   long latency_median_mean = 0;
300   long latency_median_std = 0;
301   long latency_95th_mean = 0;
302   long latency_95th_std = 0;
303   long latency_99th_mean = 0;
304   long latency_99th_std = 0;
305
```

```
306   for (len_t run = 0; run < TEST_N_SAMPLES; run++)
307   {
308     latency_median_mean +=
          std::get<0>(median_95th_99th_runs[run]);
309     latency_95th_mean +=
          std::get<1>(median_95th_99th_runs[run]);
310     latency_99th_mean +=
          std::get<2>(median_95th_99th_runs[run]);
311   }
312   latency_median_mean /= TEST_N_SAMPLES;
313   latency_95th_mean /= TEST_N_SAMPLES;
314   latency_99th_mean /= TEST_N_SAMPLES;
315
316   for (len_t run = 0; run < TEST_N_SAMPLES; run++)
317   {
318     latency_median_std +=
          std::pow(std::get<0>(median_95th_99th_runs[run]) -
          latency_median_mean, 2);
319     latency_95th_std +=
          std::pow(std::get<1>(median_95th_99th_runs[run]) -
          latency_95th_mean, 2);
320     latency_99th_std +=
          std::pow(std::get<2>(median_95th_99th_runs[run]) -
          latency_99th_mean, 2);
321   }
322   latency_median_std = std::sqrt(latency_median_std /
        TEST_N_SAMPLES);
323   latency_95th_std = std::sqrt(latency_95th_std /
        TEST_N_SAMPLES);
324   latency_99th_std = std::sqrt(latency_99th_std /
        TEST_N_SAMPLES);
325
326   return std::make_tuple(latency_median_mean,
        latency_median_std, latency_95th_mean,
        latency_95th_std, latency_99th_mean,
        latency_99th_std);
327 };
328
329 SCENARIO("search_preassigned(): test recall with SIFT1M",
      "[StorageIndex][search_preassigned][test][recall][SIFT1M]")
330 {
331   GIVEN("the SIFT1M dataset")
332   {
333     len_t vector_dim = 128;
334     len_t n_entries = (len_t)1E6;
335     len_t n_query_vectors = (len_t)1E4;
336     len_t n_results_groundtruth = N_RESULTS_GROUNDTRUTH;
337     len_t n_lists = GENERATE(TEST_N_LISTS);
338     len_t n_probes = GENERATE(TEST_N_PROBES);
```

```
339     len_t n_results = GENERATE(TEST_N_RESULTS);
340 #ifdef _OPENMP
341     len_t n_threads = GENERATE(TEST_N_THREADS);
342     omp_set_num_threads(n_threads);
343 #endif
344
345     auto run = [=](uint8_t *query_vectors, uint32_t
            *groundtruth, StorageIndex *storage_index,
            RootIndex *root_index)
346     {
347       WHEN("for each query vector, the closest centroids
              are determined, their lists are searched for the
              nearest n_results neighbors")
348       {
349 #ifdef _OPENMP
350         WARN("max_n_threads := " <<
                omp_get_max_threads());
351 #endif
352         WARN("n_lists := " << n_lists);
353         WARN("n_probes := " << n_probes);
354         WARN("n_results := " << n_results);
355
356         QueryBatch queries =
                prepare_queries(query_vectors,
                n_query_vectors, vector_dim, n_results,
                n_probes);
357
358         root_index->batch_preassign_queries(queries);
359         QueryResultsBatch results =
                storage_index->batch_search_preassigned(queries);
360
361         float recall_at_1 = get_recall_at_r(results,
                groundtruth, n_query_vectors,
                n_results_groundtruth, n_results, 1);
362
363         WARN("Recall@1 := " << recall_at_1);
364         THEN("the Recall@1 is 100% if we search all
                lists")
365         {
366           if (n_probes == n_lists)
367           {
368             REQUIRE(recall_at_1 == 1.0);
369           }
370         }
371
372         free_queries(queries);
373       }
374     };
375
```

```
376       setup_indices_and_run(n_probes, n_lists, n_entries,
              n_query_vectors, n_results_groundtruth,
              vector_dim, true, "SIFT1M", "idx_1M.ivecs", run);
377     }
378   }
379
380   SCENARIO("search_preassigned(): throughput benchmark
          querying with SIFT1M",
          "[StorageIndex][search_preassigned][benchmark][throughput][SIFT1M]")
381   {
382     len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
383     len_t n_entries = (len_t)1E6;
384     len_t n_query_vectors = (len_t)1E4;
385     len_t n_results_groundtruth = N_RESULTS_GROUNDTRUTH;
386     len_t n_lists = GENERATE(TEST_N_LISTS);
387     len_t n_probes = GENERATE(TEST_N_PROBES);
388     len_t n_results = GENERATE(TEST_N_RESULTS);
389   #ifdef _OPENMP
390     len_t n_threads = GENERATE(TEST_N_THREADS);
391     omp_set_num_threads(n_threads);
392   #endif
393
394     auto run = [=](uint8_t *query_vectors, uint32_t
            *groundtruth, StorageIndex *storage_index, RootIndex
            *root_index)
395     {
396       UNUSED(groundtruth);
397       WHEN("for each query vector, the closest centroids
              are determined, their lists are searched for the
              nearest n_results neighbors")
398       {
399   #ifdef _OPENMP
400         WARN("max_n_threads := " << omp_get_max_threads());
401   #endif
402         WARN("n_lists := " << n_lists);
403         WARN("n_probes := " << n_probes);
404         WARN("n_results := " << n_results);
405         BENCHMARK_ADVANCED("search_preassigned():
                measurement excludes preassigning queries")
406         (Catch::Benchmark::Chronometer meter)
407         {
408           QueryBatch queries =
                prepare_queries(query_vectors,
                n_query_vectors, vector_dim, n_results,
                n_probes);
409           root_index->batch_preassign_queries(queries);
410           meter.measure([&storage_index, &queries]
411                         {
                                storage_index->batch_search_preassigned(queries);
```

```
                                });
412             free_queries(queries);
413           };
414       }
415   };
416
417   setup_indices_and_run(n_probes, n_lists, n_entries,
          n_query_vectors, n_results_groundtruth, vector_dim,
          false, "SIFT1M", "idx_1M.ivecs", run);
418 }
419
420 SCENARIO("search_preassigned(): latency benchmark
      querying with SIFT1M",
      "[StorageIndex][search_preassigned][benchmark][latency][SIFT1M]")
421 {
422   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
423   len_t n_entries = (len_t)1E6;
424   len_t n_query_vectors = (len_t)1E4;
425   len_t n_results_groundtruth = N_RESULTS_GROUNDTRUTH;
426   len_t n_lists = GENERATE(TEST_N_LISTS);
427   len_t n_probes = GENERATE(TEST_N_PROBES);
428   len_t n_results = GENERATE(TEST_N_RESULTS);
429 #ifdef _OPENMP
430   len_t n_threads = GENERATE(TEST_N_THREADS);
431   omp_set_num_threads(n_threads);
432 #endif
433
434   auto run = [=](uint8_t *query_vectors, uint32_t
          *groundtruth, StorageIndex *storage_index, RootIndex
          *root_index)
435   {
436     UNUSED(groundtruth);
437     WHEN("for each query vector, the closest centroids
            are determined, their lists are searched for the
            nearest n_results neighbors")
438     {
439 #ifdef _OPENMP
440       WARN("max_n_threads := " << omp_get_max_threads());
441 #endif
442       WARN("n_lists := " << n_lists);
443       WARN("n_probes := " << n_probes);
444       WARN("n_results := " << n_results);
445
446       auto mean_std_latencies =
              get_median_95th_99th_percentile_mean_std_latency(
447             storage_index, root_index, query_vectors,
                  n_query_vectors, vector_dim, n_results,
                  n_probes);
448
```

```
449       WARN("latency_50th_mean := " <<
              std::get<0>(mean_std_latencies));
450       WARN("latency_50th_std := " <<
              std::get<1>(mean_std_latencies));
451       WARN("latency_95th_mean := " <<
              std::get<2>(mean_std_latencies));
452       WARN("latency_95th_std := " <<
              std::get<3>(mean_std_latencies));
453       WARN("latency_99th_mean := " <<
              std::get<4>(mean_std_latencies));
454       WARN("latency_99th_std := " <<
              std::get<5>(mean_std_latencies));
455     }
456   };
457
458   setup_indices_and_run(n_probes, n_lists, n_entries,
        n_query_vectors, n_results_groundtruth, vector_dim,
        false, "SIFT1M", "idx_1M.ivecs", run);
459 }
460
461 SCENARIO("search_preassigned(): throughput benchmark
    querying with SIFT100M",
    "[StorageIndex][search_preassigned][benchmark][SIFT100M]")
462 {
463   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
464   len_t n_entries = (len_t)1E8;
465   len_t n_query_vectors = (len_t)1E5;
466   len_t n_results_groundtruth = N_RESULTS_GROUNDTRUTH;
467   len_t n_lists = GENERATE(TEST_N_LISTS);
468   len_t n_probes = GENERATE(TEST_N_PROBES);
469   len_t n_results = GENERATE(TEST_N_RESULTS);
470 #ifdef _OPENMP
471   len_t n_threads = GENERATE(TEST_N_THREADS);
472   omp_set_num_threads(n_threads);
473 #endif
474
475   auto run = [=](uint8_t *query_vectors, uint32_t
        *groundtruth, StorageIndex *storage_index, RootIndex
        *root_index)
476   {
477     UNUSED(groundtruth);
478     WHEN("for each query vector, the closest centroids
          are determined, their lists are searched for the
          nearest n_results neighbors")
479     {
480 #ifdef _OPENMP
481       WARN("max_n_threads := " << omp_get_max_threads());
482 #endif
483       WARN("n_lists := " << n_lists);
```

```
484         WARN("n_probes := " << n_probes);
485         WARN("n_results := " << n_results);
486         BENCHMARK_ADVANCED("search_preassigned():
                measurement excludes preassigning queries")
487         (Catch::Benchmark::Chronometer meter)
488         {
489           QueryBatch queries =
                prepare_queries(query_vectors,
                n_query_vectors, vector_dim, n_results,
                n_probes);
490           root_index->batch_preassign_queries(queries);
491           meter.measure([&storage_index, &queries]
492                         {
                                storage_index->batch_search_preassigned(queries)
                                });
493           free_queries(queries);
494         };
495       }
496     };
497
498     setup_indices_and_run(n_probes, n_lists, n_entries,
          n_query_vectors, n_results_groundtruth, vector_dim,
          false, "SIFT100M", "idx_100M.ivecs", run);
499   }
500
501   SCENARIO("preassign_query()",
        "[RootIndex][preassign_query][benchmark][SIFT1M][throughput]")
502   {
503     len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
504     len_t n_entries = (len_t)1E6;
505     len_t n_query_vectors = (len_t)1E4;
506     len_t n_results_groundtruth = N_RESULTS_GROUNDTRUTH;
507     len_t n_lists = GENERATE(TEST_N_LISTS);
508     len_t n_probes = GENERATE(TEST_N_PROBES);
509     len_t n_results = GENERATE(TEST_N_RESULTS);
510   #ifdef _OPENMP
511     len_t n_threads = GENERATE(TEST_N_THREADS);
512     omp_set_num_threads(n_threads);
513   #endif
514
515     auto run = [=](uint8_t *query_vectors, uint32_t
          *groundtruth, StorageIndex *storage_index, RootIndex
          *root_index)
516     {
517       UNUSED(groundtruth);
518       UNUSED(storage_index);
519       WHEN("for each query vector, the closest centroids
            are determined")
520       {
```

```
521 #ifdef _OPENMP
522       WARN("max_n_threads := " << omp_get_max_threads());
523 #endif
524       WARN("n_lists := " << n_lists);
525       WARN("n_probes := " << n_probes);
526       WARN("n_results := " << n_results);
527
528       BENCHMARK_ADVANCED("preassign_query(): does not
             measure search")
529       (Catch::Benchmark::Chronometer meter)
530       {
531         QueryBatch queries =
              prepare_queries(query_vectors,
              n_query_vectors, vector_dim, n_results,
              n_probes);
532         meter.measure([&root_index, &queries]
533                       {
                              root_index->batch_preassign_queries(queries);
                              });
534         free_queries(queries);
535       };
536     }
537   };
538   setup_indices_and_run(n_probes, n_lists, n_entries,
        n_query_vectors, n_results_groundtruth, vector_dim,
        false, "SIFT1M", "idx_1M.ivecs", run);
539 }
540
541 auto measure_preassign_query_latency(StorageIndex
     *storage_index, RootIndex *root_index, uint8_t
     *query_vectors, len_t n_query_vectors, len_t
     vector_dim, len_t n_results, len_t n_probes)
542 {
543   UNUSED(storage_index);
544   std::vector<long> latencies(n_query_vectors);
545   for (len_t query_id = 0; query_id < n_query_vectors;
        query_id++)
546   {
547     Query *query = prepare_query(query_vectors, query_id,
          vector_dim, n_results, n_probes);
548     QueryBatch queries = {query};
549     auto start =
          std::chrono::high_resolution_clock::now();
550
551     root_index->batch_preassign_queries(queries);
552
553     auto end = std::chrono::high_resolution_clock::now();
554     auto latency_ns =
          std::chrono::duration_cast<std::chrono::microseconds>(end
```

```
        - start).count();
555     latencies[query_id] = latency_ns;
556     free_query(query);
557   }
558   return latencies;
559 }
560
561 SCENARIO("preassign_query(): latency benchmark querying
        with SIFT1M",
        "[RootIndex][preassign_query][benchmark][latency][SIFT1M]")
562 {
563   len_t vector_dim = GENERATE(TEST_VECTOR_DIM);
564   len_t n_entries = (len_t)1E6;
565   len_t n_query_vectors = (len_t)1E4;
566   len_t n_results_groundtruth = N_RESULTS_GROUNDTRUTH;
567   len_t n_lists = GENERATE(TEST_N_LISTS);
568   len_t n_probes = GENERATE(TEST_N_PROBES);
569   len_t n_results = GENERATE(TEST_N_RESULTS);
570 #ifdef _OPENMP
571   len_t n_threads = GENERATE(TEST_N_THREADS);
572   omp_set_num_threads(n_threads);
573 #endif
574
575   auto run = [=](uint8_t *query_vectors, uint32_t
        *groundtruth, StorageIndex *storage_index, RootIndex
        *root_index)
576   {
577     UNUSED(groundtruth);
578     UNUSED(storage_index);
579     WHEN("for each query vector, the closest centroids
          are determined")
580     {
581 #ifdef _OPENMP
582       WARN("max_n_threads := " << omp_get_max_threads());
583 #endif
584       WARN("n_lists := " << n_lists);
585       WARN("n_probes := " << n_probes);
586       WARN("n_results := " << n_results);
587
588       auto mean_std_latencies =
            get_median_95th_99th_percentile_mean_std_latency(
589          storage_index, root_index, query_vectors,
               n_query_vectors, vector_dim, n_results,
               n_probes);
590
591       WARN("latency_50th_mean := " <<
            std::get<0>(mean_std_latencies));
592       WARN("latency_50th_std := " <<
            std::get<1>(mean_std_latencies));
```

```
593         WARN("latency_95th_mean := " <<
                std::get<2>(mean_std_latencies));
594         WARN("latency_95th_std := " <<
                std::get<3>(mean_std_latencies));
595         WARN("latency_99th_mean := " <<
                std::get<4>(mean_std_latencies));
596         WARN("latency_99th_std := " <<
                std::get<5>(mean_std_latencies));
597       }
598     };
599     setup_indices_and_run(n_probes, n_lists, n_entries,
          n_query_vectors, n_results_groundtruth, vector_dim,
          false, "SIFT1M", "idx_1M.ivecs", run);
600 }
```

Listing 9.13: tests/IndexTest.cpp

```
1  #pragma once
2
3  #include <iostream>
4  #include <fstream>
5
6  #include "../../lib/catch.hpp"
7
8  #include "TestConfig.hpp"
9  #include "../../include/storage-node/StorageLists.hpp"
10
11 using namespace ann_dkvs;
12
13 #define gen_random_ints(T, MIN_VAL, MAX_VAL, N_CHUNKS,
       CHUNK_LEN, EXCLUDE_SET) (GENERATE(take(N_CHUNKS,
       chunk(CHUNK_LEN, Catch::Generators::map([](T val) {
       return (T)val; }, filter([&](T val) {std::vector<T>
       exclude{EXCLUDE_SET};return find(exclude.begin(),
       exclude.end(), val) == exclude.end(); },
       random(MIN_VAL, MAX_VAL))))))
14
15 #define gen_random_floats(T, MIN_VAL, MAX_VAL, N_CHUNKS,
       CHUNK_LEN, EXCLUDE_SET) (GENERATE(take(N_CHUNKS,
       chunk(CHUNK_LEN, Catch::Generators::map([](T val) {
       return (T)val; }, filter([&](T val) {std::vector<T>
       exclude{EXCLUDE_SET};return find(exclude.begin(),
       exclude.end(), val) == exclude.end(); },
       random(MIN_VAL, MAX_VAL))))))
16
17 #define gen_ranged_values(T, MIN_VAL, MAX_VAL, N_CHUNKS,
       CHUNK_LEN, EXCLUDE_SET) (GENERATE(take(N_CHUNKS,
       chunk(CHUNK_LEN, Catch::Generators::map([](T val) {
       return (T)val; }, filter([&](T val) {std::vector<T>
       exclude{EXCLUDE_SET};return find(exclude.begin(),
```

```
    exclude.end(), val) == exclude.end(); },
    range(MIN_VAL, MAX_VAL)))))))

#define gen_random_int(T, MIN_VAL, MAX_VAL, N_VALS,
    EXCLUDE_SET) (gen_random_ints(T, MIN_VAL, MAX_VAL,
    N_VALS, 1, EXCLUDE_SET)[0])

#define to_ptr(T, V) ((T *)V.data())

#define gen_vector_dim(EXCLUDE_SET)
    (gen_random_int(len_t, 0, MAX_VECTOR_DIM,
    N_VECTOR_DIM_SAMPLES, EXCLUDE_SET))

#define gen_list_ids(CHUNK_LEN)
    gen_ranged_values(list_id_t, 0, MAX_LIST_ID,
    N_LIST_ID_SAMPLES, CHUNK_LEN, {})

#define gen_list_id() gen_list_ids(1)[0]

#define gen_list_lengths(CHUNK_LEN, EXCLUDE_SET)
    gen_random_ints(len_t, MIN_LIST_LENGTH,
    MAX_LIST_LENGTH, N_LIST_LENGTH_SAMPLES, CHUNK_LEN,
    EXCLUDE_SET)

#define gen_list_lengths_random_length()
    gen_list_lengths(random_range(1, MAX_N_LISTS), {})

#define gen_list_length(EXCLUDE_SET) gen_list_lengths(1,
    EXCLUDE_SET)[0]

#define gen_vector_ids_fixed(CHUNK_LEN)
    gen_ranged_values(vector_id_t, 0, MAX_VECTOR_ID,
    N_VECTOR_SAMPLES, CHUNK_LEN, {})

#define gen_random_vector_values_fixed(CHUNK_LEN)
    gen_random_floats(vector_el_t, MIN_VECTOR_VAL,
    MAX_VECTOR_VAL, N_VECTOR_SAMPLES, CHUNK_LEN, {})

#define gen_vectors_fixed(CHUNK_LEN, DIM)
    make_pair(gen_random_vector_values_fixed((CHUNK_LEN) *
    (DIM)), gen_vector_ids_fixed(CHUNK_LEN))

#define random_range(MIN, MAX) (rand() % (MAX - MIN + 1)
    + MIN)

#define gen_vectors(dim)
    gen_vectors_fixed(random_range(MIN_LIST_LENGTH,
    MAX_LIST_LENGTH), dim)

```

```
45 #define get_clustered_vectors_length(data, list_ids, dim)
       (std::min(get_vector_length(data, dim),
       list_ids.size()))
46
47 #define get_vector_length(data, dim)
       (std::min(data.first.size() / dim, data.second.size()))
48
49 #define UNUSED(x) (void)(x)
50
51 namespace ann_dkvs
52 {
53   size_t round_up_to_next_power_of_two(size_t value);
54   size_t is_power_of_two(size_t value);
55   size_t get_vector_size(len_t vector_dim);
56   size_t get_list_size(len_t vector_dim, len_t n_entries);
57   size_t get_total_size(size_t used_space);
58   void write_to_file(std::string filename, void *data,
         size_t size);
59   void read_from_file(std::string filename, void *data,
         size_t size);
60   void *mmap_file(std::string filename, size_t size);
61   bool file_exists(std::string filename);
62   std::string join(const std::string &a, const
         std::string &b);
63   StorageLists get_inverted_lists_object(len_t
         vector_dim);
64   void print_vector(vector_el_t *vector, len_t
         vector_dim, len_t n_entries);
65   void are_vectors_equal(const vector_el_t *actual, const
         vector_el_t *expected, len_t vector_dim, len_t
         n_entries);
66   void are_ids_equal(const vector_id_t *actual, const
         vector_id_t *expected, len_t n_entries);
67   std::string get_lists_filename();
68   std::string get_vectors_filename(bool is_dataset_sorted
         = false, len_t n_lists = 1024);
69   std::string get_vector_ids_filename(bool
         is_dataset_sorted = false, len_t n_lists = 1024);
70   std::string get_list_ids_filename(len_t n_lists, bool
         is_dataset_sorted = false);
71   size_t get_file_size(std::string filename);
72   void setup_run_teardown_bulk_insert_entries_dataset(
73       len_t n_entries,
74       len_t vector_dim,
75       std::string vectors_filepath,
76       std::string vector_ids_filepath,
77       std::string list_ids_filepath,
78       std::function<void(len_t, size_t, vector_el_t *,
           vector_id_t *, list_id_t *, std::string,
```

```
            std::string, std::string)>
            run_bulk_insert_entries);
}
```

Listing 9.14: include/tests/StorageListsTestUtils.hpp

```
#pragma once

#define TMP_DIR "tests/tmp"
#define SIFT_OUTPUT_DIR "cluster/out"
#define SIFT_GROUNDTRUTH_DIR "cluster/data/bigann/gnd"
#define SIFT_QUERY_VECTORS_FILEPATH
    "cluster/data/bigann/bigann_query.bvecs"
#define LISTS_FILENAME "lists"
#define VECTORS_FILENAME "vectors"
#define VECTOR_IDS_FILENAME "vector_ids"
#define LIST_IDS_FILENAME "list_ids"
#define CENTROIDS_FILENAME "centroids"
#define FILE_EXT ".bin"

#define MAX_VECTOR_DIM 128
#define MIN_LIST_LENGTH 1
#define MAX_LIST_LENGTH (int)1E3
#define MAX_N_LISTS 100

#define N_VECTOR_DIM_SAMPLES 5
#define N_LIST_ID_SAMPLES 5
#define N_LIST_LENGTH_SAMPLES 5
#define N_VECTOR_SAMPLES 10

#define MAX_VECTOR_ID std::numeric_limits<int>::max()
#define MAX_LIST_ID std::numeric_limits<int>::max()
#define MIN_VECTOR_VAL -3.40282e+38
#define MAX_VECTOR_VAL 3.40282e+38
```

Listing 9.15: include/tests/TestConfig.hpp

```
#include "../include/tests/StorageListsTestUtils.hpp"
#include <sys/mman.h>

namespace ann_dkvs
{

  size_t round_up_to_next_power_of_two(size_t value)
  {
    size_t power = 1;
    while (power < value)
    {
      power *= 2;
    }
```

```
14      return power;
15    }
16
17    size_t is_power_of_two(size_t value)
18    {
19      return (value & (value - 1)) == 0;
20    }
21
22    size_t get_vector_size(len_t vector_dim)
23    {
24      return vector_dim * sizeof(vector_el_t);
25    }
26
27    size_t get_list_size(len_t vector_dim, len_t n_entries)
28    {
29      len_t entries_allocated =
            round_up_to_next_power_of_two(n_entries);
30      entries_allocated =
            std::max((len_t)MIN_N_ENTRIES_PER_LIST,
            entries_allocated);
31      size_t list_size = entries_allocated *
            (get_vector_size(vector_dim) +
            sizeof(vector_id_t));
32      return list_size;
33    }
34
35    size_t get_total_size(size_t used_space)
36    {
37      size_t total_size =
            std::max((size_t)MIN_TOTAL_SIZE_BYTES,
            round_up_to_next_power_of_two(used_space));
38      return total_size;
39    }
40
41    void write_to_file(std::string filename, void *data,
          size_t size)
42    {
43      FILE *file = fopen(filename.c_str(), "w");
44      fwrite(data, size, 1, file);
45      fclose(file);
46    }
47
48    void read_from_file(std::string filename, void *data,
          size_t size)
49    {
50      FILE *file = fopen(filename.c_str(), "r");
51      size_t size_read = fread(data, size, 1, file);
52      if (size_read != size)
53      {
```

```
54        std::cout << "Error reading from file " << filename
              << std::endl;
55        exit(1);
56      }
57      fclose(file);
58    }
59
60    void *mmap_file(std::string filename, size_t size)
61    {
62      FILE *file = fopen(filename.c_str(), "r");
63      void *data = mmap(NULL, size, PROT_READ, MAP_PRIVATE,
            fileno(file), 0);
64      fclose(file);
65      return data;
66    }
67
68    bool file_exists(std::string filename)
69    {
70      std::ifstream f(filename.c_str());
71      if (!f.is_open())
72      {
73        return false;
74      }
75      f.close();
76      return true;
77    }
78
79    std::string join(const std::string &a, const
        std::string &b)
80    {
81      return a + "/" + b;
82    }
83
84    StorageLists get_inverted_lists_object(len_t vector_dim)
85    {
86      std::string file = join(TMP_DIR,
            get_lists_filename());
87      remove(file.c_str());
88      StorageLists lists(vector_dim, file);
89      return lists;
90    }
91
92    void print_vector(vector_el_t *vector, len_t
        vector_dim, len_t n_entries)
93    {
94      for (len_t i = 0; i < n_entries; i++)
95      {
96        std::cout << " | ";
97        for (len_t j = 0; j < vector_dim; j++)
```

```
98        {
99          std::cout << vector[i * vector_dim + j] << " ";
100       }
101     }
102     std::cout << std::endl;
103   }
104
105   void are_vectors_equal(const vector_el_t *actual, const
        vector_el_t *expected, len_t vector_dim, len_t
        n_entries)
106   {
107     for (len_t i = 0; i < n_entries * vector_dim; i++)
108     {
109       REQUIRE(actual[i] == expected[i]);
110     }
111   }
112
113   void are_ids_equal(const vector_id_t *actual, const
        vector_id_t *expected, len_t n_entries)
114   {
115     for (len_t i = 0; i < n_entries; i++)
116     {
117       REQUIRE(actual[i] == expected[i]);
118     }
119   }
120
121   std::string get_lists_filename()
122   {
123     std::string filename = LISTS_FILENAME;
124     std::string file_ext = FILE_EXT;
125     return filename + file_ext;
126   }
127
128   std::string get_vectors_filename(bool
        is_dataset_sorted, len_t n_lists)
129   {
130     std::string filename = VECTORS_FILENAME;
131     std::string file_ext = FILE_EXT;
132     std::string result = filename;
133     if (is_dataset_sorted)
134     {
135       result += "_" + std::to_string(n_lists) + "_sorted";
136     }
137     return result + file_ext;
138   }
139
140   std::string get_vector_ids_filename(bool
        is_dataset_sorted, len_t n_lists)
141   {
```

```
142     std::string filename = VECTOR_IDS_FILENAME;
143     std::string file_ext = FILE_EXT;
144     std::string result = filename;
145     if (is_dataset_sorted)
146     {
147       result += "_" + std::to_string(n_lists) + "_sorted";
148     }
149     return result + file_ext;
150   }
151
152   std::string get_list_ids_filename(len_t n_lists, bool
        is_dataset_sorted)
153   {
154     std::string filename = LIST_IDS_FILENAME;
155     std::string separator = "_";
156     std::string file_ext = FILE_EXT;
157     std::string result = filename + separator +
          std::to_string(n_lists);
158     if (is_dataset_sorted)
159     {
160       result += "_sorted";
161     }
162     return result + file_ext;
163   }
164
165   size_t get_file_size(std::string filename)
166   {
167     std::ifstream in(filename, std::ifstream::ate |
          std::ifstream::binary);
168     return in.tellg();
169   }
170
171   void setup_run_teardown_bulk_insert_entries_dataset(
172       len_t n_entries,
173       len_t vector_dim,
174       std::string vectors_filepath,
175       std::string vector_ids_filepath,
176       std::string list_ids_filepath,
177       std::function<void(len_t, size_t, vector_el_t *,
            vector_id_t *, list_id_t *, std::string,
            std::string, std::string)>
            run_bulk_insert_entries)
178   {
179     THEN("the vector, vector id and list id files are
          already present")
180     {
181       REQUIRE(file_exists(vectors_filepath));
182       REQUIRE(file_exists(vector_ids_filepath));
183       REQUIRE(file_exists(list_ids_filepath));
```

```
184
185       AND_GIVEN("the files are mapped to memory")
186       {
187         size_t vectors_size = n_entries * vector_dim *
              sizeof(vector_el_t);
188         size_t ids_size = n_entries * sizeof(list_id_t);
189         size_t list_ids_size = n_entries *
              sizeof(list_id_t);
190
191         vector_el_t *vectors = (vector_el_t
              *)mmap_file(vectors_filepath, vectors_size);
192         vector_id_t *ids = (vector_id_t
              *)mmap_file(vector_ids_filepath, ids_size);
193         list_id_t *list_ids = (list_id_t
              *)mmap_file(list_ids_filepath, list_ids_size);
194
195         run_bulk_insert_entries(n_entries, vector_dim,
              vectors, ids, list_ids, vectors_filepath,
              vector_ids_filepath, list_ids_filepath);
196
197         munmap(vectors, vectors_size);
198         munmap(ids, ids_size);
199         munmap(list_ids, list_ids_size);
200       }
201     }
202   }
203 }
```

Listing 9.16: tests/StorageListsTestUtils.cpp

9.4 Evaluation

- evaluation/baselines/faiss-ondisk/bench-faiss-ondisk-ivf.py
- evaluation/benchmarks.json
- evaluation/batch-bench.py

```
1  #!/usr/bin/env python3
2
3  import time
4  import numpy as np
5  import faiss
6  from faiss.contrib.ondisk import merge_ondisk
7  try:
8      from faiss.contrib.datasets_fb import DatasetSIFT1M,
           DatasetBigANN
9  except ImportError:
10     from faiss.contrib.datasets import DatasetSIFT1M,
           DatasetBigANN
```

```
11
12 # def load_dataset(cfg):
13 #     return
       cfg['dataset'].database_iterator(bs=cfg['batch_size'],
14 # split=(cfg['n_splits'], 0))
15
16
17 def main():
18     tmpdir = 'tmp/'
19     stage = 3
20
21     cfg = {}
22     cfg['dataset'] = DatasetSIFT1M()
23     cfg['total_dataset_size'] = cfg['dataset'].nb
24     cfg['dataset_size'] = 10**6
25     # cfg['batch_size'] = 10**5
26     # cfg['n_splits'] = cfg['total_dataset_size'] //
           cfg['dataset_size']
27     cfg['n_batches'] = 1
28     cfg['n_queries'] = 10**5
29     cfg['dimension'] = 128
30     cfg['n_lists'] = 1024
31
32     if stage == 0:
33         quantizer = faiss.IndexFlatL2(cfg['dimension'])
34         index = faiss.IndexIVFFlat(quantizer,
               cfg['dimension'], cfg['n_lists'])
35         print("training index")
36         xt = cfg['dataset'].get_train()
37         index.train(xt)
38         print("write " + tmpdir + "trained.index")
39         faiss.write_index(index, tmpdir + "trained.index")
40
41     if stage == 1:
42         xb = cfg['dataset'].get_database()
43         for batch_no in range(cfg['n_batches']):
44             index = faiss.read_index(tmpdir +
                   "trained.index")
45             index.add(xb)
46             faiss.write_index(index, tmpdir +
                   "block_%s.index" % batch_no)
47
48     if stage == 2:
49         print('loading trained index')
50         index = faiss.read_index(tmpdir + "trained.index")
51         block_fnames = [
52             tmpdir + "block_%d.index" % bno for bno in
                   range(cfg['n_batches'])
53         ]
```

```
54          merge_ondisk(index, block_fnames, tmpdir +
                "merged_index.ivfdata")
55          print("write " + tmpdir + "populated.index")
56          faiss.write_index(index, tmpdir +
                "populated.index")
57
58      if stage == 3:
59          print("read " + tmpdir + "populated.index")
60          index = faiss.read_index(tmpdir +
                "populated.index")
61          index.nprobe = 16
62
63          for pmode in [0, 1, 2, 0, 1, 2, 0, 1, 2]:
64              index.parallel_mode = 2
65              print("parallel_mode = %d" % pmode)
66              xq = cfg['dataset'].get_queries()
67
68              # measure throughput
69              num_runs = 3
70              runtimes = []
71              for i in range(num_runs):
72                  start_time = time.time()
73                  D, I = index.search(xq, 10)
74                  end_time = time.time()
75                  runtime = end_time - start_time
76                  runtimes.append(runtime)
77
78              # Compute the average runtime and queries per
                    second
79              avg_runtime = sum(runtimes) / num_runs
80              queries_per_sec = cfg['n_queries'] /
                    avg_runtime
81              print("Average runtime: %.3f seconds" %
                    avg_runtime)
82              print("Queries per second: %.3f" %
                    queries_per_sec)
83
84              # measure latency
85              latencies = []
86              n_queries = xq.shape[0]
87              for i in range(n_queries):
88                  start_time = time.time()
89                  D, I = index.search(xq[i:i + 1], 10)
90                  end_time = time.time()
91                  latency = end_time - start_time
92                  latency_ms = latency * 1000
93                  latencies.append(latency_ms)
94
95              # Compute the median and 95th percentile
```

```
                latency
96              latencies = np.array(latencies)
97              median_latency = np.median(latencies)
98              percentile_95th = np.percentile(latencies, 95)
99              print("Median latency: %.3f ms" %
                    median_latency)
100             print("95th percentile latency: %.3f ms" %
                    percentile_95th)
101             print("====================================")
102
103
104 if __name__ == "__main__":
105     main()
```

Listing 9.17: evaluation/baselines/faiss-ondisk/bench-faiss-ondisk-ivf.py

```
1  {
2    "defaults": {
3      "run": false,
4      "executable": "./out/testmain",
5      "make_command": "make clean && make test",
6      "DEBUG": "0",
7      "catch_tags":
           "[StorageIndex][search_preassigned][SIFT1M][benchmark][througput]"
8      "reporter": "xml",
9      "O": "3",
10     "DYNAMIC_INSERTION": "1",
11     "MAX_BUFFER_SIZE": "100",
12     "USE_SIMD": "1",
13     "USE_OMP": "1",
14     "PMODE": "2",
15     "MIN_TOTAL_SIZE_BYTES": "32",
16     "MIN_N_ENTRIES_PER_LIST": "1",
17     "env_OMP_NUM_THREADS": "32",
18     "env_OMP_SCHEDULE": "dynamic",
19     "TEST_N_SAMPLES": "3",
20     "TEST_N_THREADS": "32",
21     "TEST_N_LISTS": "1024",
22     "TEST_N_PROBES": "16",
23     "TEST_N_RESULTS": "10",
24     "TEST_N_DIMS": "128"
25   },
26   "benchmarks": [
27     {
28       "run": true,
29       "subdir": "03 Correctness",
30       "catch_tags":
             "[StorageIndex][search_preassigned][SIFT1M][test]",
31       "TEST_N_LISTS": "1024",
32       "USE_SIMD": "1",
```

```
33        "USE_OMP": "1",
34        "PMODE": "2",
35        "O": "3"
36      },
37      {
38        "run": true,
39        "subdir": "O3 Correctness",
40        "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT1M][test]~[sorted]",
41        "TEST_N_LISTS": "1024",
42        "USE_SIMD": "1",
43        "USE_OMP": "1",
44        "PMODE": "2",
45        "O": "3"
46      },
47      {
48        "run": false,
49        "subdir": "schedule and max number of threads",
50        "catch_tags":
            "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughput]",
51        "TEST_N_THREADS":
            "1,2,4,8,16,32,64,128,256,512,1024",
52        "env_OMP_SCHEDULE": "static"
53      },
54      {
55        "run": false,
56        "subdir": "schedule and max number of threads",
57        "catch_tags":
            "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]",
58        "TEST_N_THREADS":
            "1,2,4,8,16,32,64,128,256,512,1024",
59        "env_OMP_SCHEDULE": "static"
60      },
61      {
62        "run": false,
63        "subdir": "schedule and max number of threads",
64        "catch_tags":
            "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughput]",
65        "TEST_N_THREADS":
            "1,2,4,8,16,32,64,128,256,512,1024",
66        "env_OMP_SCHEDULE": "dynamic"
67      },
68      {
69        "run": false,
70        "subdir": "schedule and max number of threads",
71        "catch_tags":
            "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]",
72        "TEST_N_THREADS":
            "1,2,4,8,16,32,64,128,256,512,1024",
```

```
73        "env_OMP_SCHEDULE": "dynamic"
74      },
75      {
76        "run": false,
77        "subdir": "schedule and max number of threads",
78        "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughpu
79        "TEST_N_THREADS":
              "1,2,4,8,16,32,64,128,256,512,1024",
80        "env_OMP_SCHEDULE": "guided"
81      },
82      {
83        "run": false,
84        "subdir": "schedule and max number of threads",
85        "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]"
86        "TEST_N_THREADS":
              "1,2,4,8,16,32,64,128,256,512,1024",
87        "env_OMP_SCHEDULE": "guided"
88      },
89      {
90        "run": true,
91        "subdir": "OpenMP PMODE 2",
92        "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughpu
93        "TEST_N_LISTS": "256,512,1024,2048,4096",
94        "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
95        "USE_SIMD": "1",
96        "USE_OMP": "1",
97        "PMODE": "2"
98      },
99      {
100       "run": true,
101       "subdir": "OpenMP PMODE 2",
102       "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]"
103       "TEST_N_LISTS": "256,512,1024,2048,4096",
104       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
105       "USE_SIMD": "1",
106       "USE_OMP": "1",
107       "PMODE": "2"
108     },
109     {
110       "run": true,
111       "subdir": "OpenMP PMODE 1",
112       "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]"
113       "TEST_N_LISTS": "256,512,1024,2048,4096",
114       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
```

```
115       "USE_SIMD": "1",
116       "USE_OMP": "1",
117       "PMODE": "1"
118     },
119     {
120       "run": true,
121       "subdir": "OpenMP PMODE 1",
122       "catch_tags":
              "[RootIndex][preassign_query][SIFT1M][benchmark][latency]",
123       "TEST_N_LISTS": "256,512,1024,2048,4096",
124       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
125       "USE_SIMD": "1",
126       "USE_OMP": "1",
127       "PMODE": "1"
128     },
129     {
130       "run": true,
131       "subdir": "OpenMP PMODE 1",
132       "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughput]",
133       "TEST_N_LISTS": "256,512,1024,2048,4096",
134       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
135       "USE_SIMD": "1",
136       "USE_OMP": "1",
137       "PMODE": "1"
138     },
139     {
140       "run": true,
141       "subdir": "OpenMP PMODE 1",
142       "catch_tags":
              "[RootIndex][preassign_query][SIFT1M][benchmark][throughput]",
143       "TEST_N_LISTS": "256,512,1024,2048,4096",
144       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
145       "USE_SIMD": "1",
146       "USE_OMP": "1",
147       "PMODE": "1"
148     },
149     {
150       "run": true,
151       "subdir": "SIMD",
152       "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughput]",
153       "TEST_N_LISTS": "256,512,1024,2048,4096",
154       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
155       "USE_SIMD": "1",
156       "USE_OMP": "0",
157       "PMODE": "0"
158     },
159     {
```

```
160         "run": true,
161         "subdir": "SIMD",
162         "catch_tags":
                "[RootIndex][preassign_query][SIFT1M][benchmark][throughput]",
163         "TEST_N_LISTS": "256,512,1024,2048,4096",
164         "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
165         "USE_SIMD": "1",
166         "USE_OMP": "0",
167         "PMODE": "0"
168       },
169       {
170         "run": true,
171         "subdir": "SIMD",
172         "catch_tags":
                "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]"
173         "TEST_N_LISTS": "256,512,1024,2048,4096",
174         "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
175         "USE_SIMD": "1",
176         "USE_OMP": "0",
177         "PMODE": "0"
178       },
179       {
180         "run": true,
181         "subdir": "SIMD",
182         "catch_tags":
                "[RootIndex][preassign_query][SIFT1M][benchmark][latency]",
183         "TEST_N_LISTS": "256,512,1024,2048,4096",
184         "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
185         "USE_SIMD": "1",
186         "USE_OMP": "0",
187         "PMODE": "0"
188       },
189       {
190         "run": true,
191         "subdir": "Vanilla",
192         "catch_tags":
                "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughpu
193         "TEST_N_LISTS": "256,512,1024,2048,4096",
194         "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
195         "USE_SIMD": "0",
196         "USE_OMP": "0",
197         "PMODE": "0"
198       },
199       {
200         "run": true,
201         "subdir": "Vanilla",
202         "catch_tags":
                "[RootIndex][preassign_query][SIFT1M][benchmark][throughput]",
203         "TEST_N_LISTS": "256,512,1024,2048,4096",
```

```
204       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
205       "USE_SIMD": "0",
206       "USE_OMP": "0",
207       "PMODE": "0"
208     },
209     {
210       "run": true,
211       "subdir": "Vanilla",
212       "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]",
213       "TEST_N_LISTS": "256,512,1024,2048,4096",
214       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
215       "USE_SIMD": "0",
216       "USE_OMP": "0",
217       "PMODE": "0"
218     },
219     {
220       "run": true,
221       "subdir": "Vanilla",
222       "catch_tags":
              "[RootIndex][preassign_query][SIFT1M][benchmark][latency]",
223       "TEST_N_LISTS": "256,512,1024,2048,4096",
224       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
225       "USE_SIMD": "0",
226       "USE_OMP": "0",
227       "PMODE": "0"
228     },
229     {
230       "run": false,
231       "subdir": "MIN_N_ENTRIES_PER_LIST",
232       "DYNAMIC_INSERTION": "1",
233       "MIN_N_ENTRIES_PER_LIST": "1",
234       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
235     },
236     {
237       "run": false,
238       "subdir": "MIN_N_ENTRIES_PER_LIST",
239       "DYNAMIC_INSERTION": "1",
240       "MIN_N_ENTRIES_PER_LIST": "10",
241       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
242     },
243     {
244       "run": false,
245       "subdir": "MIN_N_ENTRIES_PER_LIST",
246       "DYNAMIC_INSERTION": "1",
247       "MIN_N_ENTRIES_PER_LIST": "100",
248       "catch_tags":
```

```
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "MIN_N_ENTRIES_PER_LIST",
      "DYNAMIC_INSERTION": "1",
      "MIN_N_ENTRIES_PER_LIST": "1000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "MIN_N_ENTRIES_PER_LIST",
      "DYNAMIC_INSERTION": "1",
      "MIN_N_ENTRIES_PER_LIST": "10000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "MIN_N_ENTRIES_PER_LIST",
      "DYNAMIC_INSERTION": "1",
      "MIN_N_ENTRIES_PER_LIST": "100000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "MIN_N_ENTRIES_PER_LIST",
      "DYNAMIC_INSERTION": "0",
      "MIN_N_ENTRIES_PER_LIST": "1",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "MIN_N_ENTRIES_PER_LIST",
      "DYNAMIC_INSERTION": "0",
      "MIN_N_ENTRIES_PER_LIST": "10",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "MIN_N_ENTRIES_PER_LIST",
      "DYNAMIC_INSERTION": "0",
      "MIN_N_ENTRIES_PER_LIST": "100",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
```

```
291     },
292     {
293       "run": false,
294       "subdir": "MIN_N_ENTRIES_PER_LIST",
295       "DYNAMIC_INSERTION": "0",
296       "MIN_N_ENTRIES_PER_LIST": "1000",
297       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
298     },
299     {
300       "run": false,
301       "subdir": "MIN_N_ENTRIES_PER_LIST",
302       "DYNAMIC_INSERTION": "0",
303       "MIN_N_ENTRIES_PER_LIST": "10000",
304       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
305     },
306     {
307       "run": false,
308       "subdir": "MIN_N_ENTRIES_PER_LIST",
309       "DYNAMIC_INSERTION": "0",
310       "MIN_N_ENTRIES_PER_LIST": "100000",
311       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
312     },
313     {
314       "run": false,
315       "subdir": "insertion strategy and buffer size",
316       "DYNAMIC_INSERTION": "1",
317       "MAX_BUFFER_SIZE": "1",
318       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
319     },
320     {
321       "run": false,
322       "subdir": "insertion strategy and buffer size",
323       "DYNAMIC_INSERTION": "1",
324       "MAX_BUFFER_SIZE": "10",
325       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
326     },
327     {
328       "run": false,
329       "subdir": "insertion strategy and buffer size",
330       "DYNAMIC_INSERTION": "1",
331       "MAX_BUFFER_SIZE": "100",
332       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
333     },
```

```
    {
      "run": false,
      "subdir": "insertion strategy and buffer size",
      "DYNAMIC_INSERTION": "1",
      "MAX_BUFFER_SIZE": "1000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "insertion strategy and buffer size",
      "DYNAMIC_INSERTION": "1",
      "MAX_BUFFER_SIZE": "10000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "insertion strategy and buffer size",
      "DYNAMIC_INSERTION": "1",
      "MAX_BUFFER_SIZE": "100000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "insertion strategy and buffer size",
      "DYNAMIC_INSERTION": "1",
      "MAX_BUFFER_SIZE": "1000000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "insertion strategy and buffer size",
      "DYNAMIC_INSERTION": "1",
      "MAX_BUFFER_SIZE": "10000000",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
      "run": false,
      "subdir": "insertion strategy and buffer size",
      "DYNAMIC_INSERTION": "0",
      "MAX_BUFFER_SIZE": "1",
      "catch_tags":
         "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
    },
    {
```

```
377       "run": false,
378       "subdir": "insertion strategy and buffer size",
379       "DYNAMIC_INSERTION": "0",
380       "MAX_BUFFER_SIZE": "1",
381       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
382     },
383     {
384       "run": false,
385       "subdir": "insertion strategy and buffer size",
386       "DYNAMIC_INSERTION": "0",
387       "MAX_BUFFER_SIZE": "10",
388       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
389     },
390     {
391       "run": false,
392       "subdir": "insertion strategy and buffer size",
393       "DYNAMIC_INSERTION": "0",
394       "MAX_BUFFER_SIZE": "100",
395       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
396     },
397     {
398       "run": false,
399       "subdir": "insertion strategy and buffer size",
400       "DYNAMIC_INSERTION": "0",
401       "MAX_BUFFER_SIZE": "1000",
402       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
403     },
404     {
405       "run": false,
406       "subdir": "insertion strategy and buffer size",
407       "DYNAMIC_INSERTION": "0",
408       "MAX_BUFFER_SIZE": "10000",
409       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
410     },
411     {
412       "run": false,
413       "subdir": "insertion strategy and buffer size",
414       "DYNAMIC_INSERTION": "0",
415       "MAX_BUFFER_SIZE": "100000",
416       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted]"
417     },
418     {
419       "run": false,
```

```
420         "subdir": "insertion strategy and buffer size",
421         "DYNAMIC_INSERTION": "0",
422         "MAX_BUFFER_SIZE": "1000000",
423         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
424       },
425       {
426         "run": false,
427         "subdir": "insertion strategy and buffer size",
428         "DYNAMIC_INSERTION": "0",
429         "MAX_BUFFER_SIZE": "10000000",
430         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
431       },
432       {
433         "run": false,
434         "subdir": "insertion strategy and buffer size",
435         "DYNAMIC_INSERTION": "1",
436         "MAX_BUFFER_SIZE": "1",
437         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
438       },
439       {
440         "run": false,
441         "subdir": "insertion strategy and buffer size",
442         "DYNAMIC_INSERTION": "1",
443         "MAX_BUFFER_SIZE": "10",
444         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
445       },
446       {
447         "run": false,
448         "subdir": "insertion strategy and buffer size",
449         "DYNAMIC_INSERTION": "1",
450         "MAX_BUFFER_SIZE": "100",
451         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
452       },
453       {
454         "run": false,
455         "subdir": "insertion strategy and buffer size",
456         "DYNAMIC_INSERTION": "1",
457         "MAX_BUFFER_SIZE": "1000",
458         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
459       },
460       {
461         "run": false,
462         "subdir": "insertion strategy and buffer size",
```

```
463       "DYNAMIC_INSERTION": "1",
464       "MAX_BUFFER_SIZE": "10000",
465       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorted]"
466     },
467     {
468       "run": false,
469       "subdir": "insertion strategy and buffer size",
470       "DYNAMIC_INSERTION": "1",
471       "MAX_BUFFER_SIZE": "100000",
472       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorted]"
473     },
474     {
475       "run": false,
476       "subdir": "insertion strategy and buffer size",
477       "DYNAMIC_INSERTION": "1",
478       "MAX_BUFFER_SIZE": "1000000",
479       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorted]"
480     },
481     {
482       "run": false,
483       "subdir": "insertion strategy and buffer size",
484       "DYNAMIC_INSERTION": "1",
485       "MAX_BUFFER_SIZE": "10000000",
486       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorted]"
487     },
488     {
489       "run": false,
490       "subdir": "insertion strategy and buffer size",
491       "DYNAMIC_INSERTION": "0",
492       "MAX_BUFFER_SIZE": "1",
493       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorted]"
494     },
495     {
496       "run": false,
497       "subdir": "insertion strategy and buffer size",
498       "DYNAMIC_INSERTION": "0",
499       "MAX_BUFFER_SIZE": "10",
500       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorted]"
501     },
502     {
503       "run": false,
504       "subdir": "insertion strategy and buffer size",
505       "DYNAMIC_INSERTION": "0",
```

```
506         "MAX_BUFFER_SIZE": "100",
507         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
508       },
509       {
510         "run": false,
511         "subdir": "insertion strategy and buffer size",
512         "DYNAMIC_INSERTION": "0",
513         "MAX_BUFFER_SIZE": "1000",
514         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
515       },
516       {
517         "run": false,
518         "subdir": "insertion strategy and buffer size",
519         "DYNAMIC_INSERTION": "0",
520         "MAX_BUFFER_SIZE": "10000",
521         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
522       },
523       {
524         "run": false,
525         "subdir": "insertion strategy and buffer size",
526         "DYNAMIC_INSERTION": "0",
527         "MAX_BUFFER_SIZE": "100000",
528         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
529       },
530       {
531         "run": false,
532         "subdir": "insertion strategy and buffer size",
533         "DYNAMIC_INSERTION": "0",
534         "MAX_BUFFER_SIZE": "1000000",
535         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
536       },
537       {
538         "run": false,
539         "subdir": "insertion strategy and buffer size",
540         "DYNAMIC_INSERTION": "0",
541         "MAX_BUFFER_SIZE": "10000000",
542         "catch_tags":
                "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
543       },
544       {
545         "run": true,
546         "subdir": "number of results",
547         "catch_tags":
                "[StorageIndex][search_preassigned][SIFT1M][benchmark][throughpu
```

```
548       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
549       "TEST_N_RESULTS": "1,2,4,8,16,32,64,128"
550     },
551     {
552       "run": true,
553       "subdir": "number of results",
554       "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][benchmark][latency]",
555       "TEST_N_PROBES": "1,2,4,8,16,32,64,128",
556       "TEST_N_RESULTS": "1,2,4,8,16,32,64,128"
557     },
558     {
559       "run": true,
560       "subdir": "recall",
561       "catch_tags":
              "[StorageIndex][search_preassigned][SIFT1M][recall]",
562       "TEST_N_PROBES":
              "1,2,4,8,16,32,64,128,256,512,1024",
563       "TEST_N_RESULTS": "1,2,4,8,16,32"
564     },
565     {
566       "run": true,
567       "subdir": "bulk inserting sorted data",
568       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT1M][benchmark][sorted]"
569     },
570     {
571       "run": true,
572       "subdir": "bulk inserting sorted data",
573       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT10M][benchmark][sorted]"
574     },
575     {
576       "run": true,
577       "subdir": "bulk inserting sorted data",
578       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT100M][benchmark][sorted]"
579     },
580     {
581       "run": true,
582       "subdir": "bulk insertion varying n_lists",
583       "catch_tags":
              "[StorageLists][bulk_insert_entries][SIFT1M][benchmark]~[sorted]",
584       "TEST_N_LISTS": "256,512,1024,2048,4096"
585     },
586     {
587       "run": true,
588       "subdir": "scaling bulk insertion",
589       "catch_tags":
```

```
            "[StorageLists][bulk_insert_entries][SIFT1M][benchmark]~[sorted]
590     },
591     {
592       "run": true,
593       "subdir": "scaling bulk insertion",
594       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT10M][benchmark]~[sorted
595     },
596     {
597       "run": true,
598       "subdir": "scaling bulk insertion",
599       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT100M][benchmark]~[sorte
600     },
601     {
602       "run": true,
603       "subdir": "scaling bulk insertion",
604       "catch_tags":
            "[StorageLists][bulk_insert_entries][SIFT1000M][benchmark]~[sort
605     }
606   ]
607 }
```

Listing 9.18: evaluation/benchmarks.json

```
1  import json
2  import os
3  from datetime import datetime
4  from tqdm import tqdm
5  import sys
6
7
8  def read_benchmarks():
9      with open(os.path.join('evaluation',
           'benchmarks.json')) as f:
10         benchmarks = json.load(f)
11     return benchmarks
12
13
14 def get_make_command(params):
15     cmd = params['make_command']
16     for key, value in params.items():
17         if key[0].isupper():
18             cmd += f" {key}={value}"
19     return cmd
20
21
22 def get_environment_variables(params):
23     envs = ''
24     for key, value in params.items():
```

```
25          if key[:4] == 'env_':
26              key = key[4:]
27              envs += f" {key}={value}"
28      return envs
29
30
31  def get_benchmark_command(params):
32      cmd = params['executable']
33      cmd += f" {params['catch_tags']}"
34      cmd += f" --benchmark-samples
            {params['TEST_N_SAMPLES']}"
35      cmd += f" -r {params['reporter']}"
36      return cmd
37
38
39  def get_filename(params):
40      EXCLUDE_KEYS = ['DEBUG']
41      directory = os.path.join('evaluation', 'benchmarks')
42      if params['subdir']:
43          directory = os.path.join(directory,
                params['subdir'])
44          os.makedirs(directory, exist_ok=True)
45      filename = f'{params["catch_tags"]} '
46      filename += ' '.join([
47          f"{key}={value}" for key, value in
                sorted(params.items())
48          if (key not in EXCLUDE_KEYS and key[0].isupper())
                or key[:4] == 'env_'
49      ])
50      filename += f'.{params["reporter"]}'
51      with open('evaluation/parameter_abbreviations.json')
            as f:
52          abbreviations = json.load(f)
53      for key, value in abbreviations.items():
54          filename = filename.replace(key, value)
55
56      return os.path.join(directory, filename)
57
58
59  if os.path.basename(os.getcwd()) == 'evaluation':
60      os.chdir('..')
61
62  benchmarks = read_benchmarks()
63
64  default_params = benchmarks['defaults']
65  benchmarks['benchmarks'] = [b for b in
        benchmarks['benchmarks'] if b['run']]
66
67  dry_run = False
```

```
68 if len(sys.argv) > 1:
69     dry_run = sys.argv[1] == 'dry'
70
71 for benchmark in tqdm(benchmarks['benchmarks']):
72     params = default_params.copy()
73     params.update(benchmark)
74     filename = get_filename(params)
75     if dry_run:
76         print('====================')
77     print(filename)
78
79     print()
80     make_cmd = get_make_command(params)
81     print(make_cmd)
82     if not dry_run:
83         os.system(make_cmd)
84
85     if not dry_run and not
           os.path.exists(params['executable']):
86         print(f"Executable {params['executable']} does
               not exist.")
87         exit(1)
88
89     print()
90     envs = get_environment_variables(params)
91     benchmark_cmd = get_benchmark_command(params)
92     if params['reporter'] == 'xml':
93         envs_benchmark_cmd = f'export{envs};
               {benchmark_cmd} >> "{filename}"'
94     elif params['reporter'] == 'console':
95         envs_benchmark_cmd = f'export{envs};
               {benchmark_cmd}'
96     else:
97         raise ValueError(f"Unknown reporter
               {params['reporter']}")
98     print(envs_benchmark_cmd)
99     if not dry_run:
100        os.system(envs_benchmark_cmd)
```

Listing 9.19: evaluation/batch-bench.py

9.5 Preprocessing

- cluster/cluster-dataset.py
- cluster/sort-input-files.py

```
1 #!/usr/bin/env python
2 import faiss
```

```
import numpy as np
from os.path import join, exists, getsize
from os import makedirs, rename, remove, chdir, getcwd
from faiss.contrib.ondisk import merge_ondisk
import argparse

try:
    from faiss.contrib.datasets_fb import DatasetSIFT1M,
       DatasetBigANN
except ImportError:
    from faiss.contrib.datasets import DatasetSIFT1M,
       DatasetBigANN

DATASET_LINK = "http://corpus-texmex.irisa.fr/"

#################################################################
# Preparing the data
#################################################################


def load_dataset(cfg):
    n_splits = cfg.total_dataset_size // cfg.dataset_size
    xb = cfg.dataset.database_iterator(bs=cfg.batch_size,
       split=(n_splits, 0))
    return xb


#################################################################
# Training the index
#################################################################


def get_trained_index(cfg):
    if exists(cfg.trained_index_file):
        print(f"\tLoading trained index from
           {cfg.trained_index_file}")
        index = faiss.read_index(cfg.trained_index_file)
    else:
        print("\tTraining index")
        xt = cfg.dataset.get_train()
        quantizer = faiss.IndexFlatL2(cfg.dimension)
        index = faiss.IndexIVFFlat(quantizer,
           cfg.dimension, cfg.n_lists)
        index.train(xt)
        print(f"\tWriting trained index to
           {cfg.trained_index_file}")
        faiss.write_index(index, cfg.trained_index_file)
    return index

```

```
46
47 ######################################################################
48 # Building the index
49 ######################################################################
50
51
52 def write_batch_index(cfg, batch, batch_no):
53     index = None
54     if exists(cfg.get_batch_index_file(batch_no)):
55         print(
56             f"\tIndex file
                {cfg.get_batch_index_file(batch_no)}
                already populated, skipping"
57         )
58     else:
59         index = get_trained_index(cfg)
60         print(f"\tAdding vectors to index")
61         index.add(batch)
62         print(f"\tWriting index to
               {cfg.get_batch_index_file(batch_no)}")
63         faiss.write_index(index,
               cfg.get_batch_index_file(batch_no))
64     return index
65
66
67 def write_batch_indices(cfg):
68     xb = load_dataset(cfg)
69     index_files = []
70     for batch_no, batch in enumerate(xb):
71         print(f"Building index for batch {batch_no + 1}
               of {cfg.n_batches}")
72         index_files.append(cfg.get_batch_index_file(batch_no))
73         write_vectors(cfg, batch, batch_no)
74         index = write_batch_index(cfg, batch, batch_no)
75         write_list_ids(cfg, index, batch_no)
76     assert getsize(cfg.vectors_file) ==
           cfg.get_expected_vectors_file_size(
77     ), "Vectors file does not have the expected size"
78     return index_files
79
80
81 def get_merged_index(cfg, index_files):
82     if exists(cfg.merged_index_file):
83         print(f"Loading merged index from
               {cfg.merged_index_file}")
84         return faiss.read_index(cfg.merged_index_file)
85     else:
86         print("Merging indices")
87         merged_index = get_trained_index(cfg)
```

```
88          print("\t", end="")
89          merge_ondisk(merged_index, index_files,
                cfg.merged_index_file)
90          faiss.write_index(merged_index,
                cfg.merged_index_file)
91          return merged_index
92
93
94  ##################################################################
95  # Extracting clusters from the index
96  ##################################################################
97
98
99  def get_vector_ids(index, list_id):
100     list_length = index.invlists.list_size(list_id)
101     vector_ids_ptr = index.invlists.get_ids(list_id)
102     vector_ids = faiss.rev_swig_ptr(vector_ids_ptr,
            list_length)
103     return vector_ids
104
105
106 def get_ids_map(cfg, index, batch_no):
107     ids_map = {}
108     start_id = batch_no * cfg.batch_size
109     for list_id in range(cfg.n_lists):
110         vector_ids = get_vector_ids(index, list_id)
111         for vector_id in vector_ids:
112             ids_map[start_id + vector_id] = list_id
113     return ids_map
114
115
116 def get_centroids(index):
117     return index.quantizer.reconstruct_n(0, index.nlist)
118
119
120 ##################################################################
121 # Writing test files to disk
122 ##################################################################
123
124
125 def write_vectors(cfg, batch, batch_no):
126     if getsize(cfg.vectors_file) >=
            cfg.get_expected_partial_vectors_file_size(
127             batch_no):
128         print(
129             f"\tVectors file {cfg.vectors_file} is
                    already populated, skipping"
130         )
131     else:
```

```
132         expected_size =
                cfg.get_expected_partial_vectors_file_size(batch_no
                -
133                                                                 1)
134         with open(cfg.vectors_file, "ab") as f:
135             if getsize(cfg.vectors_file) > expected_size:
136                 f.truncate(expected_size)
137             print(f"\tWriting vectors to
                    {cfg.vectors_file}")
138             batch.tofile(f)
139     assert getsize(
140         cfg.vectors_file) >=
                cfg.get_expected_partial_vectors_file_size(
141             batch_no), "Vectors file does not have the
                    expected size"
142
143
144 def write_vector_ids(cfg):
145     print(f"Writing vector ids to {cfg.vector_ids_file}")
146     if exists(cfg.vector_ids_file) and getsize(
147             cfg.vector_ids_file) ==
                cfg.get_expected_vector_ids_file_size():
148         print(f"\tVector ids file already exists,
                skipping")
149         return
150     ids = np.arange(cfg.dataset_size, dtype=np.int64)
151     with open(cfg.vector_ids_file, "wb") as f:
152         ids.tofile(f)
153     assert getsize(
154         cfg.vector_ids_file) ==
                cfg.get_expected_vector_ids_file_size(
155         ), "Vector ids file does not have the expected
                size"
156
157
158 def write_list_ids(cfg, index, batch_no):
159     if getsize(cfg.list_ids_file
160                ) >=
                    cfg.get_expected_partial_list_ids_file_size(batch_no):
161         print(
162             f"\tList ids file {cfg.list_ids_file} already
                    populated, skipping")
163     else:
164         if not index:
165             print(f"\tLoading index from
                    {cfg.get_batch_index_file(batch_no)}")
166             index =
                    faiss.read_index(cfg.get_batch_index_file(batch_no))
167         expected_size =
```

```
            cfg.get_expected_partial_list_ids_file_size(batch_no
            -
168                                                                 1)
169         with open(cfg.list_ids_file, "ab") as f:
170             if getsize(cfg.list_ids_file) > expected_size:
171                 f.truncate(expected_size)
172             start_id = batch_no * cfg.batch_size
173             end_id = start_id + cfg.batch_size
174             print(
175                 f"\tPopulating ids map for vector ids
                        {start_id} to {end_id}")
176             ids_map = get_ids_map(cfg, index, batch_no)
177             map_all_to_list_ids = np.vectorize(lambda i:
                    ids_map[i])
178             ids = np.arange(start=start_id, stop=end_id,
                    dtype=np.int64)
179             list_ids = map_all_to_list_ids(ids)
180             print(f"\tWriting list ids to
                    {cfg.list_ids_file}")
181             list_ids.tofile(f)
182     assert getsize(
183         cfg.list_ids_file) >=
                cfg.get_expected_partial_list_ids_file_size(
184             batch_no), "List ids file does not have the
                    expected size"
185
186
187 def write_centroids(cfg, index):
188     if exists(cfg.centroids_file):
189         print(f"\tCentroids file already exists,
                skipping")
190     else:
191         print(f"\tWriting centroids to
                {cfg.centroids_file}")
192         centroids = get_centroids(index)
193         with open(cfg.centroids_file, "wb") as f:
194             centroids.tofile(f)
195     assert getsize(cfg.centroids_file) ==
            cfg.get_expected_centroids_file_size(
196     ), "Centroids file does not have the expected size"
197
198
199 ##################################################################
200 # Main
201 ##################################################################
202
203
204 class Config:
205
```

```
206     def __init__(self, args):
207         self.dataset_size_millions = 1000 if
                args.dataset.lower().endswith(
208              "1b") else int(args.dataset[4:-1])
209         self.indices_dir = join(args.temp_dir,
210                                 f"SIFT{self.dataset_size_millions}M")
211         self.output_dir = join(args.output_dir,
212                                f"SIFT{self.dataset_size_millions}M")
213         self.indices_base = join(self.indices_dir,
214                                  f"SIFT{self.dataset_size_millions}M")
215         self.dataset_size = self.dataset_size_millions *
                10**6
216         self.n_lists = args.n_lists
217         self.trained_index_file = join(
218             args.temp_dir,
                    f"SIFT1000M_{args.n_lists}_trained.index")
219         self.merged_index_file =
                f"{self.indices_base}_{args.n_lists}_merged.index"
220         self.vectors_file = join(self.output_dir,
                f"{args.vectors_file}.bin")
221         self.vector_ids_file = join(self.output_dir,
222                                     f"{args.vector_ids_file}.bin")
223         self.list_ids_file = join(self.output_dir,
224                                   f"{args.list_ids_file}_{args.n_lists}.b
225         self.centroids_file = join(
226             self.output_dir,
                    f"{args.centroids_file}_{args.n_lists}.bin")
227         self.batch_size = min(args.batch_size,
                self.dataset_size)
228         self.n_batches = self.dataset_size //
                self.batch_size
229         self.reconstruct_centroids =
                args.reconstruct_centroids
230         self.user_confirmation()
231         self.prepare_dataset()
232
233     def user_confirmation(self):
234         print(f"Clustering with the following config:")
235         print(f"\tDataset:
                SIFT{self.dataset_size_millions}M")
236         print(f"\tNumber of clusters: {self.n_lists}")
237         print(f"\tBatch size: {self.batch_size}")
238         print(f"\tNumber of batches: {self.n_batches}")
239         print(
240             f"\tReconstruct centroids from index:
                    {self.reconstruct_centroids}"
241         )
242         print(f"\tTemporary directory:
                {self.indices_dir}")
```

```
243         print(f"\tOutput directory: {self.output_dir}")
244         if not input("Continue? [Y/N]
                ").lower().startswith("y"):
245             print("Aborting")
246             exit(1)
247 
248     def prepare_dataset(self):
249         try:
250             # Assumes that the dataset is in
                    data/{dataset}
251             # bvecs format:
252             # n * [[int] + d * [uint8]]
253             # where n is the number of vectors,
254             # d is the dimension (128),
255             # the int is the vector dimension
256             # and the uint8 is one vector component
257             self.dataset = DatasetBigANN()
258         except FileNotFoundError:
259             print(
260                 f"Could not find bigann dataset in
                        data/bigann, please download it first
                        from",
261                 DATASET_LINK)
262             exit(1)
263         self.dimension = self.dataset.d
264         self.total_dataset_size = self.dataset.nb
265 
266     def get_batch_index_file(self, batch_no):
267         return
               f"{self.indices_base}_{self.n_lists}_{batch_no
               + 1}_of_{self.n_batches}.index"
268 
269     def get_expected_partial_vectors_file_size(self,
           batch_no):
270         vector_size = self.dimension * 4
271         no_vectors = (batch_no + 1) * self.batch_size
272         return no_vectors * vector_size
273 
274     def get_expected_vectors_file_size(self):
275         return
               self.get_expected_partial_vectors_file_size(self.n_batches
               - 1)
276 
277     def get_expected_partial_list_ids_file_size(self,
           batch_no):
278         return (batch_no + 1) * self.batch_size * 8
279 
280     def get_expected_list_ids_file_size(self):
281         return
```

```
            self.get_expected_partial_list_ids_file_size(self.n_batches
            - 1)

    def get_expected_vector_ids_file_size(self):
        return self.dataset_size * 8

    def get_expected_centroids_file_size(self):
        return self.n_lists * self.dimension * 4


def cluster_dataset(cfg):
    makedirs(cfg.output_dir, exist_ok=True)
    makedirs(cfg.indices_dir, exist_ok=True)
    if not exists(cfg.vectors_file):
        open(cfg.vectors_file, "w").close()
    assert exists(cfg.vectors_file), "Vectors file does
       not exist"
    if not exists(cfg.list_ids_file):
        open(cfg.list_ids_file, "w").close()
    assert exists(cfg.list_ids_file), "List ids file does
       not exist"
    index_files = write_batch_indices(cfg)
    if cfg.reconstruct_centroids:
        if len(index_files) > 1:
            index = get_merged_index(cfg, index_files)
        else:
            print("Only one index file, skipping merge
               and loading last index")
            index = faiss.read_index(index_files[0])
        write_centroids(cfg, index)
    write_vector_ids(cfg)


if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        prog="SIFT Dataset Clustering",
        description=
        "Cluster subsets of SIFT dataset and output as
           lists of vectors, vector ids and list ids",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument(
        "--dataset",
        type=str,
        default="SIFT100M",
        help=
        "Dataset to cluster, one of SIFT1M, SIFT10M,
           SIFT100M, SIFT1000M/1B",
        choices=["SIFT1M", "SIFT10M", "SIFT100M",
           "SIFT1000M", "SIFT1B"])
```

```
323     parser.add_argument("--batch_size",
324                         type=int,
325                         default=10**7,
326                         help="Batch size")
327     parser.add_argument("--n_lists",
328                         type=int,
329                         default=2**10,
330                         help="Number of clusters")
331     parser.add_argument("--output_dir",
332                         type=str,
333                         default="./out",
334                         help="Directory to store output
                                files in")
335     parser.add_argument("--temp_dir",
336                         type=str,
337                         default="./tmp",
338                         help="Directory to store
                                temporary indices in")
339     parser.add_argument("--vectors_file",
340                         type=str,
341                         default="vectors",
342                         help="File to output vectors to")
343     parser.add_argument("--vector_ids_file",
344                         type=str,
345                         default="vector_ids",
346                         help="File to output vector ids
                                to")
347     parser.add_argument("--list_ids_file",
348                         type=str,
349                         default="list_ids",
350                         help="File to output list ids to")
351     parser.add_argument("--centroids_file",
352                         type=str,
353                         default="centroids",
354                         help="File to output centroids
                                to")
355     parser.add_argument("--reconstruct_centroids",
356                         action="store_true",
357                         help="Construct centroids from
                                built index")
358
359     args = parser.parse_args()
360
361     cfg = Config(args)
362     # make sure working directory is correct
363     if not getcwd().endswith("cluster"):
364         chdir("cluster")
365
366     cluster_dataset(cfg)
```

Listing 9.20: cluster/cluster-dataset.py

```
import argparse
import os
from os.path import join
from tqdm import tqdm

VECTOR_BYTES = 128 * 4
VECTOR_ID_BYTES = 8
LIST_ID_BYTES = 8


def sort(vectors_file, vector_ids_file, list_ids_file,
    vectors_file_sorted,
          vector_ids_file_sorted, list_ids_file_sorted,
              dataset_size):
    list_id_entry_id_pairs = []
    print("Reading list ids")
    with open(list_ids_file, "rb") as f:
        for _ in tqdm(range(dataset_size)):
            list_id = f.read(LIST_ID_BYTES)
            if not list_id:
                print("Unexpected end of file")
                exit(1)
            list_id_entry_id_pairs.append(
                (list_id, len(list_id_entry_id_pairs)))

    print("Sorting list ids")
    list_id_entry_id_pairs.sort()

    print(f"Writing sorted list ids to
        {list_ids_file_sorted}")
    if not os.path.isfile(list_ids_file_sorted):
        with open(list_ids_file_sorted, "wb") as f:
            for list_id, _ in
                tqdm(list_id_entry_id_pairs):
                f.write(list_id)
    else:
        print(f"\tFile {list_ids_file_sorted} already
            exists, skipping")
    assert (os.path.getsize(list_ids_file_sorted) ==
            get_expected_list_ids_file_size(dataset_size))

    print(f"Writing sorted vector ids to
        {vector_ids_file_sorted}")
    if not os.path.isfile(vector_ids_file_sorted):
        with open(vector_ids_file, "rb") as f_in,
            open(vector_ids_file_sorted,
```

```
40                                                          "wb")
                                                               as
                                                               f_out:
41              for _, entry_id in
                    tqdm(list_id_entry_id_pairs):
42                  f_in.seek(VECTOR_ID_BYTES * entry_id)
43                  vector_id = f_in.read(VECTOR_ID_BYTES)
44                  f_out.write(vector_id)
45      else:
46          print(f"\tFile {vector_ids_file_sorted} already
                exists, skipping")
47      assert (os.path.getsize(vector_ids_file_sorted) ==
48              get_expected_vector_ids_file_size(dataset_size))
49
50      print(f"Writing sorted vectors to
            {vectors_file_sorted}")
51      if not os.path.isfile(vectors_file_sorted):
52          with open(vectors_file, "rb") as f_in,
                open(vectors_file_sorted,
53                                                      "wb")
                                                           as
                                                           f_out:
54              for _, entry_id in
                    tqdm(list_id_entry_id_pairs):
55                  f_in.seek(VECTOR_BYTES * entry_id)
56                  vector = f_in.read(VECTOR_BYTES)
57                  f_out.write(vector)
58      else:
59          print(f"\tFile {vectors_file_sorted} already
                exists, skipping")
60      assert (os.path.getsize(vectors_file_sorted) ==
61              get_expected_vectors_file_size(dataset_size))
62
63
64  def get_expected_vector_ids_file_size(dataset_size):
65      return dataset_size * VECTOR_ID_BYTES
66
67
68  def get_expected_vectors_file_size(dataset_size):
69      return dataset_size * VECTOR_BYTES
70
71
72  def get_expected_list_ids_file_size(dataset_size):
73      return dataset_size * LIST_ID_BYTES
74
75
76  def user_confirmation(dataset, n_lists, vectors_file,
        vector_ids_file,
77                        list_ids_file, vectors_file_sorted,
```

```
78                          vector_ids_file_sorted,
                               list_ids_file_sorted, base_dir):
79      print(f"Sorting clustered dataset with the following
            config:")
80      print(f"\tDataset: {dataset}")
81      print(f"\tNumber of clusters: {n_lists}")
82      print(f"\tBase directory: {base_dir}")
83      print(f"Sorting the following files:")
84      print(f"\tVectors: {vectors_file}")
85      print(f"\tVector ids: {vector_ids_file}")
86      print(f"\tList ids: {list_ids_file}")
87      print(f"Output files:")
88      print(f"\tVectors: {vectors_file_sorted}")
89      print(f"\tVector ids: {vector_ids_file_sorted}")
90      print(f"\tList ids: {list_ids_file_sorted}")
91      if not input("Continue? [Y/N]
            ").lower().startswith("y"):
92          print("Aborting")
93          exit(1)
94
95
96  def get_dataset_size(dataset):
97      if dataset == "SIFT1M":
98          return 1_000_000
99      elif dataset == "SIFT10M":
100         return 10_000_000
101     elif dataset == "SIFT100M":
102         return 100_000_000
103     elif dataset == "SIFT1000M" or dataset == "SIFT1B":
104         return 1_000_000_000
105     else:
106         raise ValueError(f"Unknown dataset {dataset}")
107
108
109 if __name__ == "__main__":
110     parser = argparse.ArgumentParser(
111         prog="Sort clustered SIFT dataset",
112         description=
113         "Sort pre-clustered SIFT dataset by list ids to
               test potential performance improvements when
               used as input for bulk insertion"
114     )
115     parser.add_argument(
116         "--dataset",
117         type=str,
118         default="SIFT1M",
119         help=
120         "Dataset to cluster, one of SIFT1M, SIFT10M,
               SIFT100M, SIFT1000M/1B",
```

```
121         choices=["SIFT1M", "SIFT10M", "SIFT100M",
                "SIFT1000M", "SIFT1B"])
122     parser.add_argument("--n_lists",
123                         type=int,
124                         default=2**10,
125                         help="Number of clusters")
126     parser.add_argument("--vectors_file",
127                         type=str,
128                         default="vectors",
129                         help="File to output vectors to")
130     parser.add_argument("--vector_ids_file",
131                         type=str,
132                         default="vector_ids",
133                         help="File to output vector ids
                                to")
134     parser.add_argument("--list_ids_file",
135                         type=str,
136                         default="list_ids",
137                         help="File to output list ids to")
138     parser.add_argument(
139         "--output_dir",
140         type=str,
141         default="./out",
142         help=
143         "Directory where the clustered data is located
                and where to store output files in"
144     )
145     args = parser.parse_args()
146
147     base_dir = join(args.output_dir, args.dataset)
148     vectors_file = join(base_dir, args.vectors_file +
            ".bin")
149     vector_ids_file = join(base_dir, args.vector_ids_file
            + ".bin")
150     list_ids_file = join(base_dir,
            f"{args.list_ids_file}_{args.n_lists}.bin")
151
152     suffix = f"_{args.n_lists}_sorted.bin"
153     vectors_file_sorted = join(base_dir,
            args.vectors_file + suffix)
154     vector_ids_file_sorted = join(base_dir,
            args.vector_ids_file + suffix)
155     list_ids_file_sorted = join(base_dir,
            args.list_ids_file + suffix)
156
157     user_confirmation(args.dataset, args.n_lists,
            vectors_file,
158                       vector_ids_file, list_ids_file,
                             vectors_file_sorted,
```

```
159                         vector_ids_file_sorted,
                               list_ids_file_sorted, base_dir)
160     dataset_size = get_dataset_size(args.dataset)
161
162     # ensure input files exist
163     for file in [vectors_file, vector_ids_file,
            list_ids_file]:
164         if not os.path.isfile(file):
165             print(f"Input file {file} does not exist")
166             exit(1)
167
168     sort(vectors_file, vector_ids_file, list_ids_file,
            vectors_file_sorted,
169           vector_ids_file_sorted, list_ids_file_sorted,
              dataset_size)
```

Listing 9.21: cluster/sort-input-files.py

9.6 Prototype

- prototype/ann-prototype.py

```
1  import faiss
2  import numpy as np
3  import hnswlib
4  from faiss.contrib import ivf_tools
5  import os
6  import argparse
7
8  try:
9      from faiss.contrib.datasets_fb import DatasetSIFT1M
10 except ImportError:
11     from faiss.contrib.datasets import DatasetSIFT1M
12
13
14 def import_data():
15     # Assumes that the dataset is in data/sift1M
16     ds = DatasetSIFT1M()
17     xq = ds.get_queries()
18     xb = ds.get_database()
19     gt = ds.get_groundtruth()
20     xt = ds.get_train()
21     return xq, xb, gt, xt
22
23
24 def get_invlist(index, id):
25     invlists = index.invlists
26     list_len = invlists.list_size(id)
27     list_ids = invlists.get_ids(id)
```

```
28      list_vec_ids = faiss.rev_swig_ptr(list_ids, list_len)
29      return list_vec_ids
30
31
32  def get_centroid(index, id):
33      return index.quantizer.reconstruct(id)
34
35
36  def get_centroids(index):
37      return index.quantizer.reconstruct_n(0, index.nlist)
38
39
40  def build_IVFFlat(xb, xt, d, npartitions):
41      print("Clustering using IVFFlat")
42      quantizer = faiss.IndexFlatL2(d)
43      index = faiss.IndexIVFFlat(quantizer, d, npartitions)
44      index.train(xt)
45      index.add(xb)
46      centroids = get_centroids(index)
47      inverted_lists = []
48      for i in range(npartitions):
49          inverted_lists.append(get_invlist(index, i))
50      return index, centroids, inverted_lists
51
52
53  def verify_centroids(xb, centroids, inverted_lists):
54      npartitions = centroids.shape[0]
55      assert npartitions == len(inverted_lists)
56      closest_centroid = np.zeros(npartitions, dtype=int)
57
58      for i in range(npartitions):
59          list_vec_ids = inverted_lists[i]
60          list_vecs = xb[list_vec_ids]
61          clostest_centroid_id = -1
62          clostest_centroid_dist = np.inf
63          for j in range(npartitions):
64              dist = np.min(np.linalg.norm(list_vecs -
                    centroids[j], axis=1))
65              if dist < clostest_centroid_dist:
66                  clostest_centroid_dist = dist
67                  clostest_centroid_id = j
68          closest_centroid[i] = clostest_centroid_id
69
70      # check that closest centroid of list i is the same
            as the centroid i
71      assert np.all(np.equal(closest_centroid,
            np.arange(npartitions)))
72
73
```

```
74  def build_hnsw(centroids, ef, ef_construction, m):
75      print("Creating HNSW index and inserting centroids")
76      d = centroids.shape[1]
77      npartitions = centroids.shape[0]
78      hnsw = hnswlib.Index(space='l2', dim=d)
79      hnsw.init_index(max_elements=npartitions,
80                      ef_construction=ef_construction,
81                      M=m)
82      hnsw.add_items(centroids)
83      hnsw.set_ef(ef)
84      return hnsw
85
86
87  def query(ivfflat, hnsw, xq, k, nprobe):
88      print("Querying HNSW index to find nearest centroids")
89      nq, d = xq.shape
90      partition_I, _ = hnsw.knn_query(xq, k=nprobe)
91      # convert data type of partition_ids to int64 to
            match function signature of
            faiss::IndexIVF::search_preassigned
92      partition_I = partition_I.astype(np.int64)
93      ivfflat.nprobe = nprobe
94      print("Querying IVF Flat index, restricted to
            partitions selected by HNSW")
95      D, I = ivf_tools.search_preassigned(ivfflat, xq, k,
            partition_I)
96      return D, I
97
98
99  def get_recall(I, gt):
100     nq, k = I.shape
101     assert gt.shape[0] == nq and gt.shape[1] >= k
102     n_correct = np.sum(I == gt[:, :k])
103     recall = n_correct / float(nq * k)
104     return recall
105
106
107 def pipeline(args):
108     xq, xb, gt, xt = import_data()
109     nq, d = xq.shape
110     ivfflat, centroids, inverted_lists =
            build_IVFFlat(xb, xt, d,
111                                                    args.npartitions)
112     if VERIFY_CENTROIDS:
113         verify_centroids(xb, centroids, inverted_lists)
114     hnsw = build_hnsw(centroids, args.ef,
            args.ef_construction, args.m)
115     D, I = query(ivfflat, hnsw, xq, args.k, args.nprobe)
116     recall = get_recall(I, gt)
```

```
117     for arg_key, arg_value in vars(args).items():
118         print(arg_key, ":= ", arg_value)
119     print("recall := ", recall)
120
121
122 if __name__ == "__main__":
123     if os.path.basename(os.getcwd()) != "prototype":
124         print("please run this script from the prototype
                directory")
125         exit(1)
126     parser = argparse.ArgumentParser(
127         formatter_class=argparse.ArgumentDefaultsHelpFormatter)
128     parser.add_argument(
129         "--verify-centroids",
130         action="store_true",
131         default=False,
132         help=
133         "Verify that the centroids are the closest to the
                vectors in their inverted lists"
134     )
135     parser.add_argument(
136         "--npartitions",
137         type=int,
138         default=1024,
139     )
140     parser.add_argument("--ef",
141                         type=int,
142                         default=10,
143                         help="ef parameter for HNSW")
144     parser.add_argument("--ef-construction",
145                         type=int,
146                         default=100,
147                         help="ef_construction parameter
                                for HNSW")
148     parser.add_argument("--m",
149                         type=int,
150                         default=16,
151                         help="M parameter for HNSW")
152     parser.add_argument("--k",
153                         type=int,
154                         default=1,
155                         help="number of results to
                                return")
156     parser.add_argument("--nprobe",
157                         type=int,
158                         default=10,
159                         help="nprobe parameter for
                                IVFFlat")
160     args = parser.parse_args()
```

```
161     pipeline(args)
```

Listing 9.22: prototype/ann-prototype.py

Bibliography

[1] Leonardo Andrade and Mário Silva. Indexing structures for geographic web retrieval. 05 2012.

[2] Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[3] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. Spann: Highly-efficient billion-scale approximate nearest neighbor search, 2021.

[4] L. Dagum and R. Menon. Openmp: an industry standard api for shared-memory programming. *IEEE Computational Science and Engineering*, 5(1):46–55, 1998.

[5] Christina Delimitrou and Christos Kozyrakis. Amdahl's law for tail latency. *Communications of the ACM*, 61(8):65–72, 2018.

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[8] Steffen Heinz and Justin Zobel. Efficient single-pass index construction for text databases. *Journal of the American Society for Information Science and Technology*, 54(8):713–729, 2003.

[9] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in Neural Information Processing Systems*, 32, 2019.

[10] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. *Searching with quantization: approximate nearest neighbor search using short codes and distance estimators*. PhD thesis, INRIA, 2009.

[11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[12] Der-Tsai Lee. On k-nearest neighbor voronoi diagrams in the plane. *IEEE Transactions on Computers*, C-31:478–487, 1982.

[13] David G Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60:91–110, 2004.

[14] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[15] K. Mikolajczyk and C. Schmid. Indexing based on scale invariant interest points. In *Proceedings Eighth IEEE International Conference on Computer Vision. ICCV 2001*, volume 1, pages 525–531 vol.1, 2001.

[16] Daniel Molka. *Performance Analysis of Complex Shared Memory Systems*. PhD thesis, 03 2017.

[17] Phil Nash et al. Catch2. GitHub, 2019–2022. https://github.com/catchorg/Catch2/tree/v2.x.

[18] Rolf Rabenseifner. Introduction to openmp. https://www.hlrs.de/, Aug 2020.

[19] Jie Ren, Minjia Zhang, and Dong Li. Hm-ann: Efficient billion-point nearest neighbor search on heterogeneous memory. *Advances in Neural Information Processing Systems*, 33:10672–10684, 2020.

[20] David W Walker and Jack J Dongarra. Mpi: a standard message passing interface. *Supercomputer*, 12:56–68, 1996.